

Soundness of the GKR Protocol Assembly

Jinwook Kim

August 29, 2026

Abstract

The GKR protocol (Goldwasser, Kalai and Rothblum [2]) proves the correct evaluation of a layered arithmetic circuit by reducing each layer to a sumcheck instance and chaining the reductions down to the circuit inputs. This entry mechanizes the soundness of such an assembly over the existing AFP formalization of the sumcheck protocol by Garvia, Sprenger and Bootle [1].

The development covers: a layered circuit intermediate representation with a weighted gate family of algebraic degree at most three and its evaluation semantics; multilinear extensions of value tables over the boolean hypercube and the associated Lagrange weights; the per-layer sumcheck identity connecting a layer’s wiring predicates to the values of the layer below; the consumption of the AFP sumcheck soundness and completeness theorems for one layer reduction, via an explicit multivariate-polynomial representative of the layer integrand; a union-bound soundness theorem for the full layer-reduction chain, including the two-point claim carry between layers; the equality of a derived closed-form wiring oracle with the defining table oracle under repartition completeness; and the correctness of proof batching as a map over an abstract single-proof pipeline.

The protocol model is interactive and public-coin, with challenges drawn uniformly from a finite field, matching the AFP sumcheck formalization; the Fiat–Shamir transformation is outside the model. The chain soundness bound is $(s_{out} + \sum_i (2s_i d + 1))/|\mathbb{F}|$ for layer widths 2^{s_i} and a per-round degree bound d . Related mechanization efforts in other proof assistants include ArkLib [4] in Lean; an expository account of the GKR protocol and sumcheck-based proof systems is given by Thaler [3].

Contents

1	Gate kinds and their coefficient-free semantics	4
2	Gates, layers, circuits	4
3	Well-formedness	4
4	Native semantics	5
5	Acceptance convention (seal point S-4)	6
6	Sanity instance (non-vacuity witness)	6
7	Index bits (variable 0 = MSB)	7
8	The Lagrange basis weight (eq)	8

9	Boolean-point facts	8
9.1	Head/tail decomposition of the weight	9
9.2	Partition of unity	11
10	Multilinear extension of a value table	12
11	The all-zero indicator (beta)	13
12	Small sum and packing utilities	14
13	The Lagrange weight over a concatenated point	15
14	Delta tables and their MLEs	16
15	The gate-sum form of the wiring-predicate MLE (TableWiring)	17
16	The definitional table MLE (tuple table, duplicates summed)	18
17	The definitional MLE equals the gate sum (table oracle soundness)	20
18	The derived closed-form representation (DerivedRegularWiring)	23
18.1	Re-expansion (the audit helper <i>expand-gates</i> / <i>expand-consts</i>)	24
18.2	Well-formedness of a derived representation	25
18.3	Closed-form evaluation (the derived oracle)	26
19	Re-association: the closed form equals the expanded gate sum	27
20	Main theorem: exact repartition forces oracle equality (lemma (c))	31
21	Uniform-block closed form (the <i>shifted-range-eq</i> specification lemma)	33
22	Activation instance (the theorem fires on a concrete layer)	35
23	Per-kind wiring weight tables (gate-list semantics)	37
24	Splitting a layer's gate sum by kind	37
25	Claim-point lifting of the weight tables	40
26	The layer hypercube identity (B1)	42
27	Activation instance (non-vacuity witness for B1)	44
28	Index/substitution bridge to the AFP protocol (B2 plumbing)	45
29	The layer sumcheck on the AFP protocol (B2)	49
30	Non-vacuity: the AFP mpoly instance inhabits the bridge locale	52
31	Boolean factor polynomials	52

32 Homomorphism helpers	53
33 The interpolation table polynomial	53
34 Hypercube evaluation: the interpolation property	54
35 Total-degree bounds for the interpolation construction	56
36 Discharging the representation premise (non-vacuity, bridge to the concrete verifier)	59
37 The chain recurrence of witness value vectors	60
38 Honest acceptance at a chain position (B1 corollary, via B2)	60
39 Two-point carry combination (B3 building block)	61
40 Multilinear Schwartz-Zippel over point tuples (B3 building block)	62
41 Tuple-space splitting tools (B3 probability plumbing)	67
42 The layer-reduction chain event (B3)	70
43 The chain union bound (B3)	71
44 Abstract single-proof pipeline	82
44.1 Structural lemmas (cache harmlessness, formally)	82
44.2 Interface lemmas for batch verification	83
44.3 Theorem D - batching correctness	83
45 Parameterised batch pipe - the re-instantiation surface	84
46 Activation instance ($N = 2$, accept/reject mixed)	85
46.1 Re-instantiation demo (the v0.2 path)	86
47 The amortized batch pipe (v0.2) - shared prepared state	86
47.1 Activation instance (v0.2 pipes on the mixed batch)	87

```

theory Layered-Circuit
  imports Main
begin

```

1 Gate kinds and their coefficient-free semantics

```

datatype gate-kind = GLin | GMul | GPow3

```

Coefficient-free kind semantics; the wiring weight is applied by the layer evaluation below (mirroring the Rust split where *gate-semantics* is coefficient-free and the evaluator multiplies the gate coefficient). *GLin* and *GPow3* are unary: they ignore the second argument, and well-formed circuits fix $in2 = in1$ by the frozen convention.

```

definition gate-sem :: gate-kind  $\Rightarrow$  'f::comm-ring-1  $\Rightarrow$  'f  $\Rightarrow$  'f where
  gate-sem k a b = (case k of GLin  $\Rightarrow$  a | GMul  $\Rightarrow$  a * b | GPow3  $\Rightarrow$  a * a * a)

```

```

lemma gate-sem-simps [simp]:
  gate-sem GLin a b = a
  gate-sem GMul a b = a * b
  gate-sem GPow3 a b = a * a * a
by (simp-all add: gate-sem-def)

```

2 Gates, layers, circuits

```

record 'f gate =
  g-kind  :: gate-kind
  g-out   :: nat
  g-in1   :: nat
  g-in2   :: nat
  g-coeff :: 'f

```

```

record 'f layer =
  layer-width-bits :: nat
  layer-gates      :: 'f gate list
  layer-consts     :: (nat  $\times$  'f) list

```

```

record 'f layered-circuit =
  circ-layers    :: 'f layer list — output layer first, input-adjacent last
  input-width-bits :: nat

```

```

definition layer-width :: 'f layer  $\Rightarrow$  nat where
  layer-width L = 2 ^ layer-width-bits L

```

3 Well-formedness

wf-layer bw L : every gate writes inside the layer and reads inside the layer below (width bw), unary kinds satisfy the $in2 = in1$ convention (Rust: *validate-unary-gates*), and constants target in-range wires.

```

definition wf-gate :: nat  $\Rightarrow$  nat  $\Rightarrow$  'f gate  $\Rightarrow$  bool where

```

$wf\text{-}gate\ w\ bw\ g \longleftrightarrow$
 $g\text{-}out\ g < w \wedge g\text{-}in1\ g < bw \wedge g\text{-}in2\ g < bw \wedge$
 $(g\text{-}kind\ g = GLin \vee g\text{-}kind\ g = GPow3 \longrightarrow g\text{-}in2\ g = g\text{-}in1\ g)$

definition $wf\text{-}layer :: nat \Rightarrow 'f\ layer \Rightarrow bool$ **where**
 $wf\text{-}layer\ bw\ L \longleftrightarrow$
 $(\forall g \in set\ (layer\text{-}gates\ L). wf\text{-}gate\ (layer\text{-}width\ L)\ bw\ g) \wedge$
 $(\forall (w, c) \in set\ (layer\text{-}consts\ L). w < layer\text{-}width\ L)$

Chain well-formedness: layer i reads from layer $i+1$'s width, and the deepest layer reads from the input vector.

fun $wf\text{-}chain :: 'f\ layer\ list \Rightarrow nat \Rightarrow bool$ **where**
 $wf\text{-}chain\ []\ iw \longleftrightarrow True$
 $| wf\text{-}chain\ [L]\ iw \longleftrightarrow wf\text{-}layer\ (2 \wedge iw)\ L$
 $| wf\text{-}chain\ (L \# L' \# Ls)\ iw \longleftrightarrow wf\text{-}layer\ (layer\text{-}width\ L')\ L \wedge wf\text{-}chain\ (L' \# Ls)\ iw$

definition $wf\text{-}circuit :: 'f\ layered\text{-}circuit \Rightarrow bool$ **where**
 $wf\text{-}circuit\ C \longleftrightarrow wf\text{-}chain\ (circ\text{-}layers\ C)\ (input\text{-}width\text{-}bits\ C)$

4 Native semantics

Additive constant of output wire z : sparse (wire, value) list, entries for the same wire accumulate (list semantics, mirroring the Rust $+=$ initialisation loop).

definition $const\text{-}at :: 'f::comm\text{-}monoid\text{-}add\ layer \Rightarrow nat \Rightarrow 'f$ **where**
 $const\text{-}at\ L\ z = sum\text{-}list\ (map\ snd\ (filter\ (\lambda(w, c). w = z)\ (layer\text{-}consts\ L)))$

Contribution of one gate to its output wire, given the layer below.

definition $gate\text{-}contrib :: 'f::comm\text{-}ring\text{-}1\ gate \Rightarrow 'f\ list \Rightarrow 'f$ **where**
 $gate\text{-}contrib\ g\ below = g\text{-}coeff\ g * gate\text{-}sem\ (g\text{-}kind\ g)\ (below\ !\ g\text{-}in1\ g)\ (below\ !\ g\text{-}in2\ g)$

One layer's value vector from the layer below. For each output wire: the accumulated constant plus the sum of contributions of the gates writing to it (list sum - duplicates accumulate).

definition $layer\text{-}eval :: 'f::comm\text{-}ring\text{-}1\ layer \Rightarrow 'f\ list \Rightarrow 'f\ list$ **where**
 $layer\text{-}eval\ L\ below =$
 $map\ (\lambda z. const\text{-}at\ L\ z +$
 $sum\text{-}list\ (map\ (\lambda g. gate\text{-}contrib\ g\ below)$
 $(filter\ (\lambda g. g\text{-}out\ g = z)\ (layer\text{-}gates\ L))))$
 $[0 ..< layer\text{-}width\ L]$

lemma $length\text{-}layer\text{-}eval\ [simp]: length\ (layer\text{-}eval\ L\ below) = layer\text{-}width\ L$
by $(simp\ add: layer\text{-}eval\text{-}def)$

lemma $layer\text{-}eval\text{-}nth:$
assumes $z < layer\text{-}width\ L$
shows $layer\text{-}eval\ L\ below\ !\ z =$
 $const\text{-}at\ L\ z +$
 $sum\text{-}list\ (map\ (\lambda g. gate\text{-}contrib\ g\ below)$
 $(filter\ (\lambda g. g\text{-}out\ g = z)\ (layer\text{-}gates\ L))))$
using $assms$ **by** $(simp\ add: layer\text{-}eval\text{-}def)$

All layer value vectors of one execution: output layer first, the input vector last (Rust: *CircuitWitness::layer-values* as produced by *evaluate-circuit*).

```
fun circuit-values :: 'f::comm-ring-1 layer list  $\Rightarrow$  'f list  $\Rightarrow$  'f list list where
  circuit-values [] inputs = [inputs]
| circuit-values (L # Ls) inputs =
  (let rest = circuit-values Ls inputs in layer-eval L (hd rest) # rest)
```

```
lemma circuit-values-nonempty: circuit-values Ls inputs  $\neq$  []
by (cases Ls) (auto simp: Let-def)
```

```
lemma length-circuit-values [simp]:
  length (circuit-values Ls inputs) = length Ls + 1
by (induction Ls) (auto simp: Let-def)
```

The output vector, and its foldr characterisation (spec form).

```
definition circuit-output :: 'f::comm-ring-1 layered-circuit  $\Rightarrow$  'f list  $\Rightarrow$  'f list where
  circuit-output C inputs = hd (circuit-values (circ-layers C) inputs)
```

```
lemma circuit-output-foldr:
  circuit-output C inputs = foldr layer-eval (circ-layers C) inputs
proof –
  have hd (circuit-values Ls inputs) = foldr layer-eval Ls inputs for Ls :: 'a layer list
  by (induction Ls) (auto simp: Let-def)
  then show ?thesis by (simp add: circuit-output-def)
qed
```

5 Acceptance convention (seal point S-4)

A circuit accepts an input vector iff EVERY output-layer wire evaluates to zero. Compiled circuits encode their checks as residuals that vanish exactly when the constraint holds.

```
definition circuit-accept :: 'f::comm-ring-1 layered-circuit  $\Rightarrow$  'f list  $\Rightarrow$  bool where
  circuit-accept C inputs  $\longleftrightarrow$  ( $\forall v \in \text{set } (\text{circuit-output } C \text{ inputs}). v = 0$ )
```

6 Sanity instance (non-vacuity witness)

A tiny concrete circuit exercising all three gate kinds and a constant: one layer of width 2 over an input vector of width 2,

out0 = 3*a + b*b*b - 5 (Lin + Pow3 + const) out1 = a*b (Mul)

evaluated over the integers at (a, b) = (2, 1): out0 = 6 + 1 - 5 = 2, out1 = 2. This pins the evaluator against off-by-one / orientation regressions and witnesses that the semantics is inhabited.

```
definition demo-layer :: int layer where
  demo-layer =
    [ layer-width-bits = 1,
      layer-gates =
        [ [ g-kind = GLin, g-out = 0, g-in1 = 0, g-in2 = 0, g-coeff = 3 ],
          [ g-kind = GPow3, g-out = 0, g-in1 = 1, g-in2 = 1, g-coeff = 1 ],
          [ g-kind = GMul, g-out = 1, g-in1 = 0, g-in2 = 1, g-coeff = 1 ] ],
```

$layer-consts = [(0, - 5)] \ \rangle$

definition *demo-circuit* :: *int layered-circuit* **where**

demo-circuit = $\langle \ \text{circ-layers} = [\text{demo-layer}], \text{input-width-bits} = 1 \ \rangle$

lemma *demo-wf*: *wf-circuit demo-circuit*

by (*simp add*: *wf-circuit-def demo-circuit-def demo-layer-def wf-layer-def wf-gate-def layer-width-def*)

lemma *demo-eval*: *circuit-output demo-circuit* $[2, 1] = [2, 2]$

by (*simp add*: *circuit-output-def demo-circuit-def demo-layer-def Let-def layer-eval-def layer-width-def const-at-def gate-contrib-def upt-rec*)

lemma *demo-not-accepting*: $\neg \text{circuit-accept demo-circuit } [2, 1]$

by (*simp add*: *circuit-accept-def demo-eval*)

Acceptance is witnessed with a residual-style circuit: one Lin gate copying the input minus a constant, vanishing iff the input equals it (the shape compiled constraint circuits use, S-4).

definition *residual-circuit* :: *int layered-circuit* **where**

residual-circuit =
 $\langle \ \text{circ-layers} =$
 $\quad [\ \langle \ \text{layer-width-bits} = 0,$
 $\quad \quad \text{layer-gates} = [\ \langle \ g\text{-kind} = \text{GLin}, g\text{-out} = 0, g\text{-in1} = 0, g\text{-in2} = 0, g\text{-coeff} = 1 \ \rangle],$
 $\quad \quad \text{layer-consts} = [(0, - 7)] \ \rangle],$
 $\quad \text{input-width-bits} = 0 \ \rangle$

lemma *residual-accepts-exactly*:

circuit-accept residual-circuit $[x] \longleftrightarrow x = 7$

by (*simp add*: *circuit-accept-def circuit-output-def residual-circuit-def Let-def layer-eval-def layer-width-def const-at-def gate-contrib-def*)

end

theory *Multilinear-Extension*

imports *Main*

begin

7 Index bits (variable 0 = MSB)

bit-at n idx j: the bit carried by variable j of an n -variable index, i.e. bit $n - 1 - j$ of *idx* (Rust: $(idx \gg (n-1-j)) \ \& \ 1$).

definition *bit-at* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *bool* **where**

bit-at n idx j $\longleftrightarrow \text{bit idx } (n - 1 - j)$

lemma *bit-at-iff-odd-div*: *bit-at n idx j* $\longleftrightarrow \text{odd } (idx \text{ div } 2 \wedge (n - 1 - j))$

by (*simp add*: *bit-at-def bit-iff-odd*)

The boolean point of an index: entry j is bit j (MSB first).

definition *bool-point* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *'f::comm-ring-1 list* **where**

$bool_point\ n\ idx = map\ (\lambda j. \text{if } bit_at\ n\ idx\ j \text{ then } 1 \text{ else } 0)\ [0 ..< n]$

lemma *length-bool-point* [simp]: $length\ (bool_point\ n\ idx) = n$
by (simp add: bool-point-def)

lemma *bool-point-nth*:
 $j < n \implies (bool_point\ n\ idx :: 'f::comm-ring-1\ list) ! j =$
 $(\text{if } bit_at\ n\ idx\ j \text{ then } 1 \text{ else } 0)$
by (simp add: bool-point-def)

8 The Lagrange basis weight (eq)

eq-pi point idx: product over variables of *point!**j* where bit *j* of *idx* is one, and $1 - point!j$ where it is zero (Rust: *eq-point-index*).

definition *eq-pi* :: $'f::comm-ring-1\ list \Rightarrow nat \Rightarrow 'f$ **where**
 $eq_pi\ pt\ idx =$
 $(\prod j < length\ pt. \text{if } bit_at\ (length\ pt)\ idx\ j \text{ then } pt\ !\ j \text{ else } 1 - pt\ !\ j)$

lemma *eq-pi-nil* [simp]: $eq_pi\ []\ idx = 1$
by (simp add: eq-pi-def)

First-element shift for products over an initial segment.

lemma *prod-lessThan-Suc-shift*:
fixes $f :: nat \Rightarrow 'f::comm-monoid-mult$
shows $(\prod j < Suc\ n. f\ j) = f\ 0 * (\prod j < n. f\ (Suc\ j))$
by (induction n) (simp-all add: mult.assoc)

9 Boolean-point facts

An index below $2^{\wedge}n$ is determined by its n variable bits.

lemma *bit-at-complete*:
assumes $idx < 2^{\wedge}n$ $idx' < 2^{\wedge}n$
and $\bigwedge j. j < n \implies bit_at\ n\ idx\ j = bit_at\ n\ idx'\ j$
shows $idx = idx'$
proof (rule bit-eqI)
fix $i :: nat$
show $bit\ idx\ i = bit\ idx'\ i$
proof (cases $i < n$)
case *True*
then have $bit_at\ n\ idx\ (n - 1 - i) = bit_at\ n\ idx'\ (n - 1 - i)$
using *assms*(3) **by** *simp*
with *True* **show** ?thesis **by** (simp add: bit-at-def)
next
case *False*
have *high*: $\neg bit\ m\ i$ **if** $m < 2^{\wedge}n$ **for** $m :: nat$
proof -
have $(2 :: nat)^{\wedge}n \leq 2^{\wedge}i$
using *False* **by** (simp add: power-increasing)
then have $m < 2^{\wedge}i$ **using** *that* **by** *linarith*
then show $\neg bit\ m\ i$

```

      by (simp add: bit-iff-odd div-less)
    qed
  show ?thesis using high assms(1,2) by simp
qed
qed

```

On boolean points, *eq-pi* is the Kronecker delta (Rust test *eq-is-kronecker-on-boolean-points*).

```

lemma eq-pi-bool-point:
  assumes  $idx < 2^n$   $idx' < 2^n$ 
  shows  $eq\_pi (bool\_point\ n\ idx :: 'f::comm-ring-1\ list)\ idx' =$ 
     $(if\ idx = idx' then\ 1\ else\ 0)$ 
proof (cases  $idx = idx'$ )
  case True
  have  $(if\ bit\_at\ n\ idx'\ j\ then\ (bool\_point\ n\ idx :: 'f\ list)\ !\ j$ 
     $else\ 1 - bool\_point\ n\ idx\ !\ j) = 1$  if  $j < n$  for  $j$ 
    using that True by (simp add: bool-point-nth)
  then show ?thesis
    unfolding eq-pi-def by (simp add: True)
next
  case False
  then obtain  $j$  where  $jn: j < n$  and  $neg: bit\_at\ n\ idx\ j \neq bit\_at\ n\ idx'\ j$ 
    using bit-at-complete assms by blast
  have zero-factor:
     $(if\ bit\_at\ n\ idx'\ j\ then\ (bool\_point\ n\ idx :: 'f\ list)\ !\ j$ 
       $else\ 1 - bool\_point\ n\ idx\ !\ j) = 0$ 
    using neg jn by (simp add: bool-point-nth split: if-splits)
  have  $eq\_pi (bool\_point\ n\ idx :: 'f\ list)\ idx' =$ 
     $(\prod_{k < n} if\ bit\_at\ n\ idx'\ k\ then\ (bool\_point\ n\ idx :: 'f\ list)\ !\ k$ 
       $else\ 1 - bool\_point\ n\ idx\ !\ k)$ 
    by (simp add: eq-pi-def)
  also have  $\dots = 0$ 
proof (rule prod-zero)
  show finite  $\{.. < n\}$  by simp
  show  $\exists a \in \{.. < n\}. (if\ bit\_at\ n\ idx'\ a\ then\ (bool\_point\ n\ idx :: 'f\ list)\ !\ a$ 
     $else\ 1 - bool\_point\ n\ idx\ !\ a) = 0$ 
    using jn zero-factor by (intro bexI[of - j]) auto
qed
finally show ?thesis using False by simp
qed

```

9.1 Head/tail decomposition of the weight

Splitting one variable off the front: the head variable carries the top bit, the tail variables carry the low bits. These two lemmas drive every induction over the point list.

```

lemma bit-at-head-lo:  $idx < 2^n \implies \neg bit\_at (Suc\ n)\ idx\ 0$ 
  by (simp add: bit-at-def bit-iff-odd div-less)

```

```

lemma bit-at-head-hi:  $idx < 2^n \implies bit\_at (Suc\ n)\ (idx + 2^n)\ 0$ 
  by (simp add: bit-at-def bit-iff-odd div-add-self2 div-less)

```

```

lemma bit-at-tail-lo:
  assumes  $j < n$ 

```

shows $\text{bit-at } (\text{Suc } n) \text{ idx } (\text{Suc } j) = \text{bit-at } n \text{ idx } j$
using *assms* **by** (*simp add: bit-at-def*)

lemma *bit-at-tail-hi*:

assumes $\text{idx} < 2^n$ $j < n$

shows $\text{bit-at } (\text{Suc } n) (\text{idx} + 2^n) (\text{Suc } j) = \text{bit-at } n \text{ idx } j$

proof –

have *i-lt*: $n - 1 - j < n$ **using** *assms*(2) **by** *simp*

have $\text{bit } (\text{idx} + 2^n) (n - 1 - j) = \text{bit } \text{idx } (n - 1 - j)$

proof –

have *dvd*: $(2 :: \text{nat})^n \text{ dvd } 2^{n-1-j}$

using *i-lt* **by** (*simp add: le-imp-power-dvd*)

have $(\text{idx} + 2^n) \text{ div } 2^{n-1-j} =$

$\text{idx div } 2^{n-1-j} + 2^n \text{ div } 2^{n-1-j}$

using *dvd* **by** (*simp add: div-plus-div-distrib-dvd-right*)

moreover **have** $(2 :: \text{nat})^n \text{ div } 2^{n-1-j} = 2^{n-(n-1-j)}$

proof –

have *exp-eq*: $(n - 1 - j) + (n - (n - 1 - j)) = n$

using *i-lt* **by** *arith*

have $(2 :: \text{nat})^n = 2^{n-1-j} * 2^{n-(n-1-j)}$

by (*metis exp-eq power-add*)

then show *?thesis* **by** *simp*

qed

moreover **have** *even* $((2 :: \text{nat})^{n-(n-1-j)})$

using *i-lt* **by** *simp*

ultimately show *?thesis*

by (*simp add: bit-iff-odd*)

qed

then show *?thesis*

using *assms*(2) **by** (*simp add: bit-at-def*)

qed

lemma *eq-pi-cons-lo*:

fixes $p :: 'f::\text{comm-ring-1}$

assumes $\text{idx} < 2^{\text{length } ps}$

shows $\text{eq-pi } (p \# ps) \text{ idx} = (1 - p) * \text{eq-pi } ps \text{ idx}$

proof –

let $?n = \text{length } ps$

have $\text{eq-pi } (p \# ps) \text{ idx} =$

$(\prod j < \text{Suc } ?n. \text{ if bit-at } (\text{Suc } ?n) \text{ idx } j \text{ then } (p \# ps) ! j \text{ else } 1 - (p \# ps) ! j)$

by (*simp add: eq-pi-def*)

also **have** $\dots = (\text{if bit-at } (\text{Suc } ?n) \text{ idx } 0 \text{ then } p \text{ else } 1 - p) *$

$(\prod j < ?n. \text{ if bit-at } (\text{Suc } ?n) \text{ idx } (\text{Suc } j) \text{ then } ps ! j \text{ else } 1 - ps ! j)$

by (*simp only: prod-lessThan-Suc-shift nth-Cons-0 nth-Cons-Suc*)

also **have** $\dots = (1 - p) *$

$(\prod j < ?n. \text{ if bit-at } ?n \text{ idx } j \text{ then } ps ! j \text{ else } 1 - ps ! j)$

proof –

have *head*: $(\text{if bit-at } (\text{Suc } ?n) \text{ idx } 0 \text{ then } p \text{ else } 1 - p) = 1 - p$

using *bit-at-head-lo*[*OF assms*] **by** *simp*

have *tail*: $(\prod j < ?n. \text{ if bit-at } (\text{Suc } ?n) \text{ idx } (\text{Suc } j) \text{ then } ps ! j \text{ else } 1 - ps ! j) =$

$(\prod j < ?n. \text{ if bit-at } ?n \text{ idx } j \text{ then } ps ! j \text{ else } 1 - ps ! j)$

by (*intro prod.cong refl*) (*simp add: bit-at-tail-lo*)

show *?thesis* **by** (*simp only: head tail*)

```

qed
finally show ?thesis by (simp add: eq-pi-def)
qed

lemma eq-pi-cons-hi:
  fixes p :: 'f::comm-ring-1
  assumes idx < 2 ^ length ps
  shows eq-pi (p # ps) (idx + 2 ^ length ps) = p * eq-pi ps idx
proof -
  let ?n = length ps
  have eq-pi (p # ps) (idx + 2 ^ ?n) =
    (∏ j < Suc ?n. if bit-at (Suc ?n) (idx + 2 ^ ?n) j
      then (p # ps) ! j else 1 - (p # ps) ! j)
  by (simp add: eq-pi-def)
  also have ... = (if bit-at (Suc ?n) (idx + 2 ^ ?n) 0 then p else 1 - p) *
    (∏ j < ?n. if bit-at (Suc ?n) (idx + 2 ^ ?n) (Suc j) then ps ! j else 1 - ps ! j)
  by (simp only: prod-lessThan-Suc-shift nth-Cons-0 nth-Cons-Suc)
  also have ... = p * (∏ j < ?n. if bit-at ?n idx j then ps ! j else 1 - ps ! j)
proof -
  have head: (if bit-at (Suc ?n) (idx + 2 ^ ?n) 0 then p else 1 - p) = p
  using bit-at-head-hi[OF assms] by simp
  have tail: (∏ j < ?n. if bit-at (Suc ?n) (idx + 2 ^ ?n) (Suc j)
    then ps ! j else 1 - ps ! j) =
    (∏ j < ?n. if bit-at ?n idx j then ps ! j else 1 - ps ! j)
  by (intro prod.cong refl) (simp add: bit-at-tail-hi[OF assms])
  show ?thesis by (simp only: head tail)
qed
finally show ?thesis by (simp add: eq-pi-def)
qed

```

9.2 Partition of unity

The Lagrange weights sum to one at every point (the *sum-y beta* lifting in the layer sumcheck rests on this).

```

lemma sum-eq-pi-one:
  fixes pt :: 'f::comm-ring-1 list
  shows (∑ idx < 2 ^ length pt. eq-pi pt idx) = 1
proof (induction pt)
  case Nil
  then show ?case by simp
next
  case (Cons p ps)
  let ?n = length ps
  have split: {.. $(2 :: nat) \wedge \text{Suc } ?n\} = \{.. $2 \wedge ?n\} \cup (\lambda i. i + 2 \wedge ?n) \cdot \{.. $2 \wedge ?n\}$ 
  proof
    show {.. $(2 :: nat) \wedge \text{Suc } ?n\} \subseteq \{.. $2 \wedge ?n\} \cup (\lambda i. i + 2 \wedge ?n) \cdot \{.. $2 \wedge ?n\}$ 
    proof
      fix x assume x ∈ {.. $(2 :: nat) \wedge \text{Suc } ?n\}$ 
      then have x < 2 * 2 ^ ?n by simp
      then show x ∈ {.. $2 \wedge ?n\} \cup (\lambda i. i + 2 \wedge ?n) \cdot \{.. $2 \wedge ?n\}$ 
      proof (cases x < 2 ^ ?n)
        case False$$$$$ 
```

```

    then have  $x - 2^{\text{?}n} < 2^{\text{?}n}$  and  $x = (x - 2^{\text{?}n}) + 2^{\text{?}n}$ 
      using  $\langle x < 2 * 2^{\text{?}n} \rangle$  by simp-all
    then show ?thesis by force
  qed simp
qed
next
show  $\{.. < 2^{\text{?}n}\} \cup (\lambda i. i + 2^{\text{?}n}) \cdot \{.. < 2^{\text{?}n}\} \subseteq \{.. < (2 :: \text{nat})^{\wedge} \text{Suc } \text{?}n\}$ 
  by auto
qed
have inj: inj-on  $(\lambda i. i + (2 :: \text{nat})^{\wedge} \text{?}n) \cdot \{.. < 2^{\text{?}n}\}$ 
  by (simp add: inj-on-def)
have disj:  $\{.. < (2 :: \text{nat})^{\wedge} \text{?}n\} \cap (\lambda i. i + 2^{\text{?}n}) \cdot \{.. < 2^{\text{?}n}\} = \{\}$ 
  by auto
have  $(\sum \text{idx} < 2^{\wedge} \text{length } (p \# ps). \text{eq-pi } (p \# ps) \text{ idx}) =$ 
   $(\sum \text{idx} \in \{.. < (2 :: \text{nat})^{\wedge} \text{Suc } \text{?}n\}. \text{eq-pi } (p \# ps) \text{ idx})$ 
  by simp
also have  $\dots = (\sum \text{idx} \in \{.. < 2^{\wedge} \text{?}n\} \cup (\lambda i. i + 2^{\text{?}n}) \cdot \{.. < 2^{\wedge} \text{?}n\}. \text{eq-pi } (p \# ps) \text{ idx})$ 
  by (simp only: split)
also have  $\dots = (\sum \text{idx} < 2^{\wedge} \text{?}n. \text{eq-pi } (p \# ps) \text{ idx}) +$ 
   $(\sum \text{idx} < 2^{\wedge} \text{?}n. \text{eq-pi } (p \# ps) (\text{idx} + 2^{\text{?}n}))$ 
  by (simp add: sum.union-disjoint disj inj sum.reindex)
also have  $\dots = (1 - p) * (\sum \text{idx} < 2^{\wedge} \text{?}n. \text{eq-pi } ps \text{ idx}) +$ 
   $p * (\sum \text{idx} < 2^{\wedge} \text{?}n. \text{eq-pi } ps \text{ idx})$ 
  by (simp add: eq-pi-cons-lo eq-pi-cons-hi sum-distrib-left)
also have  $\dots = 1$  using Cons.IH by (simp add: algebra-simps)
finally show ?case .
qed

```

10 Multilinear extension of a value table

mle n f pt: the multilinear extension of the table *f* (indexed by $\{0 \dots 2^{\wedge} n\}$) evaluated at *pt*. Shape pinned by the Rust test *mle-eval-as-eq-combination*: $\text{MLE}(v)(r) = \text{sum over idx of } v[\text{idx}] * \text{eq}(r, \text{idx})$.

definition $\text{mle} :: \text{nat} \Rightarrow (\text{nat} \Rightarrow 'f) \Rightarrow 'f :: \text{comm-ring-1 list} \Rightarrow 'f$ **where**
 $\text{mle } n \text{ f pt} = (\sum \text{idx} < 2^{\wedge} n. \text{eq-pi } pt \text{ idx} * f \text{ idx})$

On boolean points the MLE agrees with the table (Rust test *mle-eval-matches-table-on-hypercube*).

lemma *mle-bool-point*:

assumes $\text{idx} < 2^{\wedge} n$

shows $\text{mle } n \text{ f } (\text{bool-point } n \text{ idx} :: 'f :: \text{comm-ring-1 list}) = f \text{ idx}$

proof –

have $\text{mle } n \text{ f } (\text{bool-point } n \text{ idx} :: 'f \text{ list}) =$

$(\sum \text{idx}' < 2^{\wedge} n. (\text{if } \text{idx} = \text{idx}' \text{ then } 1 \text{ else } 0) * f \text{ idx}')$

unfolding *mle-def*

proof (*intro sum.cong refl*)

fix $\text{idx}' :: \text{nat}$ **assume** $\text{idx}' \in \{.. < 2^{\wedge} n\}$

then show $\text{eq-pi } (\text{bool-point } n \text{ idx} :: 'f \text{ list}) \text{ idx}' * f \text{ idx}' =$

$(\text{if } \text{idx} = \text{idx}' \text{ then } 1 \text{ else } 0) * f \text{ idx}'$

by (*simp add: eq-pi-bool-point assms*)

qed

also have $\dots = (\sum \text{idx}' < 2^{\wedge} n. \text{if } \text{idx} = \text{idx}' \text{ then } f \text{ idx}' \text{ else } 0)$

```

    by (intro sum.cong refl) simp
  also have ... = f idx
    using assms by (simp add: sum.delta')
  finally show ?thesis .
qed

```

Linearity in the table - the workhorse for wiring-MLE equalities (multiset repartition arguments, lemma (c)).

```

lemma mle-add:
  mle n (λi. f i + g i) pt = mle n f pt + mle n g pt
  by (simp add: mle-def sum.distrib algebra-simps)

```

```

lemma mle-cong:
  assumes ⋀i. i < 2 ^ n ⇒ f i = g i
  shows mle n f pt = mle n g pt
  unfolding mle-def by (intro sum.cong refl) (use assms in simp)

```

```

lemma mle-zero: mle n (λ-. 0) pt = 0
  by (simp add: mle-def)

```

```

lemma mle-scale:
  mle n (λi. c * f i) pt = c * mle n f pt
  by (simp add: mle-def sum-distrib-left algebra-simps)

```

11 The all-zero indicator (beta)

$\text{beta } pt = \text{eq-pi } pt \ 0 = \text{product of } 1 - pt!j \text{ (Rust: beta-eval).}$

```

definition beta :: 'f::comm-ring-1 list ⇒ 'f where
  beta pt = eq-pi pt 0

```

```

lemma beta-prod: beta pt = (∏ j < length pt. 1 - pt ! j)
proof -
  have ¬ bit-at (length pt) 0 j for j
    by (simp add: bit-at-def)
  then show ?thesis by (simp add: beta-def eq-pi-def)
qed

```

Boolean instance: beta is 1 exactly at the all-zero point, and the weights over the hypercube sum to 1 (the unary-term lifting identity).

```

lemma beta-bool-point:
  assumes idx < 2 ^ n
  shows beta (bool-point n idx :: 'f::comm-ring-1 list) = (if idx = 0 then 1 else 0)
proof -
  have (0 :: nat) < 2 ^ n by simp
  then show ?thesis
    using eq-pi-bool-point[OF assms, of 0] by (simp add: beta-def)
qed

```

```

lemma sum-beta-bool-points:
  (∑ idx < (2 :: nat) ^ n. beta (bool-point n idx :: 'f::comm-ring-1 list)) = 1
proof -

```

```

have (∑ idx < (2 :: nat) ^ n. beta (bool-point n idx :: 'f list)) =
  (∑ idx < (2 :: nat) ^ n. if idx = 0 then (1 :: 'f) else 0)
  by (intro sum.cong refl) (simp add: beta-bool-point)
also have ... = 1 by simp
finally show ?thesis .
qed

end

```

```

theory Wiring-MLE
imports
  Layered-Circuit
  Multilinear-Extension
  HOL-Library.Multiset
begin

```

12 Small sum and packing utilities

```

lemma sum-list-map-upt:
  fixes f :: nat ⇒ 'f::comm-monoid-add
  shows sum-list (map f [0 ..< n]) = (∑ b < n. f b)
  by (induction n) simp-all

lemma sum-list-concat-map:
  fixes f :: 'a ⇒ 'f::comm-monoid-add
  shows sum-list (map f (concat xss)) = sum-list (map (λxs. sum-list (map f xs)) xss)
  by (induction xss) simp-all

lemma sum-list-concat:
  fixes xss :: 'f::comm-monoid-add list list
  shows sum-list (concat xss) = sum-list (map sum-list xss)
  by (induction xss) simp-all

```

Packing a pair of bounded indices into one index is injective and stays bounded - the arithmetic backbone of the tuple encodings below.

```

lemma pack-lt:
  fixes a A b B :: nat
  assumes a < A and b < B
  shows a * B + b < A * B
proof -
  have a * B + b < (a + 1) * B using assms(2) by simp
  also have ... ≤ A * B using assms(1) by (intro mult-le-mono1) simp
  finally show ?thesis .
qed

```

```

lemma pack-inj:
  fixes a1 b1 a2 b2 B :: nat
  assumes b1 < B and b2 < B
  shows a1 * B + b1 = a2 * B + b2 ⟷ a1 = a2 ∧ b1 = b2
proof
  assume eq: a1 * B + b1 = a2 * B + b2

```

```

have b1 = (a1 * B + b1) mod B using assms(1) by simp
also have ... = b2 using eq assms(2) by simp
finally have b: b1 = b2 .
moreover have a1 = a2
proof -
  have a1 = (a1 * B + b1) div B using assms(1) by simp
  also have ... = a2 using eq assms(2) by simp
  finally show ?thesis .
qed
ultimately show a1 = a2 ∧ b1 = b2 by simp
qed simp

```

13 The Lagrange weight over a concatenated point

The split identity: over a concatenated point the *eq-pi* weight of a packed index factors into the weights of the two halves. This is the formal counterpart of the Rust comment on ‘DerivedRegularWiring’: “eq(z, ((A + b) « lb) + off) factors EXACTLY into eq(z-hi, A + b) * eq(z-lo, off) when offsets stay below the stride”.

```

lemma eq-pi-append:
  fixes p q :: 'f::comm-ring-1 list
  assumes i < 2 ^ length p and j < 2 ^ length q
  shows eq-pi (p @ q) (i * 2 ^ length q + j) = eq-pi p i * eq-pi q j
  using assms(1)
proof (induction p arbitrary: i)
  case Nil
  then have i = 0 by simp
  then show ?case by simp
next
  case (Cons a p)
  have jq: j < 2 ^ length q by (rule assms(2))
  show ?case
  proof (cases i < 2 ^ length p)
    case True
    have lt: i * 2 ^ length q + j < 2 ^ (length p + length q)
    proof -
      have i * 2 ^ length q + j < 2 ^ length p * 2 ^ length q
      using True jq by (rule pack-lt)
      then show ?thesis by (simp add: power-add)
    qed
    have eq-pi ((a # p) @ q) (i * 2 ^ length q + j)
      = eq-pi (a # (p @ q)) (i * 2 ^ length q + j) by simp
    also have ... = (1 - a) * eq-pi (p @ q) (i * 2 ^ length q + j)
      using eq-pi-cons-lo[of i * 2 ^ length q + j p @ q a] lt by simp
    also have ... = (1 - a) * (eq-pi p i * eq-pi q j)
      using Cons.IH[OF True] by simp
    also have ... = eq-pi (a # p) i * eq-pi q j
      using eq-pi-cons-lo[of i p a] True by (simp add: mult.assoc)
    finally show ?thesis .
  next
    case False
    case False
    define i' where i' = i - 2 ^ length p

```

```

have ilt:  $i' < 2^{\text{length } p}$ 
  using Cons.premis False by (simp add: i'-def)
have idec:  $i = i' + 2^{\text{length } p}$ 
  using False by (simp add: i'-def)
have lt:  $i' * 2^{\text{length } q} + j < 2^{(\text{length } p + \text{length } q)}$ 
proof -
  have  $i' * 2^{\text{length } q} + j < 2^{\text{length } p} * 2^{\text{length } q}$ 
    using ilt jQ by (rule pack-lt)
  then show ?thesis by (simp add: power-add)
qed
have shift:  $i * 2^{\text{length } q} + j = (i' * 2^{\text{length } q} + j) + 2^{(\text{length } p + \text{length } q)}$ 
  using idec by (simp add: power-add algebra-simps)
have eq-pi ((a # p) @ q) (i * 2^length q + j)
  = eq-pi (a # (p @ q)) ((i' * 2^length q + j) + 2^length (p @ q))
  by (simp add: shift)
also have ... = a * eq-pi (p @ q) (i' * 2^length q + j)
  using eq-pi-cons-hi[of i' * 2^length q + j p @ q a] lt by simp
also have ... = a * (eq-pi p i' * eq-pi q j)
  using Cons.IH[OF ilt] by simp
also have ... = eq-pi (a # p) i * eq-pi q j
  using eq-pi-cons-hi[of i' p a] ilt idec by (simp add: mult.assoc)
finally show ?thesis .
qed
qed

```

14 Delta tables and their MLEs

```

lemma mle-delta:
  fixes c :: 'f::comm-ring-1
  assumes  $e < 2^n$ 
  shows mle n ( $\lambda idx. \text{if } idx = e \text{ then } c \text{ else } 0$ ) pt = eq-pi pt e * c
proof -
  have mle n ( $\lambda idx. \text{if } idx = e \text{ then } c \text{ else } 0$ ) pt
    = ( $\sum idx < 2^n. \text{if } idx = e \text{ then } eq\text{-pi } pt \ e * c \text{ else } 0$ )
    unfolding mle-def by (intro sum.cong refl) auto
  also have ... = eq-pi pt e * c
    using assms by simp
  finally show ?thesis .
qed

lemma mle-delta':
  fixes c :: 'f::comm-ring-1
  assumes  $e < 2^n$ 
  shows mle n ( $\lambda idx. \text{if } e = idx \text{ then } c \text{ else } 0$ ) pt = eq-pi pt e * c
proof -
  have ( $\lambda idx. \text{if } e = idx \text{ then } c \text{ else } 0$ ) = ( $\lambda idx. \text{if } idx = e \text{ then } c \text{ else } 0$ )
    by (intro ext) auto
  then show ?thesis using mle-delta[OF assms] by simp
qed

```

15 The gate-sum form of the wiring-predicate MLE (TableWiring)

Per-gate contribution at an evaluation point, literally the loop body of *TableWiring::eval-predicate-mle*: coefficient times the output-wire and first-input-wire weights, times the second-input weight for the binary kind only (unary kinds live on (z, x) ; their $in2 = in1$ convention is irrelevant here because the factor is not consumed).

definition *wiring-term* :: *gate-kind* $\Rightarrow$ '*f::comm-ring-1 list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f gate* $\Rightarrow$ '*f* **where**
wiring-term *k z x y g* =
 $g\text{-coeff } g * eq\text{-pi } z (g\text{-out } g) * eq\text{-pi } x (g\text{-in1 } g) *$
 $(if\ k = GMul\ then\ eq\text{-pi } y (g\text{-in2 } g)\ else\ 1)$

definition *gates-mle-eval* :: '*f::comm-ring-1 gate list* $\Rightarrow$ *gate-kind* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f* **where**
gates-mle-eval *gs k z x y* =
 $sum\text{-list } (map\ (wiring\text{-term } k\ z\ x\ y)\ (filter\ (\lambda g. g\text{-kind } g = k)\ gs))$

lemma *gates-mle-eval-Nil* [*simp*]: *gates-mle-eval* [] *k z x y* = 0
by (*simp add: gates-mle-eval-def*)

lemma *gates-mle-eval-Cons*:
gates-mle-eval (*g # gs*) *k z x y* =
 $(if\ g\text{-kind } g = k\ then\ wiring\text{-term } k\ z\ x\ y\ g\ else\ 0) + gates\text{-mle-eval } gs\ k\ z\ x\ y$
by (*simp add: gates-mle-eval-def*)

lemma *gates-mle-eval-append*:
gates-mle-eval (*gs @ hs*) *k z x y* = *gates-mle-eval* *gs k z x y* + *gates-mle-eval* *hs k z x y*
by (*simp add: gates-mle-eval-def*)

The gate sum only depends on the MULTISSET of gates: summation is invariant under repartition/permutation of the list (list semantics preserved - duplicated gates keep contributing twice on both sides).

lemma *gates-mle-eval-mset-cong*:
assumes *mset gs* = *mset gs'*
shows *gates-mle-eval* *gs k z x y* = *gates-mle-eval* *gs' k z x y*
proof –
have *mset* (*map* (*wiring-term k z x y*) (*filter* ($\lambda g. g\text{-kind } g = k$) *gs*))
 $= mset\ (map\ (wiring\text{-term } k\ z\ x\ y)\ (filter\ (\lambda g. g\text{-kind } g = k)\ gs'))$
using *assms* **by** (*simp add: mset-filter mset-map*)
then show ?thesis
unfolding *gates-mle-eval-def* **by** (*metis sum-mset-sum-list*)
qed

Same shape for the additive-constant vector (*TableWiring::eval-const-mle*).

definition *consts-mle-eval* :: (*nat* $\times$ '*f::comm-ring-1*) *list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f* **where**
consts-mle-eval *cs z* = $sum\text{-list } (map\ (\lambda(w, v). v * eq\text{-pi } z\ w)\ cs)$

lemma *consts-mle-eval-Nil* [*simp*]: *consts-mle-eval* [] *z* = 0
by (*simp add: consts-mle-eval-def*)

lemma *consts-mle-eval-Cons*:
consts-mle-eval ((*w, v*) # *cs*) *z* = $v * eq\text{-pi } z\ w + consts\text{-mle-eval } cs\ z$

by (simp add: consts-mle-eval-def)

lemma *consts-mle-eval-append*:

consts-mle-eval (cs @ ds) z = *consts-mle-eval* cs z + *consts-mle-eval* ds z

by (simp add: consts-mle-eval-def)

lemma *consts-mle-eval-mset-cong*:

assumes mset cs = mset cs'

shows *consts-mle-eval* cs z = *consts-mle-eval* cs' z

proof –

have mset (map ($\lambda(w, v). v * eq_pi\ z\ w$) cs) = mset (map ($\lambda(w, v). v * eq_pi\ z\ w$) cs')

using *assms* **by** (simp add: mset-map)

then show ?thesis

unfolding *consts-mle-eval-def* **by** (metis sum-mset-sum-list)

qed

16 The definitional table MLE (tuple table, duplicates summed)

The weighted wiring function on index tuples: at (z, x, y) it is the SUM of the coefficients of all gates of the given kind wired exactly to that tuple (duplicated tuples accumulate), and 0 elsewhere. The binary kind lives on (z, x, y) triples, the unary kinds on (z, x) pairs. Tuples are packed into one index (MSB-first, matching the variable order of concatenated points), and the definitional MLE is the *mle* of that table.

definition *enc3* :: nat $\Rightarrow$ 'f gate $\Rightarrow$ nat **where**

enc3 ib g = (g-out g * 2^{ib} + g-in1 g) * 2^{ib} + g-in2 g

definition *enc2* :: nat $\Rightarrow$ 'f gate $\Rightarrow$ nat **where**

enc2 ib g = g-out g * 2^{ib} + g-in1 g

definition *table3* :: nat $\Rightarrow$ 'f::comm-ring-1 gate list $\Rightarrow$ nat $\Rightarrow$ 'f **where**

table3 ib gs idx = sum-list (map g-coeff (filter ($\lambda g. enc3\ ib\ g = idx$) gs))

definition *table2* :: nat $\Rightarrow$ 'f::comm-ring-1 gate list $\Rightarrow$ nat $\Rightarrow$ 'f **where**

table2 ib gs idx = sum-list (map g-coeff (filter ($\lambda g. enc2\ ib\ g = idx$) gs))

lemma *table3-Nil* [simp]: *table3* ib [] idx = 0

by (simp add: table3-def)

lemma *table3-Cons*:

table3 ib (g # gs) idx = (if *enc3* ib g = idx then g-coeff g else 0) + *table3* ib gs idx

by (simp add: table3-def)

lemma *table2-Nil* [simp]: *table2* ib [] idx = 0

by (simp add: table2-def)

lemma *table2-Cons*:

table2 ib (g # gs) idx = (if *enc2* ib g = idx then g-coeff g else 0) + *table2* ib gs idx

by (simp add: table2-def)

The packed table read at a packed in-range tuple is exactly the tuple sum: the coefficients of the gates wired to that tuple.

lemma *table3-at-tuple*:

assumes $x0: x0 < 2 \wedge ib$ **and** $y0: y0 < 2 \wedge ib$
and $wf: \forall g \in \text{set } gs. g\text{-in1 } g < 2 \wedge ib \wedge g\text{-in2 } g < 2 \wedge ib$
shows $\text{table3 } ib \text{ } gs \ ((z0 * 2 \wedge ib + x0) * 2 \wedge ib + y0)$
 $= \text{sum-list } (\text{map } g\text{-coeff}$
 $(\text{filter } (\lambda g. g\text{-out } g = z0 \wedge g\text{-in1 } g = x0 \wedge g\text{-in2 } g = y0) \text{ } gs))$

proof –

have $\text{filter } (\lambda g. \text{enc3 } ib \text{ } g = (z0 * 2 \wedge ib + x0) * 2 \wedge ib + y0) \text{ } gs$
 $= \text{filter } (\lambda g. g\text{-out } g = z0 \wedge g\text{-in1 } g = x0 \wedge g\text{-in2 } g = y0) \text{ } gs$

proof (*intro filter-cong refl*)

fix g **assume** $gin: g \in \text{set } gs$

have $gi1: g\text{-in1 } g < 2 \wedge ib$ **and** $gi2: g\text{-in2 } g < 2 \wedge ib$

using $wf \text{ } gin$ **by** *auto*

have $\text{inner}: g\text{-out } g * 2 \wedge ib + g\text{-in1 } g = z0 * 2 \wedge ib + x0$
 $\longleftrightarrow g\text{-out } g = z0 \wedge g\text{-in1 } g = x0$

using $\text{pack-inj}[OF \text{ } gi1 \text{ } x0]$.

have $\text{enc3 } ib \text{ } g = (z0 * 2 \wedge ib + x0) * 2 \wedge ib + y0$
 $\longleftrightarrow g\text{-out } g * 2 \wedge ib + g\text{-in1 } g = z0 * 2 \wedge ib + x0 \wedge g\text{-in2 } g = y0$

unfolding enc3-def **using** $\text{pack-inj}[OF \text{ } gi2 \text{ } y0]$.

then show $(\text{enc3 } ib \text{ } g = (z0 * 2 \wedge ib + x0) * 2 \wedge ib + y0)$
 $= (g\text{-out } g = z0 \wedge g\text{-in1 } g = x0 \wedge g\text{-in2 } g = y0)$

using inner **by** *auto*

qed

then show *?thesis* **by** (*simp add: table3-def*)

qed

lemma *table2-at-tuple*:

assumes $x0: x0 < 2 \wedge ib$
and $wf: \forall g \in \text{set } gs. g\text{-in1 } g < 2 \wedge ib$
shows $\text{table2 } ib \text{ } gs \ (z0 * 2 \wedge ib + x0)$
 $= \text{sum-list } (\text{map } g\text{-coeff } (\text{filter } (\lambda g. g\text{-out } g = z0 \wedge g\text{-in1 } g = x0) \text{ } gs))$

proof –

have $\text{filter } (\lambda g. \text{enc2 } ib \text{ } g = z0 * 2 \wedge ib + x0) \text{ } gs$
 $= \text{filter } (\lambda g. g\text{-out } g = z0 \wedge g\text{-in1 } g = x0) \text{ } gs$

proof (*intro filter-cong refl*)

fix g **assume** $gin: g \in \text{set } gs$

have $gi1: g\text{-in1 } g < 2 \wedge ib$ **using** $wf \text{ } gin$ **by** *auto*

show $(\text{enc2 } ib \text{ } g = z0 * 2 \wedge ib + x0) = (g\text{-out } g = z0 \wedge g\text{-in1 } g = x0)$

unfolding enc2-def **using** $\text{pack-inj}[OF \text{ } gi1 \text{ } x0]$ **by** *auto*

qed

then show *?thesis* **by** (*simp add: table2-def*)

qed

Range and split facts for the packed encodings.

lemma *enc3-lt*:

assumes $g\text{-out } g < 2 \wedge ob \text{ } g\text{-in1 } g < 2 \wedge ib \text{ } g\text{-in2 } g < 2 \wedge ib$
shows $\text{enc3 } ib \text{ } g < 2 \wedge (ob + ib + ib)$

proof –

have $\text{outer}: g\text{-out } g * 2 \wedge ib + g\text{-in1 } g < 2 \wedge ob * 2 \wedge ib$
using $\text{assms}(1,2)$ **by** (*rule pack-lt*)

have $(g\text{-out } g * 2 \wedge ib + g\text{-in1 } g) * 2 \wedge ib + g\text{-in2 } g < (2 \wedge ob * 2 \wedge ib) * 2 \wedge ib$
using $\text{outer assms}(3)$ **by** (*rule pack-lt*)

then show *?thesis* **by** (*simp add: enc3-def power-add*)

qed

lemma *enc2-lt*:

assumes $g\text{-out } g < 2^{\wedge} ob \text{ } g\text{-in1 } g < 2^{\wedge} ib$
shows $enc2 \text{ } ib \text{ } g < 2^{\wedge} (ob + ib)$
unfolding *enc2-def* **using** *pack-lt[OF assms]* **by** (*simp add: power-add*)

lemma *enc3-split*:

$enc3 \text{ } ib \text{ } g = g\text{-out } g * 2^{\wedge} (ib + ib) + (g\text{-in1 } g * 2^{\wedge} ib + g\text{-in2 } g)$
by (*simp add: enc3-def power-add algebra-simps*)

The weight of a packed gate index over a concatenated point splits into the per-block weights (two applications of the split identity).

lemma *eq-pi-enc3*:

fixes $z \text{ } x \text{ } y :: 'f::comm-ring-1 \text{ list}$
assumes $lz: \text{length } z = ob \text{ and } lx: \text{length } x = ib \text{ and } ly: \text{length } y = ib$
and $go: g\text{-out } g < 2^{\wedge} ob \text{ and } g1: g\text{-in1 } g < 2^{\wedge} ib \text{ and } g2: g\text{-in2 } g < 2^{\wedge} ib$
shows $eq\text{-pi } (z @ x @ y) (enc3 \text{ } ib \text{ } g)$
 $= eq\text{-pi } z (g\text{-out } g) * eq\text{-pi } x (g\text{-in1 } g) * eq\text{-pi } y (g\text{-in2 } g)$
proof –
have *inner-lt*: $g\text{-in1 } g * 2^{\wedge} ib + g\text{-in2 } g < 2^{\wedge} (ib + ib)$
using *pack-lt[OF g1 g2]* **by** (*simp add: power-add*)
have $eq\text{-pi } (z @ x @ y) (enc3 \text{ } ib \text{ } g)$
 $= eq\text{-pi } (z @ x @ y) (g\text{-out } g * 2^{\wedge} \text{length } (x @ y) + (g\text{-in1 } g * 2^{\wedge} ib + g\text{-in2 } g))$
by (*simp add: enc3-split lx ly power-add*)
also have $\dots = eq\text{-pi } z (g\text{-out } g) * eq\text{-pi } (x @ y) (g\text{-in1 } g * 2^{\wedge} ib + g\text{-in2 } g)$
by (*rule eq-pi-append*) (*use go lz lx ly inner-lt in <simp-all add: power-add>*)
also have $\dots = eq\text{-pi } z (g\text{-out } g) * (eq\text{-pi } x (g\text{-in1 } g) * eq\text{-pi } y (g\text{-in2 } g))$
using *eq-pi-append[of g-in1 g x g-in2 g y]* *g1 g2 lx ly* **by** *simp*
finally show *?thesis* **by** (*simp add: mult.assoc*)

qed

lemma *eq-pi-enc2*:

fixes $z \text{ } x :: 'f::comm-ring-1 \text{ list}$
assumes $lz: \text{length } z = ob \text{ and } lx: \text{length } x = ib$
and $go: g\text{-out } g < 2^{\wedge} ob \text{ and } g1: g\text{-in1 } g < 2^{\wedge} ib$
shows $eq\text{-pi } (z @ x) (enc2 \text{ } ib \text{ } g) = eq\text{-pi } z (g\text{-out } g) * eq\text{-pi } x (g\text{-in1 } g)$
unfolding *enc2-def*
using *eq-pi-append[of g-out g z g-in1 g x]* *go g1 lz lx* **by** *simp*

17 The definitional MLE equals the gate sum (table oracle soundness)

On a concatenated point the *mle* of the packed tuple table is exactly the per-gate sparse sum computed by the table oracle. This discharges the FV-CONTRACT shape "the MLE agrees with the layer's weighted wiring" at the level of the definitional table.

lemma *table-mle3-gates*:

fixes $z \text{ } x \text{ } y :: 'f::comm-ring-1 \text{ list}$
assumes $wf: \forall g \in \text{set } gs. g\text{-kind } g = GMul \longrightarrow$
 $g\text{-out } g < 2^{\wedge} ob \wedge g\text{-in1 } g < 2^{\wedge} ib \wedge g\text{-in2 } g < 2^{\wedge} ib$

```

    and lz: length z = ob and lx: length x = ib and ly: length y = ib
  shows mle (ob + ib + ib) (table3 ib (filter (λg. g-kind g = GMul) gs)) (z @ x @ y)
    = gates-mle-eval gs GMul z x y
  using wf
proof (induction gs)
  case Nil
  have table3 ib (filter (λg. g-kind g = GMul) []) = (λ-. 0) by (simp add: fun-eq-iff)
  then show ?case by (simp add: mle-zero)
next
  case (Cons g gs)
  have IH: mle (ob + ib + ib) (table3 ib (filter (λg. g-kind g = GMul) gs)) (z @ x @ y)
    = gates-mle-eval gs GMul z x y
    using Cons by simp
  show ?case
  proof (cases g-kind g = GMul)
    case False
    then show ?thesis using IH by (simp add: gates-mle-eval-Cons)
  next
    case True
    have go: g-out g < 2 ^ ob and g1: g-in1 g < 2 ^ ib and g2: g-in2 g < 2 ^ ib
      using Cons.premis True by auto
    have elt: enc3 ib g < 2 ^ (ob + ib + ib)
      by (rule enc3-lt[OF go g1 g2])
    have step: mle (ob + ib + ib) (table3 ib (filter (λg. g-kind g = GMul) (g # gs))) (z @ x @ y)
      = mle (ob + ib + ib) (λidx. if enc3 ib g = idx then g-coeff g else 0) (z @ x @ y)
        + mle (ob + ib + ib) (table3 ib (filter (λg. g-kind g = GMul) gs)) (z @ x @ y)
    proof -
      have mle (ob + ib + ib) (table3 ib (filter (λg. g-kind g = GMul) (g # gs))) (z @ x @ y)
        = mle (ob + ib + ib)
          (λidx. (if enc3 ib g = idx then g-coeff g else 0)
            + table3 ib (filter (λg. g-kind g = GMul) gs) idx) (z @ x @ y)
        by (intro mle-cong) (simp add: True table3-Cons)
      also have ... = mle (ob + ib + ib) (λidx. if enc3 ib g = idx then g-coeff g else 0) (z @ x @ y)
        + mle (ob + ib + ib) (table3 ib (filter (λg. g-kind g = GMul) gs)) (z @ x @ y)
        by (rule mle-add)
      finally show ?thesis .
    qed
  have delta: mle (ob + ib + ib) (λidx. if enc3 ib g = idx then g-coeff g else 0) (z @ x @ y)
    = wiring-term GMul z x y g
  proof -
    have mle (ob + ib + ib) (λidx. if enc3 ib g = idx then g-coeff g else 0) (z @ x @ y)
      = eq-pi (z @ x @ y) (enc3 ib g) * g-coeff g
      by (rule mle-delta'[OF elt])
    also have ... = eq-pi z (g-out g) * eq-pi x (g-in1 g) * eq-pi y (g-in2 g) * g-coeff g
      by (simp only: eq-pi-enc3[OF lz lx ly go g1 g2])
    finally show ?thesis
      by (simp add: wiring-term-def algebra-simps)
    qed
  show ?thesis
    using step delta IH True by (simp add: gates-mle-eval-Cons)
  qed
qed

```

```

lemma table-mle2-gates:
  fixes z x y :: 'f::comm-ring-1 list
  assumes kk: k = GLin  $\vee$  k = GPow3
    and wf:  $\forall g \in \text{set } gs. g\text{-kind } g = k \longrightarrow g\text{-out } g < 2 \wedge ob \wedge g\text{-in1 } g < 2 \wedge ib$ 
    and lz: length z = ob and lx: length x = ib
  shows mle (ob + ib) (table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) gs)) (z @ x)
    = gates-mle-eval gs k z x y
  using wf
proof (induction gs)
  case Nil
    have table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) []) = ( $\lambda -. 0$ ) by (simp add: fun-eq-iff)
    then show ?case by (simp add: mle-zero)
  next
  case (Cons g gs)
    have IH: mle (ob + ib) (table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) gs)) (z @ x)
      = gates-mle-eval gs k z x y
    using Cons by simp
    have knm: k  $\neq$  GMul using kk by auto
    show ?case
    proof (cases g-kind g = k)
      case False
        then show ?thesis using IH by (simp add: gates-mle-eval-Cons)
      next
      case True
        have go: g-out g < 2  $\wedge$  ob and g1: g-in1 g < 2  $\wedge$  ib
          using Cons.prem1 True by auto
        have elt: enc2 ib g < 2  $\wedge$  (ob + ib)
          by (rule enc2-lt[OF go g1])
        have step: mle (ob + ib) (table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) (g # gs))) (z @ x)
          = mle (ob + ib) ( $\lambda idx. \text{if } enc2 \text{ ib } g = idx \text{ then } g\text{-coeff } g \text{ else } 0$ ) (z @ x)
            + mle (ob + ib) (table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) gs)) (z @ x)
        proof -
          have mle (ob + ib) (table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) (g # gs))) (z @ x)
            = mle (ob + ib)
              ( $\lambda idx. (\text{if } enc2 \text{ ib } g = idx \text{ then } g\text{-coeff } g \text{ else } 0)$ 
                + table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) gs) idx) (z @ x)
            by (intro mle-cong) (simp add: True table2-Cons)
          also have ... = mle (ob + ib) ( $\lambda idx. \text{if } enc2 \text{ ib } g = idx \text{ then } g\text{-coeff } g \text{ else } 0$ ) (z @ x)
            + mle (ob + ib) (table2 ib (filter ( $\lambda g. g\text{-kind } g = k$ ) gs)) (z @ x)
            by (rule mle-add)
          finally show ?thesis .
        qed
        have delta: mle (ob + ib) ( $\lambda idx. \text{if } enc2 \text{ ib } g = idx \text{ then } g\text{-coeff } g \text{ else } 0$ ) (z @ x)
          = wiring-term k z x y g
        proof -
          have mle (ob + ib) ( $\lambda idx. \text{if } enc2 \text{ ib } g = idx \text{ then } g\text{-coeff } g \text{ else } 0$ ) (z @ x)
            = eq-pi (z @ x) (enc2 ib g) * g-coeff g
          by (rule mle-delta'[OF elt])
          also have ... = eq-pi z (g-out g) * eq-pi x (g-in1 g) * g-coeff g
          by (simp only: eq-pi-enc2[OF lz lx go g1])
          finally show ?thesis
            using knm by (simp add: wiring-term-def algebra-simps)
        qed

```

```

show ?thesis
  using step delta IH True by (simp add: gates-mle-eval-Cons)
qed
qed

```

Constant vector: the definitional MLE of the accumulated-constant table (*const-at*, list semantics) is the sparse per-entry sum of the table oracle.

definition *ctable* :: (nat × 'f::comm-monoid-add) list ⇒ nat ⇒ 'f **where**
ctable cs z = *sum-list* (*map snd* (*filter* (λ(*w*, *c*). *w* = *z*) *cs*))

lemma *ctable-Nil* [*simp*]: *ctable* [] *z* = 0
by (*simp add: ctable-def*)

lemma *ctable-Cons*:
ctable ((*w*, *v*) # *cs*) *z* = (if *w* = *z* then *v* else 0) + *ctable cs z*
by (*simp add: ctable-def*)

lemma *const-at-ctable*: *const-at L z* = *ctable* (*layer-consts L*) *z*
by (*simp add: const-at-def ctable-def*)

lemma *const-mle-eq*:
fixes *z* :: 'f::comm-ring-1 list
assumes *wf*: ∀(*w*, *v*) ∈ *set cs*. *w* < 2 ^ *ob*
and *lz*: *length z* = *ob*
shows *mle ob* (*ctable cs*) *z* = *consts-mle-eval cs z*
using *wf*
proof (*induction cs*)
case Nil
have *ctable* ([] :: (nat × 'f) list) = (λ-. 0) **by** (*simp add: fun-eq-iff*)
then show ?*case* **by** (*simp add: mle-zero*)
next
case (*Cons wv cs*)
obtain *w v* **where** *wv*: *wv* = (*w*, *v*) **by** (*cases wv*)
have *wlt*: *w* < 2 ^ *ob* **using** *Cons.prem*s *wv* **by** *auto*
have *IH*: *mle ob* (*ctable cs*) *z* = *consts-mle-eval cs z* **using** *Cons* **by** *simp*
have *mle ob* (*ctable* ((*w*, *v*) # *cs*)) *z*
= *mle ob* (λ*z0*. (if *w* = *z0* then *v* else 0) + *ctable cs z0*) *z*
by (*intro mle-cong*) (*simp add: ctable-Cons*)
also have ... = *mle ob* (λ*z0*. if *w* = *z0* then *v* else 0) *z* + *mle ob* (*ctable cs*) *z*
by (*rule mle-add*)
also have ... = *v* * *eq-pi z w* + *consts-mle-eval cs z*
using *mle-delta'*[*OF wlt*, *of v z*] *lz IH* **by** (*simp add: algebra-simps*)
finally show ?*case* **using** *wv* **by** (*simp add: consts-mle-eval-Cons*)
qed

18 The derived closed-form representation (DerivedRegular-Wiring)

One SIDE of a derived family: block *b*'s wire at local offset *off* is (*base-hi* + *b*) * 2 ^ *lb* + *off* (an aligned arithmetic progression with stride 2 ^ *lb*), or one fixed wire shared by every block (the degenerate progression). Literally *Side*::{*Prog*, *Fixed*} of wiring.rs.

```

datatype side =
  Prog nat nat    — Prog base-hi lb
| Fixed nat      — Fixed wire

fun side-lb :: side  $\Rightarrow$  nat where
  side-lb (Prog bh lb) = lb
| side-lb (Fixed w) = 0

fun side-wire :: side  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat where
  side-wire (Prog bh lb) b off = (bh + b) * 2 ^ lb + off
| side-wire (Fixed w) b off = w

```

One derived gate family: *count* blocks whose taps follow the group's side progressions. The Rust *GateGroup* keeps three kind-homogeneous tap vectors (lin / pow3 / mul); the model merges them into one kind-tagged tap list, which carries the same data (the split is a Rust layout convenience). A unary tap re-uses its *in1* wire as *in2* on expansion, exactly like *expand-gates* (*in2*: *in1*).

```

record 'f tap =
  tap-kind  :: gate-kind
  tap-out   :: nat
  tap-in1   :: nat
  tap-in2   :: nat
  tap-coeff :: 'f

record 'f gate-group =
  grp-count :: nat
  grp-out   :: side
  grp-in1   :: side
  grp-in2   :: side
  grp-taps  :: 'f tap list

record 'f const-group =
  cg-count :: nat
  cg-out   :: side
  cg-vals  :: (nat  $\times$  'f) list

record 'f derived-layer =
  drv-groups  :: 'f gate-group list
  drv-sparse  :: 'f gate list
  drv-cgroups :: 'f const-group list
  drv-csparse :: (nat  $\times$  'f) list

```

18.1 Re-expansion (the audit helper *expand-gates* / *expand-consts*)

```

definition tap-gate :: 'f gate-group  $\Rightarrow$  nat  $\Rightarrow$  'f tap  $\Rightarrow$  'f gate where
  tap-gate G b t =
    (| g-kind = tap-kind t,
      g-out   = side-wire (grp-out G) b (tap-out t),
      g-in1   = side-wire (grp-in1 G) b (tap-in1 t),
      g-in2   = (if tap-kind t = GMul then side-wire (grp-in2 G) b (tap-in2 t)
                  else side-wire (grp-in1 G) b (tap-in1 t)),
      g-coeff = tap-coeff t |)

```

lemma *tap-gate-sel* [*simp*]:

g-kind (*tap-gate* *G* *b* *t*) = *tap-kind* *t*
g-out (*tap-gate* *G* *b* *t*) = *side-wire* (*grp-out* *G*) *b* (*tap-out* *t*)
g-in1 (*tap-gate* *G* *b* *t*) = *side-wire* (*grp-in1* *G*) *b* (*tap-in1* *t*)
g-coeff (*tap-gate* *G* *b* *t*) = *tap-coeff* *t*
by (*simp-all* *add*: *tap-gate-def*)

lemma *tap-gate-in2-mul* [*simp*]:

tap-kind *t* = *GMul* $\implies$ *g-in2* (*tap-gate* *G* *b* *t*) = *side-wire* (*grp-in2* *G*) *b* (*tap-in2* *t*)
by (*simp* *add*: *tap-gate-def*)

definition *expand-group* :: 'f *gate-group* $\Rightarrow$ 'f *gate list* **where**

expand-group *G* = *concat* (*map* (λb . *map* (*tap-gate* *G* *b*) (*grp-taps* *G*)) [*0* ..< *grp-count* *G*])

definition *expand-gates* :: 'f *derived-layer* $\Rightarrow$ 'f *gate list* **where**

expand-gates *D* = *concat* (*map* *expand-group* (*drv-groups* *D*)) @ *drv-sparse* *D*

definition *expand-cgroup* :: 'f *const-group* $\Rightarrow$ (*nat* $\times$ 'f) *list* **where**

expand-cgroup *CG* =
concat (*map* (λb . *map* ($\lambda(u, v)$. (*side-wire* (*cg-out* *CG*) *b* *u*, *v*)) (*cg-vals* *CG*))
[*0* ..< *cg-count* *CG*])

definition *expand-consts* :: 'f *derived-layer* $\Rightarrow$ (*nat* $\times$ 'f) *list* **where**

expand-consts *D* = *concat* (*map* *expand-cgroup* (*drv-cgroups* *D*)) @ *drv-csparse* *D*

18.2 Well-formedness of a derived representation

wf-side *n cnt* *s*: on a progression side, the local width fits in the point and every enumerated high index is representable - literally the Rust requirement *bases*[*t*] + *count* <= $2^{\sim\{points[t].len()\}}$ of *shifted-range-eq*. *tap-off-ok*: offsets stay below the stride (the factorization condition). Fixed sides need nothing: both the expansion and the evaluation ignore the offset there.

fun *wf-side* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *side* $\Rightarrow$ *bool* **where**

wf-side *n cnt* (*Prog* *bh* *lb*) $\longleftrightarrow$ *lb* $\leq$ *n* $\wedge$ *bh* + *cnt* $\leq$ $2^{\wedge} (n - lb)$

| *wf-side* *n cnt* (*Fixed* *w*) $\longleftrightarrow$ *True*

fun *tap-off-ok* :: *side* $\Rightarrow$ *nat* $\Rightarrow$ *bool* **where**

tap-off-ok (*Prog* *bh* *lb*) *u* $\longleftrightarrow$ *u* < $2^{\wedge} lb$

| *tap-off-ok* (*Fixed* *w*) *u* $\longleftrightarrow$ *True*

definition *wf-group* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ 'f *gate-group* $\Rightarrow$ *bool* **where**

wf-group *ob* *ib* *G* $\longleftrightarrow$

wf-side *ob* (*grp-count* *G*) (*grp-out* *G*) $\wedge$

wf-side *ib* (*grp-count* *G*) (*grp-in1* *G*) $\wedge$

wf-side *ib* (*grp-count* *G*) (*grp-in2* *G*) $\wedge$

($\forall t \in set$ (*grp-taps* *G*).

tap-off-ok (*grp-out* *G*) (*tap-out* *t*) $\wedge$

tap-off-ok (*grp-in1* *G*) (*tap-in1* *t*) $\wedge$

(*tap-kind* *t* = *GMul* $\longrightarrow$ *tap-off-ok* (*grp-in2* *G*) (*tap-in2* *t*)))

definition *wf-cgroup* :: *nat* $\Rightarrow$ 'f *const-group* $\Rightarrow$ *bool* **where**

$wf\text{-}cgroup\ ob\ CG \longleftrightarrow$
 $wf\text{-}side\ ob\ (cg\text{-}count\ CG)\ (cg\text{-}out\ CG) \wedge$
 $(\forall (u, v) \in set\ (cg\text{-}vals\ CG). tap\text{-}off\text{-}ok\ (cg\text{-}out\ CG)\ u)$

definition $wf\text{-}derived :: nat \Rightarrow nat \Rightarrow 'f\ derived\text{-}layer \Rightarrow bool$ **where**
 $wf\text{-}derived\ ob\ ib\ D \longleftrightarrow$
 $(\forall G \in set\ (drv\text{-}groups\ D). wf\text{-}group\ ob\ ib\ G) \wedge$
 $(\forall CG \in set\ (drv\text{-}cgroups\ D). wf\text{-}cgroup\ ob\ CG)$

18.3 Closed-form evaluation (the derived oracle)

Block-matching factor of one side at block b : the $eq\text{-}pi$ weight of the high part on a progression side (one term of the shifted-range sum), or the full-point weight of the shared wire on a fixed side (the Rust *scalar*; block-independent, kept inside the block sum here - the same element by distributivity).

fun $side\text{-}block\text{-}eq :: 'f::comm\text{-}ring\text{-}1\ list \Rightarrow side \Rightarrow nat \Rightarrow 'f$ **where**
 $side\text{-}block\text{-}eq\ pt\ (Prog\ bh\ lb)\ b = eq\text{-}pi\ (take\ (length\ pt - lb)\ pt)\ (bh + b)$
 $| side\text{-}block\text{-}eq\ pt\ (Fixed\ w)\ b = eq\text{-}pi\ pt\ w$

definition $side\text{-}lo\text{-}pt :: 'f\ list \Rightarrow side \Rightarrow 'f\ list$ **where**
 $side\text{-}lo\text{-}pt\ pt\ s = drop\ (length\ pt - side\text{-}lb\ s)\ pt$

The specification form of *shifted-range-eq*: the sum over the block range of the product of per-term high-part weights. The Rust carry-DP is an $O(\log^2)$ evaluation strategy for exactly this sum (pinned executable by *shifted-range-eq-matches-bruteforce*); the model consumes the sum.

definition $group\text{-}block :: 'f::comm\text{-}ring\text{-}1\ gate\text{-}group \Rightarrow bool \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f$ **where**
 $group\text{-}block\ G\ binp\ z\ x\ y =$
 $(\sum b < grp\text{-}count\ G.$
 $side\text{-}block\text{-}eq\ z\ (grp\text{-}out\ G)\ b * side\text{-}block\text{-}eq\ x\ (grp\text{-}in1\ G)\ b *$
 $(if\ binp\ then\ side\text{-}block\text{-}eq\ y\ (grp\text{-}in2\ G)\ b\ else\ 1))$

definition $tap\text{-}term :: gate\text{-}kind \Rightarrow 'f::comm\text{-}ring\text{-}1\ gate\text{-}group \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f$ **where**
 $tap\text{-}term\ k\ G\ z\ x\ y\ t =$
 $tap\text{-}coeff\ t * eq\text{-}pi\ (side\text{-}lo\text{-}pt\ z\ (grp\text{-}out\ G))\ (tap\text{-}out\ t)$
 $* eq\text{-}pi\ (side\text{-}lo\text{-}pt\ x\ (grp\text{-}in1\ G))\ (tap\text{-}in1\ t)$
 $* (if\ k = GMul\ then\ eq\text{-}pi\ (side\text{-}lo\text{-}pt\ y\ (grp\text{-}in2\ G))\ (tap\text{-}in2\ t)\ else\ 1)$

definition $group\text{-}template :: gate\text{-}kind \Rightarrow 'f::comm\text{-}ring\text{-}1\ gate\text{-}group \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f$ **where**
 $group\text{-}template\ k\ G\ z\ x\ y =$
 $sum\text{-}list\ (map\ (tap\text{-}term\ k\ G\ z\ x\ y)\ (filter\ (\lambda t. tap\text{-}kind\ t = k)\ (grp\text{-}taps\ G)))$

definition $derived\text{-}eval :: 'f::comm\text{-}ring\text{-}1\ derived\text{-}layer \Rightarrow gate\text{-}kind \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow 'f$ **where**
 $derived\text{-}eval\ D\ k\ z\ x\ y =$
 $sum\text{-}list\ (map\ (\lambda G. group\text{-}block\ G\ (k = GMul)\ z\ x\ y * group\text{-}template\ k\ G\ z\ x\ y)$
 $(drv\text{-}groups\ D))$
 $+ gates\text{-}mle\text{-}eval\ (drv\text{-}sparse\ D)\ k\ z\ x\ y$

definition *cgroup-eval* :: 'f::comm-ring-1 const-group $\Rightarrow$ 'f list $\Rightarrow$ 'f **where**
cgroup-eval CG z =
 $(\sum b < \text{cg-count } CG. \text{side-block-eq } z (\text{cg-out } CG) b) *$
 $\text{sum-list } (\text{map } (\lambda(u, v). v * \text{eq-pi } (\text{side-lo-pt } z (\text{cg-out } CG)) u) (\text{cg-vals } CG))$

definition *derived-const-eval* :: 'f::comm-ring-1 derived-layer $\Rightarrow$ 'f list $\Rightarrow$ 'f **where**
derived-const-eval D z =
 $\text{sum-list } (\text{map } (\lambda CG. \text{cgroup-eval } CG z) (\text{drv-cgroups } D))$
 $+ \text{consts-mle-eval } (\text{drv-csparse } D) z$

19 Re-association: the closed form equals the expanded gate sum

The per-wire split (the Rust correctness comment, made a lemma): the weight of a progression wire factors into block part times local part.

lemma *side-eq-split*:
fixes pt :: 'f::comm-ring-1 list
assumes ws: wf-side (length pt) cnt s
and b: b < cnt
and u: tap-off-ok s u
shows eq-pi pt (side-wire s b u) = side-block-eq pt s b * eq-pi (side-lo-pt pt s) u
proof (cases s)
case (Fixed w)
then show ?thesis **by** (simp add: side-lo-pt-def)
next
case (Prog bh lb)
have lb-le: lb $\leq$ length pt **and** range: bh + cnt $\leq$ 2 $\wedge$ (length pt - lb)
using ws Prog **by** simp-all
have b-lt: bh + b < 2 $\wedge$ (length pt - lb) **using** b range **by** simp
have u-lt: u < 2 $\wedge$ lb **using** u Prog **by** simp
have len-take: length (take (length pt - lb) pt) = length pt - lb
using lb-le **by** simp
have len-drop: length (drop (length pt - lb) pt) = lb
using lb-le **by** simp
have eq-pi pt ((bh + b) * 2 $\wedge$ lb + u)
 $= \text{eq-pi } (\text{take } (\text{length pt} - \text{lb}) \text{ pt} @ \text{drop } (\text{length pt} - \text{lb}) \text{ pt})$
 $((bh + b) * 2 \wedge \text{length } (\text{drop } (\text{length pt} - \text{lb}) \text{ pt}) + u)$
using lb-le **by** simp
also have ... = eq-pi (take (length pt - lb) pt) (bh + b)
 $* \text{eq-pi } (\text{drop } (\text{length pt} - \text{lb}) \text{ pt}) u$
by (rule eq-pi-append) (use b-lt u-lt len-take len-drop **in** simp-all)
finally show ?thesis
using Prog **by** (simp add: side-lo-pt-def)
qed

Per-tap re-association: one expanded gate's sparse term is the block factor times the local template term.

lemma *wiring-term-tap*:
fixes z x y :: 'f::comm-ring-1 list
assumes wo: wf-side (length z) (grp-count G) (grp-out G)
and w1: wf-side (length x) (grp-count G) (grp-in1 G)

```

and w2: wf-side (length y) (grp-count G) (grp-in2 G)
and b: b < grp-count G
and uo: tap-off-ok (grp-out G) (tap-out t)
and u1: tap-off-ok (grp-in1 G) (tap-in1 t)
and u2: tap-kind t = GMul  $\implies$  tap-off-ok (grp-in2 G) (tap-in2 t)
and kt: tap-kind t = k
shows wiring-term k z x y (tap-gate G b t)
  = (side-block-eq z (grp-out G) b * side-block-eq x (grp-in1 G) b *
    (if k = GMul then side-block-eq y (grp-in2 G) b else 1))
    * tap-term k G z x y t
proof (cases k = GMul)
  case True
  have so: eq-pi z (side-wire (grp-out G) b (tap-out t))
    = side-block-eq z (grp-out G) b * eq-pi (side-lo-pt z (grp-out G)) (tap-out t)
  by (rule side-eq-split[OF wo b uo])
  have s1: eq-pi x (side-wire (grp-in1 G) b (tap-in1 t))
    = side-block-eq x (grp-in1 G) b * eq-pi (side-lo-pt x (grp-in1 G)) (tap-in1 t)
  by (rule side-eq-split[OF w1 b u1])
  have s2: eq-pi y (side-wire (grp-in2 G) b (tap-in2 t))
    = side-block-eq y (grp-in2 G) b * eq-pi (side-lo-pt y (grp-in2 G)) (tap-in2 t)
  by (rule side-eq-split[OF w2 b u2]) (use kt True in simp)
  show ?thesis
  using True kt
  by (simp add: wiring-term-def tap-term-def so s1 s2 algebra-simps)
next
  case False
  have so: eq-pi z (side-wire (grp-out G) b (tap-out t))
    = side-block-eq z (grp-out G) b * eq-pi (side-lo-pt z (grp-out G)) (tap-out t)
  by (rule side-eq-split[OF wo b uo])
  have s1: eq-pi x (side-wire (grp-in1 G) b (tap-in1 t))
    = side-block-eq x (grp-in1 G) b * eq-pi (side-lo-pt x (grp-in1 G)) (tap-in1 t)
  by (rule side-eq-split[OF w1 b u1])
  show ?thesis
  using False kt
  by (simp add: wiring-term-def tap-term-def so s1 algebra-simps)
qed

```

Group-level re-association: summing the expanded gates of one group IS the product of its block factor and its local template sum - the derived oracle only re-associates the table oracle's field sum.

lemma group-eval-expand:

```

fixes z x y :: 'f::comm-ring-1 list
assumes wf: wf-group (length z) (length x) G
and ly: length y = length x
shows gates-mle-eval (expand-group G) k z x y
  = group-block G (k = GMul) z x y * group-template k G z x y
proof -
  let ?B =  $\lambda b.$  side-block-eq z (grp-out G) b * side-block-eq x (grp-in1 G) b *
    (if k = GMul then side-block-eq y (grp-in2 G) b else 1)
  have wo: wf-side (length z) (grp-count G) (grp-out G)
  and w1: wf-side (length x) (grp-count G) (grp-in1 G)
  and w2: wf-side (length y) (grp-count G) (grp-in2 G)

```

```

using wf ly by (simp-all add: wf-group-def)
have taps:  $\forall t \in \text{set } (\text{grp-taps } G).$ 
  tap-off-ok (grp-out G) (tap-out t)  $\wedge$ 
  tap-off-ok (grp-in1 G) (tap-in1 t)  $\wedge$ 
  (tap-kind t = GMul  $\longrightarrow$  tap-off-ok (grp-in2 G) (tap-in2 t))
using wf by (simp add: wf-group-def)
have gates-mle-eval (expand-group G) k z x y
  = sum-list (map ( $\lambda b.$  gates-mle-eval (map (tap-gate G b) (grp-taps G)) k z x y)
    [0 ..< grp-count G])
unfolding expand-group-def gates-mle-eval-def
by (simp add: filter-concat map-concat sum-list-concat sum-list-concat-map o-def)
also have ... = sum-list (map ( $\lambda b.$  ?B b * group-template k G z x y) [0 ..< grp-count G])
proof (intro arg-cong[of - - sum-list] map-cong refl)
  fix b assume b  $\in \text{set } [0 ..< \text{grp-count } G]$ 
  then have blt: b < grp-count G by simp
  have gates-mle-eval (map (tap-gate G b) (grp-taps G)) k z x y
    = sum-list (map (wiring-term k z x y  $\circ$  tap-gate G b)
      (filter ( $\lambda t.$  tap-kind t = k) (grp-taps G)))
  unfolding gates-mle-eval-def
  by (simp add: filter-map o-def)
also have ... = sum-list (map ( $\lambda t.$  ?B b * tap-term k G z x y t)
  (filter ( $\lambda t.$  tap-kind t = k) (grp-taps G)))
proof (intro arg-cong[of - - sum-list] map-cong refl)
  fix t assume tin: t  $\in \text{set } (\text{filter } (\lambda t.$  tap-kind t = k) (grp-taps G))
  then have tset: t  $\in \text{set } (\text{grp-taps } G)$  and tk: tap-kind t = k by auto
  show (wiring-term k z x y  $\circ$  tap-gate G b) t = ?B b * tap-term k G z x y t
    using wiring-term-tap[OF wo w1 w2 blt - - tk] taps tset by (simp add: o-def)
qed
also have ... = ?B b * group-template k G z x y
  unfolding group-template-def by (simp add: sum-list-const-mult)
finally show gates-mle-eval (map (tap-gate G b) (grp-taps G)) k z x y
  = ?B b * group-template k G z x y .
qed
also have ... = ( $\sum b < \text{grp-count } G.$  ?B b * group-template k G z x y)
  by (simp add: sum-list-map-upt)
also have ... = ( $\sum b < \text{grp-count } G.$  ?B b) * group-template k G z x y
  by (simp add: sum-distrib-right)
also have ... = group-block G (k = GMul) z x y * group-template k G z x y
  by (simp add: group-block-def)
finally show ?thesis .
qed

```

lemma cgroup-eval-expand:

```

fixes z :: 'f::comm-ring-1 list
assumes wf: wf-cgroup (length z) CG
shows consts-mle-eval (expand-cgroup CG) z = cgroup-eval CG z
proof -
  let ?B =  $\lambda b.$  side-block-eq z (cg-out CG) b
  let ?T = sum-list (map ( $\lambda(u, v).$  v * eq-pi (side-lo-pt z (cg-out CG)) u) (cg-vals CG))
  have wo: wf-side (length z) (cg-count CG) (cg-out CG)
    using wf by (simp add: wf-cgroup-def)
  have offs:  $\forall (u, v) \in \text{set } (\text{cg-vals } CG).$  tap-off-ok (cg-out CG) u
    using wf by (simp add: wf-cgroup-def)

```

```

have consts-mle-eval (expand-cgroup CG) z
  = sum-list (map (λb. consts-mle-eval
    (map (λ(u, v). (side-wire (cg-out CG) b u, v)) (cg-vals CG)) z)
    [0 ..< cg-count CG])
  unfolding expand-cgroup-def consts-mle-eval-def
  by (simp add: map-concat sum-list-concat sum-list-concat-map o-def case-prod-unfold)
also have ... = sum-list (map (λb. ?B b * ?T) [0 ..< cg-count CG])
proof (intro arg-cong[of - - sum-list] map-cong refl)
  fix b assume b ∈ set [0 ..< cg-count CG]
  then have blt: b < cg-count CG by simp
  have consts-mle-eval (map (λ(u, v). (side-wire (cg-out CG) b u, v)) (cg-vals CG)) z
    = sum-list (map (λ(u, v). v * eq-pi z (side-wire (cg-out CG) b u)) (cg-vals CG))
  unfolding consts-mle-eval-def
  by (simp add: o-def case-prod-unfold)
also have ... = sum-list (map (λ(u, v). ?B b * (v * eq-pi (side-lo-pt z (cg-out CG)) u))
    (cg-vals CG))
proof (intro arg-cong[of - - sum-list] map-cong refl)
  fix uv assume uvin: uv ∈ set (cg-vals CG)
  obtain u v where uv: uv = (u, v) by (cases uv)
  have uok: tap-off-ok (cg-out CG) u using offs uvin uv by auto
  show (case uv of (u, v) ⇒ v * eq-pi z (side-wire (cg-out CG) b u))
    = (case uv of (u, v) ⇒ ?B b * (v * eq-pi (side-lo-pt z (cg-out CG)) u))
  using side-eq-split[OF wo blt uok] uv by (simp add: algebra-simps)
qed
also have ... = ?B b * ?T
  by (simp add: sum-list-const-mult case-prod-unfold)
finally show consts-mle-eval (map (λ(u, v). (side-wire (cg-out CG) b u, v)) (cg-vals CG)) z
  = ?B b * ?T .
qed
also have ... = (∑ b < cg-count CG. ?B b * ?T)
  by (simp add: sum-list-map-upt)
also have ... = (∑ b < cg-count CG. ?B b) * ?T
  by (simp add: sum-distrib-right)
finally show ?thesis by (simp add: cgroup-eval-def)
qed

```

Layer-level re-association: the derived oracle equals the sparse gate sum of its own re-expansion.

lemma *derived-eval-expand*:

```

fixes z x y :: 'f::comm-ring-1 list
assumes wf: ∀ G ∈ set (drv-groups D). wf-group (length z) (length x) G
  and ly: length y = length x
shows derived-eval D k z x y = gates-mle-eval (expand-gates D) k z x y
proof -
  have gates-mle-eval (concat (map expand-group (drv-groups D))) k z x y
    = sum-list (map (λG. gates-mle-eval (expand-group G) k z x y) (drv-groups D))
  unfolding gates-mle-eval-def
  by (simp add: filter-concat map-concat sum-list-concat sum-list-concat-map o-def)
also have ... = sum-list (map (λG. group-block G (k = GMul) z x y * group-template k G z x y)
    (drv-groups D))
  using wf by (intro arg-cong[of - - sum-list] map-cong refl)
    (simp add: group-eval-expand ly)
finally show ?thesis
  by (simp add: derived-eval-def expand-gates-def gates-mle-eval-append)

```

qed

lemma *derived-const-eval-expand*:

fixes $z :: 'f::comm-ring-1$ list

assumes $wf: \forall CG \in \text{set } (drv\text{-}cgroups\ D). wf\text{-}cgroup\ (\text{length } z)\ CG$

shows $derived\text{-}const\text{-}eval\ D\ z = \text{consts}\text{-}mle\text{-}eval\ (\text{expand}\text{-}consts\ D)\ z$

proof –

have $\text{consts}\text{-}mle\text{-}eval\ (\text{concat } (\text{map } \text{expand}\text{-}cgroup\ (drv\text{-}cgroups\ D)))\ z$

$= \text{sum}\text{-}list\ (\text{map } (\lambda CG. \text{consts}\text{-}mle\text{-}eval\ (\text{expand}\text{-}cgroup\ CG)\ z)\ (drv\text{-}cgroups\ D))$

unfolding *consts-mle-eval-def*

by (*simp add: map-concat sum-list-concat sum-list-concat-map o-def*)

also have $\dots = \text{sum}\text{-}list\ (\text{map } (\lambda CG. \text{cgroup}\text{-}eval\ CG\ z)\ (drv\text{-}cgroups\ D))$

using wf **by** (*intro arg-cong[of - - sum-list] map-cong refl*) (*simp add: cgroup-eval-expand*)

finally show *?thesis*

by (*simp add: derived-const-eval-def expand-consts-def consts-mle-eval-append*)

qed

20 Main theorem: exact repartition forces oracle equality (lemma (c))

If re-expanding the derived representation reproduces the layer's gate list AS A MULTISSET (the exact-repartition property, Rust pin 2 of tests/*verify-succinct.rs*), then the derived closed-form oracle and the table oracle agree AT EVERY EVALUATION POINT (Rust pin 1) - for every gate kind, and likewise for the constant vector. Wrong or missing layout hints change only which side of the repartition a gate lands on, never this equality (hints are soundness-inert).

theorem *derived-mle-eq-table*:

fixes $z\ x\ y :: 'f::comm-ring-1$ list

assumes $\text{repart}: \text{mset } (\text{expand}\text{-}gates\ D) = \text{mset } (\text{layer}\text{-}gates\ L)$

and $wf: \forall G \in \text{set } (drv\text{-}groups\ D). wf\text{-}group\ (\text{length } z)\ (\text{length } x)\ G$

and $ly: \text{length } y = \text{length } x$

shows $derived\text{-}eval\ D\ k\ z\ x\ y = \text{gates}\text{-}mle\text{-}eval\ (\text{layer}\text{-}gates\ L)\ k\ z\ x\ y$

proof –

have $derived\text{-}eval\ D\ k\ z\ x\ y = \text{gates}\text{-}mle\text{-}eval\ (\text{expand}\text{-}gates\ D)\ k\ z\ x\ y$

by (*rule derived-eval-expand[OF wf ly]*)

also have $\dots = \text{gates}\text{-}mle\text{-}eval\ (\text{layer}\text{-}gates\ L)\ k\ z\ x\ y$

by (*rule gates-mle-eval-mset-cong[OF repart]*)

finally show *?thesis* .

qed

theorem *derived-const-mle-eq-table*:

fixes $z :: 'f::comm-ring-1$ list

assumes $\text{repart}: \text{mset } (\text{expand}\text{-}consts\ D) = \text{mset } (\text{layer}\text{-}consts\ L)$

and $wf: \forall CG \in \text{set } (drv\text{-}cgroups\ D). wf\text{-}cgroup\ (\text{length } z)\ CG$

shows $derived\text{-}const\text{-}eval\ D\ z = \text{consts}\text{-}mle\text{-}eval\ (\text{layer}\text{-}consts\ L)\ z$

proof –

have $derived\text{-}const\text{-}eval\ D\ z = \text{consts}\text{-}mle\text{-}eval\ (\text{expand}\text{-}consts\ D)\ z$

by (*rule derived-const-eval-expand[OF wf]*)

also have $\dots = \text{consts}\text{-}mle\text{-}eval\ (\text{layer}\text{-}consts\ L)\ z$

by (*rule consts-mle-eval-mset-cong[OF repart]*)

finally show ?thesis .
qed

Corollaries against the DEFINITIONAL table MLE (the *mle* of the tuple table): the derived oracle computes the same polynomial, at every point, for the binary kind, the unary kinds, and the constants.

corollary *derived-eval-is-table-mle3:*

fixes $z\ x\ y :: 'f::comm-ring-1\ list$
 assumes *repart*: $mset\ (expand-gates\ D) = mset\ (layer-gates\ L)$
 and *wf*: $\forall G \in set\ (drv-groups\ D). wf-group\ ob\ ib\ G$
 and *wfl*: $\forall g \in set\ (layer-gates\ L). g-kind\ g = GMul \longrightarrow$
 $g-out\ g < 2 \wedge ob \wedge g-in1\ g < 2 \wedge ib \wedge g-in2\ g < 2 \wedge ib$
 and *lz*: $length\ z = ob$ and *lx*: $length\ x = ib$ and *ly*: $length\ y = ib$
 shows *derived-eval* $D\ GMul\ z\ x\ y$
 $= mle\ (ob + ib + ib)\ (table3\ ib\ (filter\ (\lambda g. g-kind\ g = GMul)\ (layer-gates\ L)))\ (z\ @\ x\ @\ y)$
 using *derived-mle-eq-table*[*of* $D\ L\ z\ x\ y\ GMul$] *table-mle3-gates*[*OF* *wfl* *lz* *lx* *ly*]
repart *wf* *lz* *lx* *ly* **by** *simp*

corollary *derived-eval-is-table-mle2:*

fixes $z\ x\ y :: 'f::comm-ring-1\ list$
 assumes *kk*: $k = GLin \vee k = GPow3$
 and *repart*: $mset\ (expand-gates\ D) = mset\ (layer-gates\ L)$
 and *wf*: $\forall G \in set\ (drv-groups\ D). wf-group\ ob\ ib\ G$
 and *wfl*: $\forall g \in set\ (layer-gates\ L). g-kind\ g = k \longrightarrow$
 $g-out\ g < 2 \wedge ob \wedge g-in1\ g < 2 \wedge ib$
 and *lz*: $length\ z = ob$ and *lx*: $length\ x = ib$ and *ly*: $length\ y = ib$
 shows *derived-eval* $D\ k\ z\ x\ y$
 $= mle\ (ob + ib)\ (table2\ ib\ (filter\ (\lambda g. g-kind\ g = k)\ (layer-gates\ L)))\ (z\ @\ x)$
 using *derived-mle-eq-table*[*of* $D\ L\ z\ x\ y\ k$] *table-mle2-gates*[*OF* *kk* *wfl* *lz* *lx*, *of* *y*]
repart *wf* *lz* *lx* *ly* **by** *simp*

corollary *derived-const-eval-is-const-mle:*

fixes $z :: 'f::comm-ring-1\ list$
 assumes *repart*: $mset\ (expand-consts\ D) = mset\ (layer-consts\ L)$
 and *wf*: $\forall CG \in set\ (drv-cgroups\ D). wf-cgroup\ ob\ CG$
 and *wfl*: $\forall (w, v) \in set\ (layer-consts\ L). w < 2 \wedge ob$
 and *lz*: $length\ z = ob$
 shows *derived-const-eval* $D\ z = mle\ ob\ (const-at\ L)\ z$
proof –
 have $mle\ ob\ (const-at\ L)\ z = mle\ ob\ (ctable\ (layer-consts\ L))\ z$
 by (*intro* *mle-cong*) (*simp* *add*: *const-at-ctable*)
 also have $\dots = consts-mle-eval\ (layer-consts\ L)\ z$
 by (*rule* *const-mle-eq*[*OF* *wfl* *lz*])
 finally show ?thesis
 using *derived-const-mle-eq-table*[*OF* *repart*] *wf* *lz* **by** *simp*
 qed

21 Uniform-block closed form (the *shifted-range-eq* specification lemma)

The full-range two-term instance of the shifted-range sum has the product closed form the Rust code calls *eq-points*: summing the diagonal weights over the whole cube collapses to a per-variable product. (This is the *with all bases zero and count* = 2^m reduction of the wiring.rs module doc; the partial-range carry-DP closed form itself is an evaluation algorithm, out of the model's scope.)

```

lemma srange-full-two-terms:
  fixes  $p\ q :: 'f::comm-ring-1\ list$ 
  assumes  $length\ p = length\ q$ 
  shows  $(\sum b < 2 \wedge length\ p. eq\_pi\ p\ b * eq\_pi\ q\ b)$ 
     $= (\prod j < length\ p. p!\ j * q!\ j + (1 - p!\ j) * (1 - q!\ j))$ 
  using assms
proof (induction p q rule: list-induct2)
  case Nil
  show ?case by simp
next
  case (Cons a p c q)
  have lpg:  $length\ p = length\ q$  by fact
  have split:  $(\sum b < 2 \wedge length\ (a \# p). eq\_pi\ (a \# p)\ b * eq\_pi\ (c \# q)\ b)$ 
     $= (\sum b < 2 \wedge length\ p. eq\_pi\ (a \# p)\ b * eq\_pi\ (c \# q)\ b)$ 
     $+ (\sum b < 2 \wedge length\ p. eq\_pi\ (a \# p)\ (b + 2 \wedge length\ p) * eq\_pi\ (c \# q)\ (b + 2 \wedge length\ p))$ 
  proof -
  have decomp:  $\{..< (2 :: nat) \wedge Suc\ (length\ p)\}$ 
     $= \{..< 2 \wedge length\ p\} \cup (\lambda i. i + 2 \wedge length\ p) \cdot \{..< 2 \wedge length\ p\}$ 
  proof
  show  $\{..< (2 :: nat) \wedge Suc\ (length\ p)\}$ 
     $\subseteq \{..< 2 \wedge length\ p\} \cup (\lambda i. i + 2 \wedge length\ p) \cdot \{..< 2 \wedge length\ p\}$ 
  proof
  fix  $v$  assume  $v \in \{..< (2 :: nat) \wedge Suc\ (length\ p)\}$ 
  then have  $v < 2 * 2 \wedge length\ p$  by simp
  then show  $v \in \{..< 2 \wedge length\ p\} \cup (\lambda i. i + 2 \wedge length\ p) \cdot \{..< 2 \wedge length\ p\}$ 
  proof (cases v < 2 \wedge length\ p)
  case False
  then have  $v - 2 \wedge length\ p < 2 \wedge length\ p$  and  $v = (v - 2 \wedge length\ p) + 2 \wedge length\ p$ 
  using  $\langle v < 2 * 2 \wedge length\ p \rangle$  by simp-all
  then show ?thesis by force
  qed simp
  qed
next
  show  $\{..< 2 \wedge length\ p\} \cup (\lambda i. i + 2 \wedge length\ p) \cdot \{..< 2 \wedge length\ p\}$ 
     $\subseteq \{..< (2 :: nat) \wedge Suc\ (length\ p)\}$  by auto
  qed
  have inj:  $inj\_on\ (\lambda i. i + (2 :: nat) \wedge length\ p)\ \{..< 2 \wedge length\ p\}$ 
    by (simp add: inj-on-def)
  have disj:  $\{..< (2 :: nat) \wedge length\ p\} \cap (\lambda i. i + 2 \wedge length\ p) \cdot \{..< 2 \wedge length\ p\} = \{\}$ 
    by auto
  have  $(\sum b < 2 \wedge length\ (a \# p). eq\_pi\ (a \# p)\ b * eq\_pi\ (c \# q)\ b)$ 
     $= (\sum b \in \{..< 2 \wedge length\ p\} \cup (\lambda i. i + 2 \wedge length\ p) \cdot \{..< 2 \wedge length\ p\}.$ 
     $eq\_pi\ (a \# p)\ b * eq\_pi\ (c \# q)\ b)$ 

```

by (simp only: length-Cons decomp)
 also have ... = $(\sum b < 2 \wedge \text{length } p. \text{eq-pi } (a \# p) \ b * \text{eq-pi } (c \# q) \ b)$
 $+ (\sum b \in (\lambda i. i + 2 \wedge \text{length } p) \ ' \{.. < 2 \wedge \text{length } p\}.$
 $\text{eq-pi } (a \# p) \ b * \text{eq-pi } (c \# q) \ b)$
 by (rule sum.union-disjoint) (use disj in simp-all)
 also have $(\sum b \in (\lambda i. i + 2 \wedge \text{length } p) \ ' \{.. < 2 \wedge \text{length } p\}.$
 $\text{eq-pi } (a \# p) \ b * \text{eq-pi } (c \# q) \ b)$
 = $(\sum b < 2 \wedge \text{length } p.$
 $\text{eq-pi } (a \# p) \ (b + 2 \wedge \text{length } p) * \text{eq-pi } (c \# q) \ (b + 2 \wedge \text{length } p))$
 by (subst sum.reindex[OF inj]) (simp add: o-def)
 finally show ?thesis .
 qed
 have lo: $(\sum b < 2 \wedge \text{length } p. \text{eq-pi } (a \# p) \ b * \text{eq-pi } (c \# q) \ b)$
 = $(1 - a) * (1 - c) * (\sum b < 2 \wedge \text{length } p. \text{eq-pi } p \ b * \text{eq-pi } q \ b)$
 proof -
 have $(\sum b < 2 \wedge \text{length } p. \text{eq-pi } (a \# p) \ b * \text{eq-pi } (c \# q) \ b)$
 = $(\sum b < 2 \wedge \text{length } p. ((1 - a) * (1 - c)) * (\text{eq-pi } p \ b * \text{eq-pi } q \ b))$
 by (intro sum.cong refl) (simp add: eq-pi-cons-lo lpq algebra-simps)
 then show ?thesis
 by (simp add: sum-distrib-left algebra-simps)
 qed
 have hi: $(\sum b < 2 \wedge \text{length } p. \text{eq-pi } (a \# p) \ (b + 2 \wedge \text{length } p) * \text{eq-pi } (c \# q) \ (b + 2 \wedge \text{length } p))$
 = $a * c * (\sum b < 2 \wedge \text{length } p. \text{eq-pi } p \ b * \text{eq-pi } q \ b)$
 proof -
 have $(\sum b < 2 \wedge \text{length } p. \text{eq-pi } (a \# p) \ (b + 2 \wedge \text{length } p) * \text{eq-pi } (c \# q) \ (b + 2 \wedge \text{length } p))$
 = $(\sum b < 2 \wedge \text{length } p. (a * c) * (\text{eq-pi } p \ b * \text{eq-pi } q \ b))$
 proof (intro sum.cong refl)
 fix b assume $b \in \{.. < (2 :: \text{nat}) \wedge \text{length } p\}$
 then have blt: $b < 2 \wedge \text{length } p$ by simp
 have bltq: $b < 2 \wedge \text{length } q$ using blt lpq by simp
 have e1: $\text{eq-pi } (a \# p) \ (b + 2 \wedge \text{length } p) = a * \text{eq-pi } p \ b$
 by (rule eq-pi-cons-hi[OF blt])
 have e2: $\text{eq-pi } (c \# q) \ (b + 2 \wedge \text{length } p) = c * \text{eq-pi } q \ b$
 using lpq bltq by (simp add: eq-pi-cons-hi)
 show $\text{eq-pi } (a \# p) \ (b + 2 \wedge \text{length } p) * \text{eq-pi } (c \# q) \ (b + 2 \wedge \text{length } p)$
 = $(a * c) * (\text{eq-pi } p \ b * \text{eq-pi } q \ b)$
 by (simp add: e1 e2 algebra-simps)
 qed
 then show ?thesis
 by (simp add: sum-distrib-left algebra-simps)
 qed
 have prod-step: $(\prod j < \text{length } (a \# p). (a \# p) ! j * (c \# q) ! j$
 $+ (1 - (a \# p) ! j) * (1 - (c \# q) ! j))$
 = $(a * c + (1 - a) * (1 - c)) * (\prod j < \text{length } p. p ! j * q ! j + (1 - p ! j) * (1 - q ! j))$
 by (simp only: length-Cons prod-lessThan-Suc-shift nth-Cons-0 nth-Cons-Suc)
 show ?case
 unfolding split lo hi Cons.IH prod-step
 by (simp add: algebra-simps)
 qed

22 Activation instance (the theorem fires on a concrete layer)

A width-4 layer (2 output bits, 2 input bits) assembled from TWO derived families plus ONE sparse leftover gate, against the same gates listed in a different order in the layer. Family 1: two blocks of one Lin tap on stride-2 progressions. Family 2: one Mul block on window progressions. Sparse: one Pow3 gate. The repartition holds as a multiset (the lists are permutations), so the main theorem applies at every point.

definition *demo-group1* :: int gate-group **where**

```
demo-group1 =
  (| grp-count = 2, grp-out = Prog 0 1, grp-in1 = Prog 0 1, grp-in2 = Fixed 3,
    grp-taps = [| tap-kind = GLin, tap-out = 1, tap-in1 = 0, tap-in2 = 0, tap-coeff = 3 |] |)
```

definition *demo-group2* :: int gate-group **where**

```
demo-group2 =
  (| grp-count = 1, grp-out = Prog 1 1, grp-in1 = Prog 0 2, grp-in2 = Prog 0 2,
    grp-taps = [| tap-kind = GMul, tap-out = 0, tap-in1 = 1, tap-in2 = 2, tap-coeff = 1 |] |)
```

definition *demo-sparse-gate* :: int gate **where**

```
demo-sparse-gate = (| g-kind = GPow3, g-out = 0, g-in1 = 3, g-in2 = 3, g-coeff = 5 |)
```

definition *demo-derived* :: int derived-layer **where**

```
demo-derived =
  (| drv-groups = [demo-group1, demo-group2],
    drv-sparse = [demo-sparse-gate],
    drv-cgroups = [| cg-count = 2, cg-out = Prog 0 1, cg-vals = [(0, 7)] |],
    drv-csparse = [(3, 11)] |)
```

The same gates and constants, hand-listed in a shuffled order.

definition *demo-layer-c* :: int layer **where**

```
demo-layer-c =
  (| layer-width-bits = 2,
    layer-gates =
      [ (| g-kind = GMul, g-out = 2, g-in1 = 1, g-in2 = 2, g-coeff = 1 |),
        (| g-kind = GLin, g-out = 1, g-in1 = 0, g-in2 = 0, g-coeff = 3 |),
        demo-sparse-gate,
        (| g-kind = GLin, g-out = 3, g-in1 = 2, g-in2 = 2, g-coeff = 3 |) ],
    layer-consts = [(3, 11), (0, 7), (2, 7)] |)
```

lemma *demo-expand-gates*:

```
expand-gates demo-derived =
  [ (| g-kind = GLin, g-out = 1, g-in1 = 0, g-in2 = 0, g-coeff = 3 |),
    (| g-kind = GLin, g-out = 3, g-in1 = 2, g-in2 = 2, g-coeff = 3 |),
    (| g-kind = GMul, g-out = 2, g-in1 = 1, g-in2 = 2, g-coeff = 1 |),
    demo-sparse-gate ]
```

by (simp add: expand-gates-def demo-derived-def expand-group-def
demo-group1-def demo-group2-def tap-gate-def upt-rec)

lemma *demo-expand-consts*:

```
expand-consts demo-derived = [(0, 7), (2, 7), (3, 11)]
```

by (simp add: expand-consts-def demo-derived-def expand-cgroup-def upt-rec)

lemma *demo-repartition*:

```

mset (expand-gates demo-derived) = mset (layer-gates demo-layer-c)
by (simp add: demo-expand-gates demo-layer-c-def add-mset-commute)

```

lemma *demo-repartition-consts*:

```

mset (expand-consts demo-derived) = mset (layer-consts demo-layer-c)
by (simp add: demo-expand-consts demo-layer-c-def add-mset-commute)

```

lemma *demo-wf-groups*:

```

∀ G ∈ set (drv-groups demo-derived). wf-group 2 2 G
by (simp add: demo-derived-def demo-group1-def demo-group2-def wf-group-def)

```

lemma *demo-wf-cgroups*:

```

∀ CG ∈ set (drv-cgroups demo-derived). wf-cgroup 2 CG
by (simp add: demo-derived-def wf-cgroup-def)

```

The equivalence fires: at EVERY evaluation point (arbitrary field lists of the layer's widths) and for EVERY gate kind, the derived closed-form oracle of the demo representation equals the table oracle of the shuffled layer - and likewise for the constant vector.

lemma *demo-derived-eq-table*:

```

fixes z x y :: int list
assumes length z = 2 and length x = 2 and length y = 2
shows derived-eval demo-derived k z x y = gates-mle-eval (layer-gates demo-layer-c) k z x y
by (rule derived-mle-eq-table[OF demo-repartition])
   (use assms demo-wf-groups in simp-all)

```

lemma *demo-derived-const-eq-table*:

```

fixes z :: int list
assumes length z = 2
shows derived-const-eval demo-derived z = consts-mle-eval (layer-consts demo-layer-c) z
by (rule derived-const-mle-eq-table[OF demo-repartition-consts])
   (use assms demo-wf-cgroups in simp-all)

```

And against the definitional MLE, at a sample point: sanity that the layer's Mul table reads back the gate coefficient on the hypercube (the boolean-point agreement of the packed tuple table).

lemma *demo-table3-bool-point*:

```

mle (2 + 2 + 2) (table3 2 (filter (λg. g-kind g = GMul) (layer-gates demo-layer-c)))
  (bool-point (2 + 2 + 2) ((2 * 2 ^ 2 + 1) * 2 ^ 2 + 2)) = (1 :: int)

```

proof –

```

have enc-lt: (2 * 2 ^ 2 + 1) * 2 ^ 2 + 2 < (2 :: nat) ^ (2 + 2 + 2) by simp
have mle (2 + 2 + 2) (table3 2 (filter (λg. g-kind g = GMul) (layer-gates demo-layer-c)))
  (bool-point (2 + 2 + 2) ((2 * 2 ^ 2 + 1) * 2 ^ 2 + 2) :: int list)
  = table3 2 (filter (λg. g-kind g = GMul) (layer-gates demo-layer-c))
    ((2 * 2 ^ 2 + 1) * 2 ^ 2 + 2)
  by (rule mle-bool-point[OF enc-lt])
also have ... = 1
  by (simp add: demo-layer-c-def demo-sparse-gate-def table3-def enc3-def)
finally show ?thesis .

```

qed

end

```

theory Sumcheck-Instance
imports
  Layered-Circuit
  Multilinear-Extension
  Sumcheck-Protocol.Completeness-Proof
  Sumcheck-Protocol.Soundness-Proof
  Sumcheck-Protocol.Concrete-Multivariate-Polynomials
begin

```

23 Per-kind wiring weight tables (gate-list semantics)

The coefficient booked by a gate list for one (kind, out, in-wire) cell. Duplicated tuples accumulate (list semantics): this is the boolean-point restriction of the Rust wiring oracle, where `TableWiring` sums $\text{coeff} * \text{eq}(z, \text{out}) * \text{eq}(x, \text{in1}) [* \text{eq}(y, \text{in2})]$ over the gate list.

```

definition lin-wt :: 'f::comm-ring-1 gate list  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f where
  lin-wt gs z x =
    sum-list (map g-coeff
      (filter ( $\lambda g. g\text{-kind } g = \text{GLin} \wedge g\text{-out } g = z \wedge g\text{-in1 } g = x$ ) gs))

```

```

definition mul-wt :: 'f::comm-ring-1 gate list  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f where
  mul-wt gs z x y =
    sum-list (map g-coeff
      (filter ( $\lambda g. g\text{-kind } g = \text{GMul} \wedge g\text{-out } g = z \wedge g\text{-in1 } g = x \wedge g\text{-in2 } g = y$ ) gs))

```

```

definition pow3-wt :: 'f::comm-ring-1 gate list  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f where
  pow3-wt gs z x =
    sum-list (map g-coeff
      (filter ( $\lambda g. g\text{-kind } g = \text{GPow3} \wedge g\text{-out } g = z \wedge g\text{-in1 } g = x$ ) gs))

```

24 Splitting a layer's gate sum by kind

The wire-*z* gate contribution sum of *layer-eval*, regrouped into the three per-kind weight tables. This is the boolean-point core of the layer identity: each gate's contribution is recovered by a Kronecker selection over the input cube.

```

lemma gate-sum-kind-split:
  fixes gs :: 'f::comm-ring-1 gate list and below :: 'f list and z :: nat
  assumes  $\forall g \in \text{set } gs. g\text{-in1 } g < 2^{\wedge s} \wedge g\text{-in2 } g < 2^{\wedge s}$ 
  shows sum-list (map ( $\lambda g. \text{gate-contrib } g \text{ below}$ ) (filter ( $\lambda g. g\text{-out } g = z$ ) gs))
    = ( $\sum x < 2^{\wedge s}. \text{lin-wt } gs \text{ } z \text{ } x * \text{below ! } x$ )
    + ( $\sum x < 2^{\wedge s}. \sum y < 2^{\wedge s}. \text{mul-wt } gs \text{ } z \text{ } x \text{ } y * (\text{below ! } x * \text{below ! } y)$ )
    + ( $\sum x < 2^{\wedge s}. \text{pow3-wt } gs \text{ } z \text{ } x * (\text{below ! } x * \text{below ! } x * \text{below ! } x)$ )
  using assms
proof (induction gs)
  case Nil
  show ?case by (simp add: lin-wt-def mul-wt-def pow3-wt-def)
next
  case (Cons g gs)
  have in-range:  $\forall g' \in \text{set } gs. g\text{-in1 } g' < 2^{\wedge s} \wedge g\text{-in2 } g' < 2^{\wedge s}$ 
  using Cons.premis by simp

```

```

have in1:  $g\text{-in1 } g < 2^s$  and in2:  $g\text{-in2 } g < 2^s$ 
  using Cons.prem by simp-all
note IH = Cons.IH[OF in-range]
show ?case
proof (cases  $g\text{-out } g = z$ )
  case False
  then show ?thesis using IH
    by (simp add: lin-wt-def mul-wt-def pow3-wt-def)
next
  case True
  show ?thesis
  proof (cases  $g\text{-kind } g$ )
    case GLin
    have lw:  $\bigwedge x. \text{lin-wt } (g \# gs) z x =$ 
       $(\text{if } g\text{-in1 } g = x \text{ then } g\text{-coeff } g \text{ else } 0) + \text{lin-wt } gs z x$ 
      using True GLin by (simp add: lin-wt-def)
    have mw:  $\bigwedge x y. \text{mul-wt } (g \# gs) z x y = \text{mul-wt } gs z x y$ 
      using GLin by (simp add: mul-wt-def)
    have pw:  $\bigwedge x. \text{pow3-wt } (g \# gs) z x = \text{pow3-wt } gs z x$ 
      using GLin by (simp add: pow3-wt-def)
    have delta:  $(\sum x < 2^s. (\text{if } g\text{-in1 } g = x \text{ then } g\text{-coeff } g \text{ else } 0) * \text{below } ! x)$ 
       $= g\text{-coeff } g * \text{below } ! (g\text{-in1 } g)$ 
    proof -
      have  $(\sum x < 2^s. (\text{if } g\text{-in1 } g = x \text{ then } g\text{-coeff } g \text{ else } 0) * \text{below } ! x)$ 
         $= (\sum x < 2^s. (\text{if } g\text{-in1 } g = x \text{ then } g\text{-coeff } g * \text{below } ! x \text{ else } 0))$ 
        by (intro sum.cong refl) simp
      also have  $\dots = g\text{-coeff } g * \text{below } ! (g\text{-in1 } g)$ 
        using in1 by (simp add: sum.delta')
      finally show ?thesis .
    qed
    have  $(\sum x < 2^s. \text{lin-wt } (g \# gs) z x * \text{below } ! x)$ 
       $= g\text{-coeff } g * \text{below } ! (g\text{-in1 } g) + (\sum x < 2^s. \text{lin-wt } gs z x * \text{below } ! x)$ 
      by (simp add: lw distrib-right sum.distrib delta)
    then show ?thesis using True GLin IH
      by (simp add: mw pw gate-contrib-def algebra-simps)
  next
    case GMul
    have lw:  $\bigwedge x. \text{lin-wt } (g \# gs) z x = \text{lin-wt } gs z x$ 
      using GMul by (simp add: lin-wt-def)
    have mw:  $\bigwedge x y. \text{mul-wt } (g \# gs) z x y =$ 
       $(\text{if } g\text{-in1 } g = x \wedge g\text{-in2 } g = y \text{ then } g\text{-coeff } g \text{ else } 0) + \text{mul-wt } gs z x y$ 
      using True GMul by (simp add: mul-wt-def)
    have pw:  $\bigwedge x. \text{pow3-wt } (g \# gs) z x = \text{pow3-wt } gs z x$ 
      using GMul by (simp add: pow3-wt-def)
    have inner:  $\bigwedge x. (\sum y < 2^s.$ 
       $(\text{if } g\text{-in1 } g = x \wedge g\text{-in2 } g = y \text{ then } g\text{-coeff } g \text{ else } 0) * (\text{below } ! x * \text{below } ! y))$ 
       $= (\text{if } g\text{-in1 } g = x \text{ then } g\text{-coeff } g * (\text{below } ! x * \text{below } ! (g\text{-in2 } g)) \text{ else } 0)$ 
    proof -
      fix x
      show  $(\sum y < 2^s.$ 
         $(\text{if } g\text{-in1 } g = x \wedge g\text{-in2 } g = y \text{ then } g\text{-coeff } g \text{ else } 0) * (\text{below } ! x * \text{below } ! y))$ 
         $= (\text{if } g\text{-in1 } g = x \text{ then } g\text{-coeff } g * (\text{below } ! x * \text{below } ! (g\text{-in2 } g)) \text{ else } 0)$ 
      proof (cases  $g\text{-in1 } g = x$ )

```

```

case True
have  $(\sum y < 2^{\wedge}s. (if\ g-in2\ g = y\ then\ g-coeff\ g\ else\ 0) * (below\ !\ x * below\ !\ y))$ 
  =  $(\sum y < 2^{\wedge}s. (if\ g-in2\ g = y\ then\ g-coeff\ g * (below\ !\ x * below\ !\ y)\ else\ 0))$ 
  by (intro sum.cong refl) simp
also have ... =  $g-coeff\ g * (below\ !\ x * below\ !\ (g-in2\ g))$ 
  using in2 by (simp add: sum.delta')
finally show ?thesis using True by simp
next
case False
then show ?thesis by simp
qed
qed
have delta2:  $(\sum x < 2^{\wedge}s. \sum y < 2^{\wedge}s.$ 
   $(if\ g-in1\ g = x \wedge g-in2\ g = y\ then\ g-coeff\ g\ else\ 0) * (below\ !\ x * below\ !\ y))$ 
  =  $g-coeff\ g * (below\ !\ (g-in1\ g) * below\ !\ (g-in2\ g))$ 
  using in1 by (simp add: inner sum.delta')
have  $(\sum x < 2^{\wedge}s. \sum y < 2^{\wedge}s. mul-wt\ (g\ \# \ gs)\ z\ x\ y * (below\ !\ x * below\ !\ y))$ 
  =  $g-coeff\ g * (below\ !\ (g-in1\ g) * below\ !\ (g-in2\ g))$ 
  +  $(\sum x < 2^{\wedge}s. \sum y < 2^{\wedge}s. mul-wt\ gs\ z\ x\ y * (below\ !\ x * below\ !\ y))$ 
  by (simp add: mw distrib-right sum.distrib delta2)
then show ?thesis using True GMul IH
  by (simp add: lw pw gate-contrib-def algebra-simps)
next
case GPow3
have lw:  $\bigwedge x. lin-wt\ (g\ \# \ gs)\ z\ x = lin-wt\ gs\ z\ x$ 
  using GPow3 by (simp add: lin-wt-def)
have mw:  $\bigwedge x\ y. mul-wt\ (g\ \# \ gs)\ z\ x\ y = mul-wt\ gs\ z\ x\ y$ 
  using GPow3 by (simp add: mul-wt-def)
have pw:  $\bigwedge x. pow3-wt\ (g\ \# \ gs)\ z\ x =$ 
   $(if\ g-in1\ g = x\ then\ g-coeff\ g\ else\ 0) + pow3-wt\ gs\ z\ x$ 
  using True GPow3 by (simp add: pow3-wt-def)
have delta:  $(\sum x < 2^{\wedge}s. (if\ g-in1\ g = x\ then\ g-coeff\ g\ else\ 0)$ 
   $* (below\ !\ x * below\ !\ x * below\ !\ x))$ 
  =  $g-coeff\ g * (below\ !\ (g-in1\ g) * below\ !\ (g-in1\ g) * below\ !\ (g-in1\ g))$ 
proof -
  have  $(\sum x < 2^{\wedge}s. (if\ g-in1\ g = x\ then\ g-coeff\ g\ else\ 0)$ 
   $* (below\ !\ x * below\ !\ x * below\ !\ x))$ 
  =  $(\sum x < 2^{\wedge}s. (if\ g-in1\ g = x$ 
   $then\ g-coeff\ g * (below\ !\ x * below\ !\ x * below\ !\ x)\ else\ 0))$ 
  by (intro sum.cong refl) simp
also have ... =  $g-coeff\ g * (below\ !\ (g-in1\ g) * below\ !\ (g-in1\ g) * below\ !\ (g-in1\ g))$ 
  using in1 by (simp add: sum.delta')
finally show ?thesis .
qed
have  $(\sum x < 2^{\wedge}s. pow3-wt\ (g\ \# \ gs)\ z\ x * (below\ !\ x * below\ !\ x * below\ !\ x))$ 
  =  $g-coeff\ g * (below\ !\ (g-in1\ g) * below\ !\ (g-in1\ g) * below\ !\ (g-in1\ g))$ 
  +  $(\sum x < 2^{\wedge}s. pow3-wt\ gs\ z\ x * (below\ !\ x * below\ !\ x * below\ !\ x))$ 
  by (simp add: pw distrib-right sum.distrib delta)
then show ?thesis using True GPow3 IH
  by (simp add: lw mw gate-contrib-def algebra-simps)
qed
qed
qed

```

25 Claim-point lifting of the weight tables

Multilinear extension in the OUTPUT-wire coordinate only: the claim point zpt is a general field point, the input wires x (and y) stay boolean indices. Unfolding mle , $lin\text{-}mle$ $L\ zpt\ x = \text{sum over } Lin\ \text{gates with } in1 = x\ \text{of } coeff * eq\text{-}pi\ zpt\ (out)$, which is exactly $TableWiring::eval\text{-}predicate\text{-}mle$ at boolean x (where $eq(x, in1)$ is the Kronecker delta). Same for mul (boolean x, y) and $pow3$; $const\text{-}mle$ is $eval\text{-}const\text{-}mle$ verbatim.

definition $lin\text{-}mle :: 'f::comm\text{-}ring\text{-}1\ layer \Rightarrow 'f\ list \Rightarrow nat \Rightarrow 'f\ \mathbf{where}$
 $lin\text{-}mle\ L\ zpt\ x = mle\ (layer\text{-}width\text{-}bits\ L)\ (\lambda z. lin\text{-}wt\ (layer\text{-}gates\ L)\ z\ x)\ zpt$

definition $mul\text{-}mle :: 'f::comm\text{-}ring\text{-}1\ layer \Rightarrow 'f\ list \Rightarrow nat \Rightarrow nat \Rightarrow 'f\ \mathbf{where}$
 $mul\text{-}mle\ L\ zpt\ x\ y = mle\ (layer\text{-}width\text{-}bits\ L)\ (\lambda z. mul\text{-}wt\ (layer\text{-}gates\ L)\ z\ x\ y)\ zpt$

definition $pow3\text{-}mle :: 'f::comm\text{-}ring\text{-}1\ layer \Rightarrow 'f\ list \Rightarrow nat \Rightarrow 'f\ \mathbf{where}$
 $pow3\text{-}mle\ L\ zpt\ x = mle\ (layer\text{-}width\text{-}bits\ L)\ (\lambda z. pow3\text{-}wt\ (layer\text{-}gates\ L)\ z\ x)\ zpt$

definition $const\text{-}mle :: 'f::comm\text{-}ring\text{-}1\ layer \Rightarrow 'f\ list \Rightarrow 'f\ \mathbf{where}$
 $const\text{-}mle\ L\ zpt = mle\ (layer\text{-}width\text{-}bits\ L)\ (const\text{-}at\ L)\ zpt$

The layer-value MLE, split along the three gate kinds. Instantiating zpt with a boolean point recovers the native layer identity of *Layered-Circuit*; at a general point it is the wiring identity $V\text{-}i^{\sim}(z) = const^{\sim}(z) + \text{sum-}x\ lin^{\sim}\ V + \text{sum-}xy\ mul^{\sim}\ V\ V + \text{sum-}x\ pow3^{\sim}\ V^{\sim}3$ of the `reduce.rs` module documentation.

lemma *layer-value-mle-split*:

fixes $L :: 'f::comm\text{-}ring\text{-}1\ layer$ **and** $below :: 'f\ list$
assumes $gates\text{-}in: \forall g \in set\ (layer\text{-}gates\ L). g\text{-}in1\ g < 2^{\wedge} s \wedge g\text{-}in2\ g < 2^{\wedge} s$
shows $mle\ (layer\text{-}width\text{-}bits\ L)\ (!)\ (layer\text{-}eval\ L\ below))\ zpt$
 $= const\text{-}mle\ L\ zpt$
 $+ ((\sum x < 2^{\wedge} s. lin\text{-}mle\ L\ zpt\ x * below\ !\ x)$
 $+ (\sum x < 2^{\wedge} s. \sum y < 2^{\wedge} s. mul\text{-}mle\ L\ zpt\ x\ y * (below\ !\ x * below\ !\ y))$
 $+ (\sum x < 2^{\wedge} s. pow3\text{-}mle\ L\ zpt\ x * (below\ !\ x * below\ !\ x * below\ !\ x)))$

proof –

let $?n = layer\text{-}width\text{-}bits\ L$
let $?A = \lambda z. (\sum x < 2^{\wedge} s. lin\text{-}wt\ (layer\text{-}gates\ L)\ z\ x * below\ !\ x)$
let $?B = \lambda z. (\sum x < 2^{\wedge} s. \sum y < 2^{\wedge} s. mul\text{-}wt\ (layer\text{-}gates\ L)\ z\ x\ y * (below\ !\ x * below\ !\ y))$
let $?C = \lambda z. (\sum x < 2^{\wedge} s. pow3\text{-}wt\ (layer\text{-}gates\ L)\ z\ x * (below\ !\ x * below\ !\ x * below\ !\ x))$

have $nth\text{-}eval: \bigwedge z. z < 2^{\wedge} ?n \implies$

$layer\text{-}eval\ L\ below\ !\ z = const\text{-}at\ L\ z + (?A\ z + ?B\ z + ?C\ z)$

proof –

fix $z :: nat$ **assume** $z < 2^{\wedge} ?n$
then have $z < layer\text{-}width\ L$ **by** (*simp add: layer-width-def*)
then show $layer\text{-}eval\ L\ below\ !\ z = const\text{-}at\ L\ z + (?A\ z + ?B\ z + ?C\ z)$
using *gate-sum-kind-split*[*OF gates-in*]
by (*simp add: layer-eval-nth add.assoc*)

qed

have $mle\ ?n\ (!)\ (layer\text{-}eval\ L\ below))\ zpt$
 $= (\sum z < 2^{\wedge} ?n. eq\text{-}pi\ zpt\ z * (const\text{-}at\ L\ z + (?A\ z + ?B\ z + ?C\ z)))$
unfolding *mle-def* **by** (*intro sum.cong refl*) (*simp add: nth-eval*)
also have $\dots = (\sum z < 2^{\wedge} ?n. eq\text{-}pi\ zpt\ z * const\text{-}at\ L\ z)$

$$\begin{aligned}
& + ((\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?A \ z) \\
& + (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?B \ z) \\
& + (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?C \ z)) \\
& \text{by (simp add: distrib-left sum.distrib)} \\
& \text{also have } (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * \text{const-at } L \ z) = \text{const-mle } L \ \text{zpt} \\
& \text{by (simp add: const-mle-def mle-def)} \\
& \text{also have } (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?A \ z) \\
& \quad = (\sum x < 2^s. \text{lin-mle } L \ \text{zpt } x * \text{below } ! \ x) \\
& \text{proof -} \\
& \text{have } (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?A \ z) \\
& \quad = (\sum z < 2^{?n}. \sum x < 2^s. \text{eq-pi } \text{zpt } z * \text{lin-wt (layer-gates } L) \ z \ x * \text{below } ! \ x) \\
& \text{by (simp add: sum-distrib-left mult.assoc)} \\
& \text{also have } \dots = (\sum x < 2^s. \sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * \text{lin-wt (layer-gates } L) \ z \ x * \text{below } ! \ x) \\
& \text{by (rule sum.swap)} \\
& \text{also have } \dots = (\sum x < 2^s. (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * \text{lin-wt (layer-gates } L) \ z \ x) * \text{below } ! \ x) \\
& \text{by (simp add: sum-distrib-right)} \\
& \text{also have } \dots = (\sum x < 2^s. \text{lin-mle } L \ \text{zpt } x * \text{below } ! \ x) \\
& \text{by (simp add: lin-mle-def mle-def)} \\
& \text{finally show ?thesis .} \\
& \text{qed} \\
& \text{also have } (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?B \ z) \\
& \quad = (\sum x < 2^s. \sum y < 2^s. \text{mul-mle } L \ \text{zpt } x \ y * (\text{below } ! \ x * \text{below } ! \ y)) \\
& \text{proof -} \\
& \text{have } (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?B \ z) \\
& \quad = (\sum z < 2^{?n}. \sum x < 2^s. \sum y < 2^s. \\
& \quad \quad \text{eq-pi } \text{zpt } z * \text{mul-wt (layer-gates } L) \ z \ x \ y * (\text{below } ! \ x * \text{below } ! \ y)) \\
& \text{by (simp add: sum-distrib-left mult.assoc)} \\
& \text{also have } \dots = (\sum x < 2^s. \sum z < 2^{?n}. \sum y < 2^s. \\
& \quad \quad \text{eq-pi } \text{zpt } z * \text{mul-wt (layer-gates } L) \ z \ x \ y * (\text{below } ! \ x * \text{below } ! \ y)) \\
& \text{by (rule sum.swap)} \\
& \text{also have } \dots = (\sum x < 2^s. \sum y < 2^s. \sum z < 2^{?n}. \\
& \quad \quad \text{eq-pi } \text{zpt } z * \text{mul-wt (layer-gates } L) \ z \ x \ y * (\text{below } ! \ x * \text{below } ! \ y)) \\
& \text{by (intro sum.cong refl sum.swap)} \\
& \text{also have } \dots = (\sum x < 2^s. \sum y < 2^s. \\
& \quad \quad (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * \text{mul-wt (layer-gates } L) \ z \ x \ y) * (\text{below } ! \ x * \text{below } ! \ y)) \\
& \text{by (simp add: sum-distrib-right)} \\
& \text{also have } \dots = (\sum x < 2^s. \sum y < 2^s. \text{mul-mle } L \ \text{zpt } x \ y * (\text{below } ! \ x * \text{below } ! \ y)) \\
& \text{by (simp add: mul-mle-def mle-def)} \\
& \text{finally show ?thesis .} \\
& \text{qed} \\
& \text{also have } (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?C \ z) \\
& \quad = (\sum x < 2^s. \text{pow3-mle } L \ \text{zpt } x * (\text{below } ! \ x * \text{below } ! \ x * \text{below } ! \ x)) \\
& \text{proof -} \\
& \text{have } (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * ?C \ z) \\
& \quad = (\sum z < 2^{?n}. \sum x < 2^s. \\
& \quad \quad \text{eq-pi } \text{zpt } z * \text{pow3-wt (layer-gates } L) \ z \ x * (\text{below } ! \ x * \text{below } ! \ x * \text{below } ! \ x)) \\
& \text{by (simp add: sum-distrib-left mult.assoc)} \\
& \text{also have } \dots = (\sum x < 2^s. \sum z < 2^{?n}. \\
& \quad \quad \text{eq-pi } \text{zpt } z * \text{pow3-wt (layer-gates } L) \ z \ x * (\text{below } ! \ x * \text{below } ! \ x * \text{below } ! \ x)) \\
& \text{by (rule sum.swap)} \\
& \text{also have } \dots = (\sum x < 2^s. \\
& \quad \quad (\sum z < 2^{?n}. \text{eq-pi } \text{zpt } z * \text{pow3-wt (layer-gates } L) \ z \ x) \\
& \quad \quad * (\text{below } ! \ x * \text{below } ! \ x * \text{below } ! \ x))
\end{aligned}$$

```

    by (simp add: sum-distrib-right)
  also have ... = (∑ x < 2^s. pow3-mle L zpt x * (below ! x * below ! x * below ! x))
    by (simp add: pow3-mle-def mle-def)
  finally show ?thesis .
qed
finally show ?thesis .
qed

```

26 The layer hypercube identity (B1)

$\text{beta-idx } s \ y$ = the all-zero indicator beta at the boolean point of index y (Rust beta-eval at a boolean point). On the cube it is the Kronecker delta at 0 and its cube sum is 1, which is what lifts the unary (lin/pow3) terms onto the (x, y) hypercube.

definition $\text{beta-idx} :: \text{nat} \Rightarrow \text{nat} \Rightarrow 'f::\text{comm-ring-1}$ **where**
 $\text{beta-idx } s \ y = \text{beta } (\text{bool-point } s \ y)$

lemma beta-idx-delta :
assumes $y < 2^s$
shows $\text{beta-idx } s \ y = (\text{if } y = 0 \text{ then } 1 \text{ else } (0 :: 'f::\text{comm-ring-1}))$
using *assms* **by** (simp add: beta-idx-def beta-bool-point)

lemma sum-beta-idx : $(\sum y < 2^s. \text{beta-idx } s \ y) = (1 :: 'f::\text{comm-ring-1})$
by (simp add: beta-idx-def sum-beta-bool-points)

The single-claim integrand of the layer sumcheck polynomial P at a boolean cube cell (x, y) (reduce.rs module documentation, one claim point z with coefficient 1).

definition $\text{layer-integrand1} ::$
 $'f::\text{comm-ring-1} \ \text{layer} \Rightarrow \text{nat} \Rightarrow 'f \ \text{list} \Rightarrow 'f \ \text{list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f$ **where**
 $\text{layer-integrand1 } L \ s \ \text{zpt below } x \ y =$
 $\text{lin-mle } L \ \text{zpt } x * \text{below ! } x * \text{beta-idx } s \ y$
 $+ \text{mul-mle } L \ \text{zpt } x \ y * (\text{below ! } x * \text{below ! } y)$
 $+ \text{pow3-mle } L \ \text{zpt } x * (\text{below ! } x * \text{below ! } x * \text{below ! } x) * \text{beta-idx } s \ y$

theorem $\text{layer-hypercube-identity1}$:
fixes $L :: 'f::\text{comm-ring-1} \ \text{layer}$
assumes $\text{gates-in}: \forall g \in \text{set } (\text{layer-gates } L). \ g\text{-in1 } g < 2^s \wedge g\text{-in2 } g < 2^s$
shows $(\sum x < 2^s. \sum y < 2^s. \text{layer-integrand1 } L \ s \ \text{zpt below } x \ y)$
 $= \text{mle } (\text{layer-width-bits } L) \ (!) (\text{layer-eval } L \ \text{below}) \ \text{zpt} - \text{const-mle } L \ \text{zpt}$

proof –

have $\text{lin-lift}: (\sum x < 2^s. \sum y < 2^s. \text{lin-mle } L \ \text{zpt } x * \text{below ! } x * \text{beta-idx } s \ y)$
 $= (\sum x < 2^s. \text{lin-mle } L \ \text{zpt } x * \text{below ! } x)$

proof –

have $(\sum x < 2^s. \sum y < 2^s. \text{lin-mle } L \ \text{zpt } x * \text{below ! } x * \text{beta-idx } s \ y)$
 $= (\sum x < 2^s. \text{lin-mle } L \ \text{zpt } x * \text{below ! } x * (\sum y < 2^s. \text{beta-idx } s \ y))$

by (simp add: sum-distrib-left)

then show ?thesis **by** (simp add: sum-beta-idx)

qed

have $\text{pow3-lift}: (\sum x < 2^s. \sum y < 2^s. \text{pow3-mle } L \ \text{zpt } x * (\text{below ! } x * \text{below ! } x * \text{below ! } x) * \text{beta-idx } s \ y)$
 $= (\sum x < 2^s. \text{pow3-mle } L \ \text{zpt } x * (\text{below ! } x * \text{below ! } x * \text{below ! } x))$

proof –

```

have (∑ x<2^s. ∑ y<2^s.
  pow3-mle L zpt x * (below ! x * below ! x * below ! x) * beta-idx s y)
  = (∑ x<2^s. pow3-mle L zpt x * (below ! x * below ! x * below ! x)
    * (∑ y<2^s. beta-idx s y))
  by (simp add: sum-distrib-left)
then show ?thesis by (simp add: sum-beta-idx)
qed
have (∑ x<2^s. ∑ y<2^s. layer-integrand1 L s zpt below x y)
  = (∑ x<2^s. ∑ y<2^s. lin-mle L zpt x * below ! x * beta-idx s y)
  + ((∑ x<2^s. ∑ y<2^s. mul-mle L zpt x y * (below ! x * below ! y))
    + (∑ x<2^s. ∑ y<2^s.
      pow3-mle L zpt x * (below ! x * below ! x * below ! x) * beta-idx s y))
  by (simp add: layer-integrand1-def sum.distrib add.assoc)
also have ... = (∑ x<2^s. lin-mle L zpt x * below ! x)
  + ((∑ x<2^s. ∑ y<2^s. mul-mle L zpt x y * (below ! x * below ! y))
    + (∑ x<2^s. pow3-mle L zpt x * (below ! x * below ! x * below ! x)))
  by (simp add: lin-lift pow3-lift)
also have ... = mle (layer-width-bits L) (!) (layer-eval L below)) zpt - const-mle L zpt
  using layer-value-mle-split[OF gates-in]
  by (simp add: algebra-simps)
finally show ?thesis .
qed

```

Multi-claim combination: incoming claim points with verifier coefficients, exactly the shape `reduce.rs` carries between layers (`incoming` = $[(x^*, 1), (y^*, r)]$ after the first layer).

definition *layer-integrand* ::

```

'f::comm-ring-1 layer ⇒ nat ⇒ ('f list × 'f) list ⇒ 'f list ⇒ nat ⇒ nat ⇒ 'f where
layer-integrand L s claims below x y =
  (∑ (z, c)←claims. c * layer-integrand1 L s z below x y)

```

definition *layer-claim-sum* ::

```

'f::comm-ring-1 layer ⇒ ('f list × 'f) list ⇒ 'f list ⇒ 'f where
layer-claim-sum L claims below =
  (∑ (z, c)←claims. c * (mle (layer-width-bits L) (!) (layer-eval L below)) z
    - const-mle L z))

```

theorem *layer-hypercube-identity*:

```

fixes L :: 'f::comm-ring-1 layer
assumes gates-in: ∀ g ∈ set (layer-gates L). g-in1 g < 2 ^ s ∧ g-in2 g < 2 ^ s
shows (∑ x<2^s. ∑ y<2^s. layer-integrand L s claims below x y)
  = layer-claim-sum L claims below

```

proof (*induction claims*)

case *Nil*

show ?case by (simp add: layer-integrand-def layer-claim-sum-def)

next

case (*Cons zc claims*)

obtain *z c* where *zc*: *zc* = (*z*, *c*) by (cases *zc*)

```

have (∑ x<2^s. ∑ y<2^s. layer-integrand L s (zc # claims) below x y)
  = (∑ x<2^s. ∑ y<2^s.
    c * layer-integrand1 L s z below x y + layer-integrand L s claims below x y)
  by (simp add: layer-integrand-def zc)

```

```

also have ... = c * (∑ x<2^s. ∑ y<2^s. layer-integrand1 L s z below x y)
  + (∑ x<2^s. ∑ y<2^s. layer-integrand L s claims below x y)

```

```

    by (simp add: sum.distrib sum-distrib-left)
  also have ... = c * (mle (layer-width-bits L) (!) (layer-eval L below)) z
    - const-mle L z)
    + layer-claim-sum L claims below
  by (simp add: layer-hypercube-identity1[OF gates-in] Cons.IH)
  also have ... = layer-claim-sum L (zc # claims) below
  by (simp add: layer-claim-sum-def zc)
  finally show ?case .
qed

```

27 Activation instance (non-vacuity witness for B1)

The demo layer of *Layered-Circuit* (all three gate kinds and a constant), claim point [3] over the integers, below = [2, 1]:

layer values = [2, 2] (demo-eval) V ([3]) = (1-3)*2 + 3*2 = 2 const ([3]) = (1-3)*(-5) = 10
 rhs = 2 - 10 = -8

lhs: lin x=0: (1-3)*3 * 2 * [y=0] = -12 mul x=0,y=1: 3 * (2*1) = 6 pow3 x=1: (1-3)*1 * 1 * [y=0] = -2 total -8.

lemma demo-hypercube-identity-value:

($\sum x < 2^1. \sum y < 2^1. \text{layer-integrand1 demo-layer 1 [3] [2, 1] x y}$) = (- 8 :: int)

proof –

have expand: {.. $(2::\text{nat})$ } = {0, 1} by auto

show ?thesis

unfolding layer-integrand1-def lin-mle-def mul-mle-def pow3-mle-def
 lin-wt-def mul-wt-def pow3-wt-def beta-idx-def

by (simp add: expand demo-layer-def mle-def eq-pi-def beta-prod
 bool-point-def bit-at-iff-odd-div upt-rec)

qed

lemma demo-hypercube-identity-rhs:

mle (layer-width-bits demo-layer) (!) (layer-eval demo-layer [2, 1])) [3]
 - const-mle demo-layer [3] = (- 8 :: int)

proof –

have expand: {.. $(2::\text{nat})$ } = {0, 1} by auto

have wb: layer-width-bits demo-layer = 1

by (simp add: demo-layer-def)

have vals: layer-eval demo-layer [2, 1] = [2, 2]

using demo-eval

by (simp add: circuit-output-def demo-circuit-def Let-def)

have cvals: const-at demo-layer 0 = (- 5 :: int)

const-at demo-layer (Suc 0) = (0 :: int)

by (simp-all add: const-at-def demo-layer-def)

show ?thesis

unfolding const-mle-def

by (simp add: expand wb vals cvals mle-def eq-pi-def
 bool-point-def bit-at-iff-odd-div upt-rec)

qed

28 Index/substitution bridge to the AFP protocol (B2 plumbing)

The AFP sumcheck sums over substitutions $substs\ V\ H$; the layer identity sums over cube indices. This section builds the bijection between $substs\ \{0..<2s\}\ \{0,1\}$ and index pairs $\{..<2^s\}\ x\ \{..<2^s\}$, in the frozen variable convention (S-2 / poly.rs): variable 0 is the most significant bit, the x block (variables $0..<s$) precedes the y block (variables $s..<2s$), matching the Rust binding order x then y.

definition $bits-idx :: (nat \Rightarrow bool) \Rightarrow nat \Rightarrow nat$ **where**
 $bits-idx\ b\ n = (\sum j < n. \text{if } b\ j \text{ then } 2^{(n-1-j)} \text{ else } 0)$

lemma $sum-lessThan-Suc-shift$:

fixes $f :: nat \Rightarrow 'b::comm-monoid-add$

shows $(\sum j < Suc\ n. f\ j) = f\ 0 + (\sum j < n. f\ (Suc\ j))$

proof ($induction\ n$)

case 0

show $?case$ **by** $simp$

next

case ($Suc\ n$)

then show $?case$ **by** ($simp\ add: add.assoc$)

qed

lemma $bits-idx-Suc$:

$bits-idx\ b\ (Suc\ n) = (\text{if } b\ 0 \text{ then } 2^n \text{ else } 0) + bits-idx\ (b \circ Suc)\ n$

proof –

have $bits-idx\ b\ (Suc\ n) = (\sum j < Suc\ n. \text{if } b\ j \text{ then } 2^{(Suc\ n-1-j)} \text{ else } 0)$

by ($simp\ add: bits-idx-def$)

also have $\dots = (\text{if } b\ 0 \text{ then } 2^{(Suc\ n-1-0)} \text{ else } 0)$

$+ (\sum j < n. \text{if } b\ (Suc\ j) \text{ then } 2^{(Suc\ n-1-Suc\ j)} \text{ else } 0)$

by ($rule\ sum-lessThan-Suc-shift$)

also have $(\sum j < n. \text{if } b\ (Suc\ j) \text{ then } 2^{(Suc\ n-1-Suc\ j)} \text{ else } (0::nat))$

$= (\sum j < n. \text{if } b\ (Suc\ j) \text{ then } 2^{(n-1-j)} \text{ else } 0)$

proof ($intro\ sum.cong\ refl$)

fix j **assume** $j \in \{..<n\}$

have $Suc\ n-1-Suc\ j = n-1-j$ **by** $arith$

then show $(\text{if } b\ (Suc\ j) \text{ then } 2^{(Suc\ n-1-Suc\ j)} \text{ else } (0::nat))$

$= (\text{if } b\ (Suc\ j) \text{ then } 2^{(n-1-j)} \text{ else } 0)$ **by** $simp$

qed

also have $(\text{if } b\ 0 \text{ then } 2^{(Suc\ n-1-0)} \text{ else } (0::nat))$

$+ (\sum j < n. \text{if } b\ (Suc\ j) \text{ then } 2^{(n-1-j)} \text{ else } 0)$

$= (\text{if } b\ 0 \text{ then } 2^n \text{ else } 0) + bits-idx\ (b \circ Suc)\ n$

by ($simp\ add: bits-idx-def\ o-def$)

finally show $?thesis$.

qed

lemma $bits-idx-lt$: $bits-idx\ b\ n < 2^n$

proof ($induction\ n\ arbitrary: b$)

case 0

show $?case$ **by** ($simp\ add: bits-idx-def$)

next

case ($Suc\ n$)

```

  have bits-idx (b  $\circ$  Suc) n <  $2 \wedge n$  by (rule Suc.IH)
  then show ?case by (auto simp add: bits-idx-Suc)
qed

lemma bit-at-bits-idx:
  assumes j < n
  shows bit-at n (bits-idx b n) j = b j
  using assms
proof (induction n arbitrary: b j)
  case 0
  then show ?case by simp
next
  case (Suc n)
  have lt: bits-idx (b  $\circ$  Suc) n <  $2 \wedge n$  by (rule bits-idx-lt)
  show ?case
  proof (cases j)
    case 0
    show ?thesis
    proof (cases b 0)
      case True
      then have bits-idx b (Suc n) = bits-idx (b  $\circ$  Suc) n +  $2 \wedge n$ 
        by (simp add: bits-idx-Suc)
      then show ?thesis using 0 True bit-at-head-hi[OF lt] by simp
    next
    case False
    then have bits-idx b (Suc n) = bits-idx (b  $\circ$  Suc) n
      by (simp add: bits-idx-Suc)
    then show ?thesis using 0 False bit-at-head-lo[OF lt] by simp
  qed
  next
  case (Suc k)
  then have kn: k < n using Suc.prems by simp
  show ?thesis
  proof (cases b 0)
    case True
    then have eq: bits-idx b (Suc n) = bits-idx (b  $\circ$  Suc) n +  $2 \wedge n$ 
      by (simp add: bits-idx-Suc)
    have bit-at (Suc n) (bits-idx b (Suc n)) (Suc k)
      = bit-at n (bits-idx (b  $\circ$  Suc) n) k
      unfolding eq by (rule bit-at-tail-hi[OF lt kn])
    also have ... = (b  $\circ$  Suc) k by (rule Suc.IH[OF kn])
    finally show ?thesis using Suc by simp
  next
  case False
  then have eq: bits-idx b (Suc n) = bits-idx (b  $\circ$  Suc) n
    by (simp add: bits-idx-Suc)
  have bit-at (Suc n) (bits-idx b (Suc n)) (Suc k)
    = bit-at n (bits-idx (b  $\circ$  Suc) n) k
    unfolding eq by (rule bit-at-tail-lo[OF kn])
  also have ... = (b  $\circ$  Suc) k by (rule Suc.IH[OF kn])
  finally show ?thesis using Suc by simp
  qed
qed

```

qed

The substitution carried by an index pair: x block on variables $0..<s$ (MSB first), y block on variables $s..<2s$.

definition $idx\text{-}subst :: nat \Rightarrow nat \Rightarrow nat \Rightarrow (nat, 'a::zero\text{-}neq\text{-}one) \text{ subst}$ **where**
 $idx\text{-}subst\ s\ x\ y\ j =$
 (if $j < s$ then $Some\ (if\ bit\text{-}at\ s\ x\ j\ then\ 1\ else\ 0)$
 else if $j < 2 * s$ then $Some\ (if\ bit\text{-}at\ s\ y\ (j - s)\ then\ 1\ else\ 0)$
 else $None$)

Notation note: in this import context the TIGHT token sequence $2*s$ fails inner-syntax parsing (an interaction along the imported Polynomials / HOL-Computational-Algebra chain), while the spaced form $2 * s$ parses fine - hence explicit spacing in all interval terms below. $upt\ 0\ (2 * s)$ is the exact variable-list shape consumed by the AFP theorems ($set\ vs$ with $vs = upt\ 0\ (2 * s)$).

lemma $idx\text{-}subst\text{-}dom$: $dom\ (idx\text{-}subst\ s\ x\ y) = set\ (upt\ 0\ (2 * s))$
by (auto simp add: $idx\text{-}subst\text{-}def\ dom\text{-}def\ split$: if-splits)

lemma $idx\text{-}subst\text{-}ran$: $ran\ (idx\text{-}subst\ s\ x\ y :: (nat, 'a::zero\text{-}neq\text{-}one) \text{ subst}) \subseteq \{0, 1\}$
by (auto simp add: $idx\text{-}subst\text{-}def\ ran\text{-}def\ split$: if-splits)

lemma $idx\text{-}subst\text{-}in\text{-}substs$:
 $idx\text{-}subst\ s\ x\ y \in substs\ (set\ (upt\ 0\ (2 * s)))\ (\{0, 1\} :: 'a::zero\text{-}neq\text{-}one\ set)$
by (simp add: $substs\text{-}def\ idx\text{-}subst\text{-}dom\ idx\text{-}subst\text{-}ran$)

lemma $idx\text{-}subst\text{-}inj$:
fixes $x\ x'\ y\ y' :: nat$
assumes $bounds$: $x < 2^s\ x' < 2^s\ y < 2^s\ y' < 2^s$
and eq : $(idx\text{-}subst\ s\ x\ y :: (nat, 'a::zero\text{-}neq\text{-}one) \text{ subst}) = idx\text{-}subst\ s\ x'\ y'$
shows $x = x' \wedge y = y'$

proof

have bx : $\bigwedge j. j < s \implies bit\text{-}at\ s\ x\ j = bit\text{-}at\ s\ x'\ j$

proof -

fix j **assume** js : $j < s$

from eq **have** $(idx\text{-}subst\ s\ x\ y :: (nat, 'a) \text{ subst})\ j = idx\text{-}subst\ s\ x'\ y'\ j$ **by** simp

then **have** $(if\ bit\text{-}at\ s\ x\ j\ then\ 1\ else\ (0::'a)) = (if\ bit\text{-}at\ s\ x'\ j\ then\ 1\ else\ 0)$

using js **by** (simp add: $idx\text{-}subst\text{-}def$)

then **show** $bit\text{-}at\ s\ x\ j = bit\text{-}at\ s\ x'\ j$

by (metis (full-types) zero-neq-one)

qed

show $x = x'$ **using** $bit\text{-}at\text{-}complete[OF\ bounds(1)\ bounds(2)]\ bx$ **by** blast

have by : $\bigwedge j. j < s \implies bit\text{-}at\ s\ y\ j = bit\text{-}at\ s\ y'\ j$

proof -

fix j **assume** js : $j < s$

from eq **have** $(idx\text{-}subst\ s\ x\ y :: (nat, 'a) \text{ subst})\ (s + j) = idx\text{-}subst\ s\ x'\ y'\ (s + j)$

by simp

then **have** $(if\ bit\text{-}at\ s\ y\ j\ then\ 1\ else\ (0::'a)) = (if\ bit\text{-}at\ s\ y'\ j\ then\ 1\ else\ 0)$

using js **by** (simp add: $idx\text{-}subst\text{-}def$)

then **show** $bit\text{-}at\ s\ y\ j = bit\text{-}at\ s\ y'\ j$

by (metis (full-types) zero-neq-one)

qed

show $y = y'$ **using** $bit\text{-}at\text{-}complete[OF\ bounds(3)\ bounds(4)]\ by$ **by** blast

qed

lemma *idx-subst-surj*:

fixes $\sigma :: (\text{nat}, 'a::\text{zero-neg-one}) \text{ subst}$

assumes $\sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set})$

shows $\exists x < 2^{\wedge}s. \exists y < 2^{\wedge}s. \sigma = \text{idx-subst } s \ x \ y$

proof –

have $\text{dom}\sigma$: $\text{dom } \sigma = \text{set } (\text{upt } 0 \ (2 * s))$ and $\text{ran}\sigma$: $\text{ran } \sigma \subseteq \{0, 1\}$

using *assms* by (auto simp add: *substs-def*)

define x where $x = \text{bits-idx } (\lambda j. \sigma \ j = \text{Some } 1) \ s$

define y where $y = \text{bits-idx } (\lambda j. \sigma \ (s + j) = \text{Some } 1) \ s$

have xs : $x < 2^{\wedge}s$ and ys : $y < 2^{\wedge}s$

by (simp-all add: *x-def y-def bits-idx-lt*)

have val : $\bigwedge j. j < 2 * s \implies \sigma \ j = \text{Some } 0 \vee \sigma \ j = \text{Some } 1$

proof –

fix j assume $j < 2 * s$

then have $j \in \text{dom } \sigma$ using $\text{dom}\sigma$ by *simp*

then obtain v where v : $\sigma \ j = \text{Some } v$ by (auto simp add: *dom-def*)

then have $v \in \text{ran } \sigma$ by (auto simp add: *ran-def*)

then have $v = 0 \vee v = 1$ using $\text{ran}\sigma$ by *auto*

then show $\sigma \ j = \text{Some } 0 \vee \sigma \ j = \text{Some } 1$ using v by *auto*

qed

have $\sigma = \text{idx-subst } s \ x \ y$

proof

fix j

show $\sigma \ j = \text{idx-subst } s \ x \ y \ j$

proof (cases $j < s$)

case *True*

then have bit : $\text{bit-at } s \ x \ j = (\sigma \ j = \text{Some } 1)$

by (simp add: *x-def bit-at-bits-idx*)

have $j < 2 * s$ using *True* by *simp*

then show *?thesis* using $\text{val}[\text{of } j]$ *True bit*

by (auto simp add: *idx-subst-def*)

next

case *False*

show *?thesis*

proof (cases $j < 2 * s$)

case *True*

then have $j2$: $j = s + (j - s)$ and js : $j - s < s$ using *False* by *simp-all*

have bit : $\text{bit-at } s \ y \ (j - s) = (\sigma \ (s + (j - s)) = \text{Some } 1)$

by (simp add: *y-def bit-at-bits-idx js*)

then show *?thesis* using $\text{val}[\text{of } j]$ *False True j2*

by (auto simp add: *idx-subst-def*)

next

case *False*

then have $\sigma \ j = \text{None}$ using $\text{dom}\sigma$ by (auto simp add: *dom-def*)

then show *?thesis* using $\neg j < s$ $\neg j < 2 * s$

by (simp add: *idx-subst-def*)

qed

qed

qed

then show *?thesis* using $xs \ ys$ by *blast*

qed

```

lemma bij-idx-subst:
  bij-betw ( $\lambda(x, y). \text{idx-subst } s \ x \ y$ ) ( $\{..<2^s\} \times \{..<2^s\}$ )
    (substs (set (upt 0 ( $2 * s$ ))) ( $\{0, 1\} :: 'a::\text{zero-neq-one set}$ ))
proof (rule bij-betw-imageI)
  show inj-on ( $\lambda(x, y). \text{idx-subst } s \ x \ y :: (\text{nat}, 'a) \text{ subst}$ ) ( $\{..<2^s\} \times \{..<2^s\}$ )
    by (intro inj-onI) (auto dest: idx-subst-inj)
  show ( $\lambda(x, y). \text{idx-subst } s \ x \ y :: (\text{nat}, 'a) \text{ subst}$ ) ' $(\{..<2^s\} \times \{..<2^s\})$ 
    = substs (set (upt 0 ( $2 * s$ )))  $\{0, 1\}$ 
proof
  show ( $\lambda(x, y). \text{idx-subst } s \ x \ y :: (\text{nat}, 'a) \text{ subst}$ ) ' $(\{..<2^s\} \times \{..<2^s\})$ 
     $\subseteq$  substs (set (upt 0 ( $2 * s$ )))  $\{0, 1\}$ 
proof
  fix  $\sigma :: (\text{nat}, 'a) \text{ subst}$ 
  assume  $\sigma \in (\lambda(x, y). \text{idx-subst } s \ x \ y) '(\{..<2^s\} \times \{..<2^s\})$ 
  then obtain  $x \ y$  where  $\sigma = \text{idx-subst } s \ x \ y$  by auto
  then show  $\sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ \{0, 1\}$ 
    using idx-subst-in-substs by simp
qed
show substs (set (upt 0 ( $2 * s$ )))  $\{0, 1\}$ 
   $\subseteq$  ( $\lambda(x, y). \text{idx-subst } s \ x \ y :: (\text{nat}, 'a) \text{ subst}$ ) ' $(\{..<2^s\} \times \{..<2^s\})$ 
proof
  fix  $\sigma :: (\text{nat}, 'a) \text{ subst}$  assume  $\sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ \{0, 1\}$ 
  then obtain  $x \ y$  where  $x < 2^s \ y < 2^s \ \sigma = \text{idx-subst } s \ x \ y$ 
    using idx-subst-surj by blast
  then show  $\sigma \in (\lambda(x, y). \text{idx-subst } s \ x \ y) '(\{..<2^s\} \times \{..<2^s\})$ 
    by (auto simp add: image-def)
qed
qed
qed

```

lemma *sum-substs-index*:

```

fixes  $G :: (\text{nat}, 'a::\text{zero-neq-one}) \text{ subst} \Rightarrow 'b::\text{comm-monoid-add}$ 
shows ( $\sum \sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set}). G \ \sigma$ )
  = ( $\sum x < 2^s. \sum y < 2^s. G \ (\text{idx-subst } s \ x \ y)$ )
proof –
  have ( $\sum \sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set}). G \ \sigma$ )
    = ( $\sum p \in \{..<2^s\} \times \{..<2^s\}. G \ (\text{case } p \text{ of } (x, y) \Rightarrow \text{idx-subst } s \ x \ y)$ )
    by (rule sum.reindex-bij-betw[OF bij-idx-subst, symmetric])
  also have  $\dots = (\sum p \in \{..<2^s\} \times \{..<2^s\}. G \ (\text{idx-subst } s \ (\text{fst } p) \ (\text{snd } p)))$ 
    by (intro sum.cong reft) (simp add: split-def)
  also have  $\dots = (\sum x < 2^s. \sum y < 2^s. G \ (\text{idx-subst } s \ x \ y))$ 
    by (simp add: sum.cartesian-product split-def)
  finally show ?thesis .
qed

```

29 The layer sumcheck on the AFP protocol (B2)

The bridge locale: the AFP abstract multivariate polynomial operations, with variables fixed to naturals and both the challenge type and the evaluation codomain fixed to one finite FIELD type 'a (the challenge field; KoalaBear degree-4 extension in production). No axiom

is added beyond the AFP locale, so every fact proved here is dischargeable in any AFP instance (no orphan assumptions); the connection to the layer identity is packaged in the definition *layer-poly-repr* and consumed as a theorem ASSUMPTION, discharged at instantiation time.

```

locale gkr-layer-sumcheck =
  multi-variate-polynomial vars deg eval inst
  for vars :: 'p::comm-monoid-add  $\Rightarrow$  nat set
  and deg :: 'p  $\Rightarrow$  nat
  and eval :: 'p  $\Rightarrow$  (nat, 'a::{finite, field}) subst  $\Rightarrow$  'a
  and inst :: 'p  $\Rightarrow$  (nat, 'a) subst  $\Rightarrow$  'p
begin

```

layer-poly-repr p L s *claims below* $dbnd$: the abstract polynomial p represents the layer sum-check instance - variables in the 2s-cube (x block then y block), AFP degree at most $dbnd$, and boolean-hypercube evaluations equal to the layer integrand. Only hypercube agreement is required: the sumcheck sum ranges over boolean substitutions, and the AFP round machinery never inspects non-boolean values of p itself.

```

definition layer-poly-repr ::
  'p  $\Rightarrow$  'a layer  $\Rightarrow$  nat  $\Rightarrow$  ('a list  $\times$  'a) list  $\Rightarrow$  'a list  $\Rightarrow$  nat  $\Rightarrow$  bool where
  layer-poly-repr  $p$   $L$   $s$  claims below  $dbnd \iff$ 
    vars  $p \subseteq \text{set } (\text{upt } 0 \ (2 * s)) \wedge \text{deg } p \leq dbnd \wedge$ 
    ( $\forall x < 2^s. \forall y < 2^s. \text{eval } p \ (\text{idx-subst } s \ x \ y) = \text{layer-integrand } L \ s \ \text{claims below } x \ y$ )

```

lemma *layer-poly-repr-sum*:

```

assumes repr: layer-poly-repr  $p$   $L$   $s$  claims below  $dbnd$ 
and gates-in:  $\forall g \in \text{set } (\text{layer-gates } L). \ g\text{-in1 } g < 2^s \wedge g\text{-in2 } g < 2^s$ 
shows ( $\sum \sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set}). \text{eval } p \ \sigma$ )
  = layer-claim-sum  $L$  claims below

```

proof –

```

have ( $\sum \sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set}). \text{eval } p \ \sigma$ )
  = ( $\sum x < 2^s. \sum y < 2^s. \text{eval } p \ (\text{idx-subst } s \ x \ y)$ )
by (rule sum-substs-index)
also have ... = ( $\sum x < 2^s. \sum y < 2^s. \text{layer-integrand } L \ s \ \text{claims below } x \ y$ )
using repr by (intro sum.cong refl) (simp add: layer-poly-repr-def)
also have ... = layer-claim-sum  $L$  claims below
by (rule layer-hypercube-identity[OF gates-in])
finally show ?thesis .

```

qed

B2 soundness: one layer reduction, AFP *soundness-inductive* consumed with $vs = [0..<2s]$ and $d = dbnd$. If the claimed value v differs from the true combined claim (the right side of the hypercube identity), then ANY prover makes the sumcheck verifier accept with probability at most $2s * dbnd / |a|$ over the uniformly drawn challenge tuple.

theorem *layer-sumcheck-soundness*:

```

assumes repr: layer-poly-repr  $p$   $L$   $s$  claims below  $dbnd$ 
and gates-in:  $\forall g \in \text{set } (\text{layer-gates } L). \ g\text{-in1 } g < 2^s \wedge g\text{-in2 } g < 2^s$ 
and false-claim:  $v \neq \text{layer-claim-sum } L \ \text{claims below}$ 
shows measure-pmf.prob (pmf-of-set (tuples UNIV  $(2 * s)$ ))
  {rs. sumcheck pr ps ( $\{0, 1\}$ ,  $p$ ,  $v$ )  $r$  (zip (upt 0  $(2 * s)$ ) rs)}
   $\leq \text{real } (2 * s) * \text{real } dbnd / \text{real } \text{CARD}('a)$ 

```

proof –

```

have vp: vars  $p \subseteq \text{set } (\text{upt } 0 \ (2 * s))$  and dp: deg  $p \leq dbnd$ 

```

```

using repr by (auto simp add: layer-poly-repr-def)
have neg:  $v \neq (\sum \sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set}). \text{eval } p \ \sigma)$ 
using layer-poly-repr-sum[OF repr gates-in] false-claim by simp
have set-eq:  $\{rs. \text{sumcheck } pr \ ps \ (\{0, 1\}, p, v) \ r \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs)\}$ 
 $= \{rs. \text{sumcheck } pr \ ps \ (\{0, 1\}, p, v) \ r \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs) \wedge$ 
 $v \neq (\sum \sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set}). \text{eval } p \ \sigma)\}$ 
using neg by auto
have measure-pmf.prob (pmf-of-set (tuples UNIV (length (upt 0 (2 * s)))))
 $\{rs. \text{sumcheck } pr \ ps \ (\{0, 1\}, p, v) \ r \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs) \wedge$ 
 $v \neq (\sum \sigma \in \text{substs } (\text{set } (\text{upt } 0 \ (2 * s))) \ (\{0, 1\} :: 'a \text{ set}). \text{eval } p \ \sigma)\}$ 
 $\leq \text{real } (\text{length } (\text{upt } 0 \ (2 * s))) * \text{real } \text{dbnd} / \text{real } \text{CARD}('a)$ 
by (rule soundness-inductive[OF vp dp]) simp-all
then show ?thesis by (simp add: set-eq)
qed

```

Specialisation to the frozen per-round degree bound of the Rust verifier (reduce.rs *LAYER-ROUND-DEGREE* = 4): a representation with dbnd = 4 gives the design bound $2s * 4 / |a|$ per layer. Whether a given AFP instance admits a dbnd = 4 representation depends on that instance's degree measure (the AFP-shipped mpoly instance uses TOTAL degree, for which the representative has dbnd on the order of s ; a max-per-variable-degree instance yields dbnd = 4). The bound parameter keeps both dischargeable.

corollary layer-sumcheck-soundness-deg4:

```

assumes layer-poly-repr  $p \ L \ s$  claims below 4
and  $\forall g \in \text{set } (\text{layer-gates } L). \ g\text{-in1 } g < 2^s \wedge g\text{-in2 } g < 2^s$ 
and  $v \neq \text{layer-claim-sum } L$  claims below
shows measure-pmf.prob (pmf-of-set (tuples UNIV (2 * s)))
 $\{rs. \text{sumcheck } pr \ ps \ (\{0, 1\}, p, v) \ r \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs)\}$ 
 $\leq \text{real } (2 * s) * 4 / \text{real } \text{CARD}('a)$ 
using layer-sumcheck-soundness[OF assms] by simp

```

B2 completeness (the honest-acceptance corollary of the hypercube identity): if the claimed value IS the true combined claim, the AFP honest prover passes every round for every challenge tuple. Consumed from AFP *completeness-inductive*.

theorem layer-sumcheck-completeness:

```

assumes repr: layer-poly-repr  $p \ L \ s$  claims below dbnd
and gates-in:  $\forall g \in \text{set } (\text{layer-gates } L). \ g\text{-in1 } g < 2^s \wedge g\text{-in2 } g < 2^s$ 
and true-claim:  $v = \text{layer-claim-sum } L$  claims below
and len-rs:  $\text{length } rs = 2 * s$ 
shows sumcheck honest-prover  $u \ (\{0, 1\}, p, v) \ r \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs)$ 
proof –
have fst-zip:  $\text{map } fst \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs) = (\text{upt } 0 \ (2 * s))$ 
using len-rs by (simp add: map-fst-zip)
have  $v = (\sum \sigma \in \text{substs } (\text{set } (\text{map } fst \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs)))) \ (\{0, 1\} :: 'a \text{ set}). \text{eval } p \ \sigma$ 
using layer-poly-repr-sum[OF repr gates-in] true-claim by (simp add: fst-zip)
moreover have vars  $p \subseteq \text{set } (\text{map } fst \ (\text{zip } (\text{upt } 0 \ (2 * s)) \ rs))$ 
using repr by (simp add: fst-zip layer-poly-repr-def)
moreover have distinct (map fst (zip (upt 0 (2 * s)) rs))
by (simp add: fst-zip)
moreover have  $(\{0, 1\} :: 'a \text{ set}) \neq \{\}$ 
by simp
ultimately show ?thesis
by (rule completeness-inductive)

```

qed

end

30 Non-vacuity: the AFP mpoly instance inhabits the bridge locale

The bridge locale adds no axioms, so the AFP concrete instantiation (mpoly with *total-degree*; *Concrete-Multivariate-Polynomials*) discharges it for any finite field challenge type. This pins non-vacuity of the locale itself; an actual *layer-poly-repr* witness inside mpoly (the interpolation representative of the layer integrand, with total-degree bound 2s) is constructed in *Layer-Representative*.

interpretation *gkr-mpoly*:

gkr-layer-sumcheck vars :: 'a::{finite, field} mpoly $\Rightarrow$ nat set total-degree
 $\lambda p \sigma. \text{insertion } (the \circ \sigma) p \text{ inst}$
by *unfold-locale* (auto simp add: multi-variate-polynomial-lemmas)

end

theory *Layer-Representative*
imports *Sumcheck-Instance*
begin

31 Boolean factor polynomials

bfac v b: the degree-one factor $X \cdot v$ (bit set) or $1 - X \cdot v$ (bit clear), as a concrete mpoly.

definition *bfac* :: nat $\Rightarrow$ bool $\Rightarrow$ 'a::comm-ring-1 mpoly **where**

bfac v b = (if b then monom (Poly-Mapping.single v 1) 1
 else 1 + monom (Poly-Mapping.single v 1) (- 1))

lemma *insertion-bfac* [simp]:

insertion f (bfac v b) = (if b then *f v* else $1 - f v$)
by (simp add: bfac-def insertion-add)

lemma *vars-one*: vars (1 :: 'a::comm-semiring-1 mpoly) $\subseteq \{\}$
using *vars-monom-subset*[of 0 1 :: 'a] **by** simp

lemma *vars-bfac*: vars (bfac v b) $\subseteq \{v\}$

proof (cases b)

case True

then show ?thesis

by (simp add: bfac-def vars-monom-single)

next

case False

have vars (1 + monom (Poly-Mapping.single v 1) (- 1) :: 'a mpoly)

$\subseteq$ vars (1 :: 'a mpoly) $\cup$ vars (monom (Poly-Mapping.single v 1) (- 1 :: 'a))

by (rule vars-add)

also have ... $\subseteq \{\} \cup \{v\}$

```

    using vars-one vars-monom-single[of v 1 - 1 :: 'a] by auto
    finally show ?thesis using False by (simp add: bfac-def)
qed

```

32 Homomorphism helpers

```

lemma insertion-sum-family:
  insertion f ( $\sum i \in A. p\ i$ ) = ( $\sum i \in A. \text{insertion } f\ (p\ i)$ )
  by (induction A rule: infinite-finite-induct) (simp-all add: insertion-add)

```

```

lemma insertion-prod-family:
  insertion f ( $\prod i \in A. p\ i$ ) = ( $\prod i \in A. \text{insertion } f\ (p\ i)$ )
  by (induction A rule: infinite-finite-induct) (simp-all add: insertion-mult)

```

```

lemma vars-prod-family:
  vars ( $\prod i \in A. p\ i$ )  $\subseteq$  ( $\bigcup i \in A. \text{vars } (p\ i)$ )
proof (induction A rule: infinite-finite-induct)
  case (infinite A)
  then show ?case using vars-one by auto
next
  case empty
  then show ?case using vars-one by auto
next
  case (insert a A)
  have vars ( $\prod i \in \text{insert } a\ A. p\ i$ )  $\subseteq$  vars ( $p\ a$ )  $\cup$  vars ( $\prod i \in A. p\ i$ )
    using insert.hyps by (simp add: vars-mult)
  with insert.IH show ?case by auto
qed

```

```

lemma insertion-monom-const: insertion f (monom 0 c) = c
  using insertion-single[of f 0 0 c] by simp

```

```

lemma vars-monom-const: vars (monom 0 c)  $\subseteq$  {}
  using vars-monom-subset[of 0 c] by simp

```

33 The interpolation table polynomial

Bit j of the (x, y) pair in the sumcheck variable order: variables $0..<s$ carry x 's bits, $s..<2s$ carry y 's (*idx-subst*).

```

definition bit2 :: nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  bool where
  bit2 s x y j  $\longleftrightarrow$  (if j < s then bit-at s x j else bit-at s y (j - s))

```

```

lemma the-idx-subst:
  assumes j < 2 * s
  shows the (idx-subst s x y j :: 'a::zero-neq-one option)
    = (if bit2 s x y j then 1 else 0)
  using assms by (auto simp add: idx-subst-def bit2-def)

```

```

definition table-repr :: nat  $\Rightarrow$  (nat  $\Rightarrow$  nat  $\Rightarrow$  'a)  $\Rightarrow$  'a::comm-ring-1 mpoly where
  table-repr s g =
    ( $\sum x < 2^s. \sum y < 2^s.$ 

```

$\text{monom } 0 \ (g \ x \ y) * (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j))$

lemma *vars-table-repr*: $\text{vars } (\text{table-repr } s \ g) \subseteq \text{set } (\text{upt } 0 \ (2 * s))$

proof –

have *term-vars*: $\text{vars } (\text{monom } 0 \ (g \ x \ y) * (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j)))$
 $\subseteq \text{set } (\text{upt } 0 \ (2 * s))$ **for** $x \ y$

proof –

have *prod-vars*: $\text{vars } (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j) :: 'a \ \text{mpoly})$
 $\subseteq (\bigcup j < 2 * s. \text{vars } (\text{bfac } j \ (\text{bit2 } s \ x \ y \ j) :: 'a \ \text{mpoly}))$
by (*rule vars-prod-family*)

have $(\bigcup j < 2 * s. \text{vars } (\text{bfac } j \ (\text{bit2 } s \ x \ y \ j) :: 'a \ \text{mpoly}))$
 $\subseteq \text{set } (\text{upt } 0 \ (2 * s))$

proof (*rule UN-least*)

fix j **assume** $j \in \{..<2 * s\}$

then show $\text{vars } (\text{bfac } j \ (\text{bit2 } s \ x \ y \ j) :: 'a \ \text{mpoly}) \subseteq \text{set } (\text{upt } 0 \ (2 * s))$
using *vars-bfac[of j bit2 s x y j]* **by** *auto*

qed

with *prod-vars* **have** *pv*: $\text{vars } (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j) :: 'a \ \text{mpoly})$
 $\subseteq \text{set } (\text{upt } 0 \ (2 * s))$ **by** *blast*

have $\text{vars } (\text{monom } 0 \ (g \ x \ y) * (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j)))$
 $\subseteq \text{vars } (\text{monom } 0 \ (g \ x \ y))$
 $\cup \text{vars } (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j) :: 'a \ \text{mpoly})$
by (*rule vars-mult*)

with *pv vars-monom-const* **show** *?thesis* **by** *blast*

qed

have $\text{vars } (\text{table-repr } s \ g)$
 $\subseteq (\bigcup x \in \{..<2 \wedge s\}. \text{vars } (\sum y < 2 \wedge s. \text{monom } 0 \ (g \ x \ y) * (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j))))$

unfolding *table-repr-def* **by** (*intro vars-setsum*) *simp*

also have $\dots \subseteq \text{set } (\text{upt } 0 \ (2 * s))$

proof (*intro UN-least*)

fix $x :: \text{nat}$ **assume** $x \in \{..<2 \wedge s\}$

have $\text{vars } (\sum y < 2 \wedge s. \text{monom } 0 \ (g \ x \ y) * (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j)))$
 $\subseteq (\bigcup y \in \{..<2 \wedge s\}. \text{vars } (\text{monom } 0 \ (g \ x \ y) * (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j))))$
by (*intro vars-setsum*) *simp*

also have $\dots \subseteq \text{set } (\text{upt } 0 \ (2 * s))$
using *term-vars* **by** (*intro UN-least*)

finally show $\text{vars } (\sum y < 2 \wedge s. \text{monom } 0 \ (g \ x \ y) * (\prod j < 2 * s. \text{bfac } j \ (\text{bit2 } s \ x \ y \ j)))$
 $\subseteq \text{set } (\text{upt } 0 \ (2 * s))$.

qed

finally show *?thesis* .

qed

34 Hypercube evaluation: the interpolation property

lemma *prod-bool-indicator*:

fixes $n :: \text{nat}$

shows $(\prod j < n. \text{if } P \ j \text{ then } (1 :: 'a :: \text{comm-ring-1}) \text{ else } 0)$

$= (\text{if } \forall j < n. P \ j \text{ then } 1 \text{ else } 0)$

by (*induction n*) (*auto simp add: less-Suc-eq*)

```

lemma bit2-complete:
  assumes  $x: x < 2^s$  and  $x': x' < 2^s$ 
    and  $y: y < 2^s$  and  $y': y' < 2^s$ 
    and agree:  $\forall j < 2 * s. \text{bit2 } s \ x \ y \ j = \text{bit2 } s \ x' \ y' \ j$ 
  shows  $x = x' \wedge y = y'$ 
proof
  have  $\text{bit-at } s \ x \ j = \text{bit-at } s \ x' \ j$  if  $j: j < s$  for  $j$ 
  proof -
    from  $j$  have  $j < 2 * s$  by linarith
    with agree have  $\text{bit2 } s \ x \ y \ j = \text{bit2 } s \ x' \ y' \ j$  by blast
    with  $j$  show ?thesis by (simp add: bit2-def)
  qed
  then show  $x = x'$  using bit-at-complete[OF  $x \ x'$ ] by simp
  have  $\text{bit-at } s \ y \ j = \text{bit-at } s \ y' \ j$  if  $j: j < s$  for  $j$ 
  proof -
    from  $j$  have  $s + j < 2 * s$  by linarith
    with agree have  $\text{bit2 } s \ x \ y \ (s + j) = \text{bit2 } s \ x' \ y' \ (s + j)$  by blast
    then show ?thesis by (simp add: bit2-def)
  qed
  then show  $y = y'$  using bit-at-complete[OF  $y \ y'$ ] by simp
qed

lemma insertion-table-repr:
  fixes  $g :: \text{nat} \Rightarrow \text{nat} \Rightarrow 'a::\text{comm-ring-1}$ 
  assumes  $x': x' < 2^s$  and  $y': y' < 2^s$ 
  shows  $\text{insertion } (\text{the} \circ \text{idx-subst } s \ x' \ y') (\text{table-repr } s \ g) = g \ x' \ y'$ 
proof -
  let  $?f = \text{the} \circ \text{idx-subst } s \ x' \ y'$ 
  have kron:  $(\prod j < 2 * s. \text{if } \text{bit2 } s \ x \ y \ j \text{ then } ?f \ j \text{ else } 1 - ?f \ j)$ 
     $= (\text{if } x = x' \wedge y = y' \text{ then } (1 :: 'a) \text{ else } 0)$ 
    if  $x: x < 2^s$  and  $y: y < 2^s$  for  $x \ y$ 
  proof -
    have  $(\prod j < 2 * s. \text{if } \text{bit2 } s \ x \ y \ j \text{ then } ?f \ j \text{ else } 1 - ?f \ j)$ 
       $= (\prod j < 2 * s. \text{if } \text{bit2 } s \ x \ y \ j = \text{bit2 } s \ x' \ y' \ j \text{ then } (1 :: 'a) \text{ else } 0)$ 
      by (intro prod.cong refl) (auto simp add: the-idx-subst)
    also have  $\dots = (\text{if } \forall j < 2 * s. \text{bit2 } s \ x \ y \ j = \text{bit2 } s \ x' \ y' \ j \text{ then } 1 \text{ else } 0)$ 
      by (rule prod-bool-indicator)
    also have  $\dots = (\text{if } x = x' \wedge y = y' \text{ then } 1 \text{ else } 0)$ 
  proof (cases  $\forall j < 2 * s. \text{bit2 } s \ x \ y \ j = \text{bit2 } s \ x' \ y' \ j$ )
    case True
      with bit2-complete[OF  $x \ x' \ y \ y'$ ] show ?thesis by simp
    next
      case False
      then have  $\neg (x = x' \wedge y = y')$  by auto
      with False show ?thesis by simp
  qed
  finally show ?thesis .
qed
have inner:  $(\sum y < 2^s. g \ x \ y * (\text{if } x = x' \wedge y = y' \text{ then } 1 \text{ else } 0))$ 
   $= (\text{if } x = x' \text{ then } g \ x \ y' \text{ else } 0)$  for  $x$ 
proof (cases  $x = x'$ )
  case True
    have  $(\sum y < 2^s. g \ x \ y * (\text{if } x = x' \wedge y = y' \text{ then } 1 \text{ else } 0))$ 

```

```

      = (∑ y < 2 ^ s. if y = y' then g x y else 0)
    by (intro sum.cong refl) (simp add: True)
  also have ... = g x y'
    using y' by (simp add: sum.delta')
  finally show ?thesis by (simp add: True)
next
  case False
  then show ?thesis by simp
qed
have insertion ?f (table-repr s g)
  = (∑ x < 2 ^ s. ∑ y < 2 ^ s.
    g x y * (∏ j < 2 * s. if bit2 s x y j then ?f j else 1 - ?f j))
  unfolding table-repr-def
  by (simp add: insertion-sum-family insertion-mult insertion-prod-family
    insertion-monom-const)
also have ... = (∑ x < 2 ^ s. ∑ y < 2 ^ s.
  g x y * (if x = x' ∧ y = y' then 1 else 0))
  by (intro sum.cong refl) (simp add: kron)
also have ... = (∑ x < 2 ^ s. if x = x' then g x y' else 0)
  by (intro sum.cong refl inner)
also have ... = g x' y'
  using x' by (simp add: sum.delta')
finally show ?thesis .
qed

```

35 Total-degree bounds for the interpolation construction

The AFP degree measure is TOTAL degree; the interpolation is a sum of products of $2s$ degree-at-most-one factors, so its total degree is at most $2s$. This makes the representation premise dischargeable with the width-dependent bound $dbnd = \lambda s. 2 * s$.

The degree of a monomial exponent vector, in the raw form the *total-degree* representation uses. (Self-contained: no dependency on the power-products theory, whose heavy syntax overloading interferes with this theory's notation.)

abbreviation $mdeg :: (nat \Rightarrow_0 nat) \Rightarrow nat$ **where**
 $mdeg\ m \equiv sum\ (lookup\ m)\ (keys\ m)$

lemma *total-degree-le-iff*:
 $total-degree\ p \leq d \iff (\forall m \in keys\ (mapping-of\ p). mdeg\ m \leq d)$
 by (simp add: total-degree.rep-eq Max-le-iff)

lemma *mdeg-plus*: $mdeg\ (a + b) = mdeg\ a + mdeg\ b$

proof –

```

  let ?S = keys a ∪ keys b
  have finS: finite ?S by simp
  have keys-sub: keys (a + b) ⊆ ?S
    by (rule Poly-Mapping.keys-add)
  have mdeg (a + b) = sum (lookup (a + b)) ?S
    by (rule sum.mono-neutral-left[OF finS keys-sub]) (simp add: in-keys-iff)
  also have ... = sum (lookup a) ?S + sum (lookup b) ?S
    by (simp add: lookup-add sum.distrib)

```

also have $\text{sum } (\text{lookup } a) \text{ ?}S = \text{mdeg } a$
by (rule *sum.mono-neutral-right*[*OF finS Un-upper1*]) (simp add: *in-keys-iff*)
also have $\text{sum } (\text{lookup } b) \text{ ?}S = \text{mdeg } b$
by (rule *sum.mono-neutral-right*[*OF finS Un-upper2*]) (simp add: *in-keys-iff*)
finally show *?thesis* .
qed

lemma *mdeg-le-total-degree*:
assumes $m \in \text{keys } (\text{mapping-of } p)$
shows $\text{mdeg } m \leq \text{total-degree } p$
proof –
have $\text{mdeg } m \in \text{insert } 0 \text{ (mdeg 'keys (mapping-of } p))$
using *assms* **by** *simp*
then have $\text{mdeg } m \leq \text{Max } (\text{insert } 0 \text{ (mdeg 'keys (mapping-of } p)))$
by (*intro Max-ge*) *simp*
then show *?thesis*
by (simp add: *total-degree.rep-eq*)
qed

lemma *total-degree-mult-le*:
fixes $p \ q :: 'a::\text{comm-semiring-0 mpoly}$
shows $\text{total-degree } (p * q) \leq \text{total-degree } p + \text{total-degree } q$
proof (*subst total-degree-le-iff, intro ballI*)
fix m **assume** $m \in \text{keys } (\text{mapping-of } (p * q))$
then have $m \in \text{keys } (\text{mapping-of } p * \text{mapping-of } q)$
by (simp add: *times-mpoly.rep-eq*)
then obtain $a \ b$ **where** $m = a + b$
and $a : a \in \text{keys } (\text{mapping-of } p)$ **and** $b : b \in \text{keys } (\text{mapping-of } q)$
using *keys-mult* **by** *blast*
have $\text{mdeg } m = \text{mdeg } a + \text{mdeg } b$
by (simp add: *m mdeg-plus*)
also have $\dots \leq \text{total-degree } p + \text{total-degree } q$
using *mdeg-le-total-degree*[*OF a*] *mdeg-le-total-degree*[*OF b*]
by (rule *add-mono*)
finally show $\text{mdeg } m \leq \text{total-degree } p + \text{total-degree } q$.
qed

lemma *total-degree-monom-le*: $\text{total-degree } (\text{monom } m \ a) \leq \text{mdeg } m$
proof (*subst total-degree-le-iff, intro ballI*)
fix m' **assume** $m' \in \text{keys } (\text{mapping-of } (\text{monom } m \ a))$
then have $m' = m$
by (simp add: *monom.rep-eq split: if-splits*)
then show $\text{mdeg } m' \leq \text{mdeg } m$
by *simp*
qed

lemma *mdeg-single*: $\text{mdeg } (\text{Poly-Mapping.single } v \ (\text{Suc } 0)) = 1$
by *simp*

lemma *total-degree-bfac-le*: $\text{total-degree } (\text{bfac } v \ b :: 'a::\text{comm-ring-1 mpoly}) \leq 1$
proof (*cases b*)
case *True*
then show *?thesis*

```

    using total-degree-monom-le[of Poly-Mapping.single v 1 1 :: 'a]
    by (simp add: bfac-def)
next
case False
have total-degree (1 + monom (Poly-Mapping.single v 1) (- 1) :: 'a mpoly)
  ≤ max (total-degree (1 :: 'a mpoly))
    (total-degree (monom (Poly-Mapping.single v 1) (- 1 :: 'a)))
  by (rule deg-add)
also have ... ≤ 1
  using total-degree-monom-le[of Poly-Mapping.single v 1 - 1 :: 'a]
  by simp
finally show ?thesis using False by (simp add: bfac-def)
qed

```

```

lemma total-degree-prod-le:
  fixes p :: 'i ⇒ 'a::comm-semiring-1 mpoly
  shows total-degree (∏ i ∈ A. p i) ≤ (∑ i ∈ A. total-degree (p i))
proof (induction A rule: infinite-finite-induct)
  case (insert a A)
  have total-degree (∏ i ∈ insert a A. p i)
    ≤ total-degree (p a) + total-degree (∏ i ∈ A. p i)
    using insert.hyps by (simp add: total-degree-mult-le)
  also have ... ≤ total-degree (p a) + (∑ i ∈ A. total-degree (p i))
    using insert.IH by (rule add-left-mono)
  finally show ?case using insert.hyps by simp
qed simp-all

```

```

lemma total-degree-sum-le:
  fixes p :: 'i ⇒ 'a::comm-monoid-add mpoly
  assumes ∧i. i ∈ A ⇒ total-degree (p i) ≤ d
  shows total-degree (∑ i ∈ A. p i) ≤ d
  using assms
proof (induction A rule: infinite-finite-induct)
  case (insert a A)
  have total-degree (∑ i ∈ insert a A. p i)
    ≤ max (total-degree (p a)) (total-degree (∑ i ∈ A. p i))
    using insert.hyps by (simp add: deg-add)
  also have ... ≤ d
    using insert.premis insert.IH by simp
  finally show ?case .
qed (simp-all add: total-degree-zero)

```

```

lemma total-degree-table-repr: total-degree (table-repr s g) ≤ 2 * s
proof -
  have term-bound: total-degree
    (monom 0 (g x y) * (∏ j < 2 * s. bfac j (bit2 s x y j))) ≤ 2 * s
  for x y
  proof (subst total-degree-le-iff, intro ballI)
    — every standalone occurrence of the product below carries a type annotation: an unanchored
    occurrence would be inferred at a FRESH type variable (the type-binding pathology this corpus guards
    against), making it a different term from the chained facts
    fix m
    assume m ∈ keys (mapping-of

```

```

      (monom 0 (g x y) * (∏ j < 2 * s. bfac j (bit2 s x y j)))
then have m ∈ keys (mapping-of (monom 0 (g x y))
      * mapping-of (∏ j < 2 * s. bfac j (bit2 s x y j) :: 'a mpoly))
by (simp add: times-mpoly.rep-eq)
then have m ∈ {a + b | a b.
      a ∈ keys (mapping-of (monom 0 (g x y)))
      ∧ b ∈ keys (mapping-of (∏ j < 2 * s. bfac j (bit2 s x y j) :: 'a mpoly))}
by (rule subsetD[OF keys-mult])
then obtain a b where m: m = a + b
and a: a ∈ keys (mapping-of (monom 0 (g x y)))
and b: b ∈ keys (mapping-of (∏ j < 2 * s. bfac j (bit2 s x y j) :: 'a mpoly))
by blast
from a have a0: a = 0
by (simp add: monom.rep-eq split: if-splits)
have mdeg b ≤ total-degree (∏ j < 2 * s. bfac j (bit2 s x y j) :: 'a mpoly)
by (rule mdeg-le-total-degree[OF b])
also have ... ≤ (∑ j < 2 * s. total-degree (bfac j (bit2 s x y j) :: 'a mpoly))
by (rule total-degree-prod-le)
also have ... ≤ (∑ j ∈ {.. $2 * s$ }. 1)
by (intro sum-mono total-degree-bfac-le)
finally have mdeg b ≤  $2 * s$  by simp
then show mdeg m ≤  $2 * s$ 
by (simp add: m a0 mdeg-plus)
qed
show ?thesis
unfolding table-repr-def
by (intro total-degree-sum-le term-bound)
qed

```

36 Discharging the representation premise (non-vacuity, bridge to the concrete verifier)

Every layer sumcheck instance has an actual mpoly representative: the interpolation of its integrand, with the width-dependent degree bound $2s$ - exactly the $dbnd = \lambda s. 2 * s$ function the assembly soundness theorems consume in this instance.

```

theorem layer-repr-mpoly:
  fixes L :: 'a::{finite, field} layer
  shows gkr-mpoly.layer-poly-repr
    (table-repr s (layer-integrand L s claims below))
    L s claims below (2 * s)
  unfolding gkr-mpoly.layer-poly-repr-def
  by (intro conjI vars-table-repr total-degree-table-repr)
    (simp add: insertion-table-repr)

```

The B2 soundness theorem is consumable with this witness: binding the soundness theorem against the interpolation representative yields the per-layer bound $2s \cdot 2s / |F|$ with every representation premise discharged - the premise chain is inhabited end to end. (Stated as a fact binding: the sumcheck predicate is a locale constant of the AFP development, so the specialised statement lives in the instance's terms rather than a fresh top-level formula.)

lemmas layer-sumcheck-soundness-via-mpoly =

```
gkr-mpoly.layer-sumcheck-soundness[OF layer-repr-mpoly]
```

```
end
```

```
theory GKR-Assembly
  imports Sumcheck-Instance
begin
```

37 The chain recurrence of witness value vectors

Value vector i is the layer- i evaluation of value vector $i+1$ (output layer first, Rust *CircuitWitness::layer-values* ordering).

```
lemma circuit-values-step:
  fixes Ls :: 'f::comm-ring-1 layer list
  assumes i < length Ls
  shows circuit-values Ls inputs ! i
    = layer-eval (Ls ! i) (circuit-values Ls inputs ! Suc i)
  using assms
proof (induction Ls arbitrary: i)
  case Nil
  then show ?case by simp
next
  case (Cons L Ls)
  show ?case
  proof (cases i)
    case 0
    have ne: circuit-values Ls inputs ≠ [] by (rule circuit-values-nonempty)
    show ?thesis
    by (simp add: 0 Let-def hd-conv-nth[OF ne])
  next
    case (Suc j)
    then have j < length Ls using Cons.prems by simp
    then show ?thesis
    by (simp add: Suc Let-def Cons.IH)
  qed
qed
```

38 Honest acceptance at a chain position (B1 corollary, via B2)

```
context gkr-layer-sumcheck
begin
```

At chain position i , the verifier's true targets are the MLE evaluations of the layer- i value vector (Rust: *targets* after the layer- i end check). A claim value assembled from the true targets IS the layer claim sum for the layer-below value vector, so the honest prover is accepted on every challenge tuple. This is the honest-prover acceptance corollary of the hypercube identity, lifted to the chain.

```
theorem gkr-layer-reduction-complete:
```

```

fixes  $Ls :: 'a$  layer list
assumes  $i\text{-lt}: i < \text{length } Ls$ 
  and  $\text{repr}: \text{layer-poly-repr } p (Ls ! i) s \text{ claims } (\text{circuit-values } Ls \text{ inputs } ! \text{Suc } i) \text{ dbnd}$ 
  and  $\text{gates-in}: \forall g \in \text{set } (\text{layer-gates } (Ls ! i)).$ 
     $g\text{-in1 } g < 2^{\wedge} s \wedge g\text{-in2 } g < 2^{\wedge} s$ 
  and  $\text{true-targets}: v = (\sum (z, c) \leftarrow \text{claims}.$ 
     $c * (\text{mle } (\text{layer-width-bits } (Ls ! i)) (!) (\text{circuit-values } Ls \text{ inputs } ! i)) z$ 
     $- \text{const-mle } (Ls ! i) z))$ 
  and  $\text{len-rs}: \text{length } rs = 2 * s$ 
shows  $\text{sumcheck honest-prover } u (\{0, 1\}, p, v) r (\text{zip } (\text{upt } 0 (2 * s)) rs)$ 
proof –
  have  $v = \text{layer-claim-sum } (Ls ! i) \text{ claims } (\text{circuit-values } Ls \text{ inputs } ! \text{Suc } i)$ 
  unfolding  $\text{true-targets layer-claim-sum-def}$ 
  by  $(\text{simp add: circuit-values-step[OF } i\text{-lt}])$ 
  then show  $?thesis$ 
  using  $\text{layer-sumcheck-completeness[OF repr gates-in - len-rs]}$  by  $\text{simp}$ 
qed

end

```

39 Two-point carry combination (B3 building block)

A nonzero affine polynomial in one fresh uniform challenge vanishes with probability at most $1/|a|$ (one root if the slope is nonzero, none otherwise).

```

lemma  $\text{affine-root-prob}:$ 
  fixes  $a \ b :: 'a :: \{\text{finite}, \text{field}\}$ 
  assumes  $a \neq 0 \vee b \neq 0$ 
  shows  $\text{measure-pmf.prob } (\text{pmf-of-set } (UNIV :: 'a \text{ set})) \{r. a + r * b = 0\}$ 
     $\leq 1 / \text{real CARD}('a)$ 
proof  $(\text{cases } b = 0)$ 
  case  $\text{True}$ 
  with  $\text{assms have } a \neq 0$  by  $\text{simp}$ 
  then have  $\{r :: 'a. a + r * b = 0\} = \{\}$  using  $\text{True}$  by  $\text{simp}$ 
  then show  $?thesis$  by  $\text{simp}$ 
next
  case  $\text{False}$ 
  have  $\{r :: 'a. a + r * b = 0\} = \{- (a / b)\}$ 
  proof
    show  $\{r :: 'a. a + r * b = 0\} \subseteq \{- (a / b)\}$ 
    proof
      fix  $r :: 'a$  assume  $r \in \{r. a + r * b = 0\}$ 
      then have  $r * b = - a$  by  $(\text{simp add: eq-neg-iff-add-eq-0 add.commute})$ 
      with  $\text{False have } r = - a / b$  by  $(\text{metis eq-divide-imp mult.commute})$ 
      then show  $r \in \{- (a / b)\}$  by  $(\text{simp add: divide-minus-left})$ 
    qed
    show  $\{- (a / b)\} \subseteq \{r :: 'a. a + r * b = 0\}$ 
    using  $\text{False}$  by  $(\text{auto simp add: field-simps})$ 
  qed
  then show  $?thesis$ 
  by  $(\text{simp add: measure-pmf-of-set})$ 
qed

```

The carry step of reduce.rs: after layer i the residual evaluations ($eval\text{-}x$ at x^* , $eval\text{-}y$ at y^*) become the next incoming claims, combined with coefficients $(1, r)$ for a fresh challenge r . If either claimed target differs from the true one, the combined claims collide with probability at most $1/|a|$ over r .

lemma *two-point-carry-prob*:
fixes $tx\ ty\ vx\ vy :: 'a::\{finite, field\}$
assumes $tx \neq vx \vee ty \neq vy$
shows $measure\text{-}pmf.\text{prob}\ (pmf\text{-}of\text{-}set\ (UNIV :: 'a\ set))$
 $\{r. tx + r * ty = vx + r * vy\}$
 $\leq 1 / \text{real}\ CARD('a)$
proof –
have $eq: \{r :: 'a. tx + r * ty = vx + r * vy\}$
 $= \{r. (tx - vx) + r * (ty - vy) = 0\}$
by (*auto simp add: algebra-simps*)
have $tx - vx \neq 0 \vee ty - vy \neq 0$ **using** *assms* **by** *auto*
then show *?thesis* **unfolding** *eq* **by** (*rule affine-root-prob*)
qed

The same, phrased against the layer claim sum: a carried claim value assembled from FALSE targets (tx, ty) equals the true combined claim *layer-claim-sum* $L [(x^*, 1), (y^*, r)]$ below with probability at most $1/|a|$ over the fresh combination challenge r . This is the per-layer "+1/CARD" summand of the B3 union bound.

lemma *carried-claim-carry-sound*:
fixes $L :: 'a::\{finite, field\}$ **layer** **and** $below :: 'a\ list$
assumes $tx \neq mle\ (layer\text{-}width\text{-}bits\ L)\ (!)\ (layer\text{-}eval\ L\ below))\ xstar \vee$
 $ty \neq mle\ (layer\text{-}width\text{-}bits\ L)\ (!)\ (layer\text{-}eval\ L\ below))\ ystar$
shows $measure\text{-}pmf.\text{prob}\ (pmf\text{-}of\text{-}set\ (UNIV :: 'a\ set))$
 $\{r. (tx - const\text{-}mle\ L\ xstar) + r * (ty - const\text{-}mle\ L\ ystar)$
 $= layer\text{-}claim\text{-}sum\ L [(xstar, 1), (ystar, r)]\ below\}$
 $\leq 1 / \text{real}\ CARD('a)$
proof –
let $?M = \lambda zpt. mle\ (layer\text{-}width\text{-}bits\ L)\ (!)\ (layer\text{-}eval\ L\ below))\ zpt$
have $expand: \bigwedge r. layer\text{-}claim\text{-}sum\ L [(xstar, 1), (ystar, r)]\ below$
 $= (?M\ xstar - const\text{-}mle\ L\ xstar) + r * (?M\ ystar - const\text{-}mle\ L\ ystar)$
by (*simp add: layer-claim-sum-def*)
have $set\text{-}eq: \{r. (tx - const\text{-}mle\ L\ xstar) + r * (ty - const\text{-}mle\ L\ ystar)$
 $= layer\text{-}claim\text{-}sum\ L [(xstar, 1), (ystar, r)]\ below\}$
 $= \{r. tx + r * ty = ?M\ xstar + r * ?M\ ystar\}$
unfolding *expand* **by** (*auto simp add: algebra-simps*)
show *?thesis* **unfolding** *set-eq*
by (*rule two-point-carry-prob*) (*use assms in auto*)
qed

40 Multilinear Schwartz-Zippel over point tuples (B3 building block)

Head-coordinate decomposition of the MLE: at a point $p \# ps$ the MLE is AFFINE in the head coordinate p , interpolating between the low-half and high-half tables (variable 0 is the MSB, matching poly.rs binding).

lemma *mle-cons*:

fixes $p :: 'f::comm-ring-1$ **and** $ps :: 'f list$

assumes len : $length\ ps = n$

shows $mle\ (Suc\ n)\ f\ (p\ \# \ ps)$

$= (1 - p) * mle\ n\ f\ ps + p * mle\ n\ (\lambda i. f\ (i + 2^{\wedge} n))\ ps$

proof –

have $split2$: $\{.. $(2::nat) ^ n\} = \{.. $2 ^ n\} \cup (\lambda i. i + 2 ^ n) \cdot \{.. $2 ^ n\}$$$$

proof

show $\{.. $(2::nat) ^ n\} \subseteq \{.. $2 ^ n\} \cup (\lambda i. i + 2 ^ n) \cdot \{.. $2 ^ n\}$$$$

proof

fix x **assume** $x \in \{.. $(2::nat) ^ n\}$$

then have $x < 2 * 2^{\wedge} n$ **by** *simp*

then show $x \in \{.. $2 ^ n\} \cup (\lambda i. i + 2 ^ n) \cdot \{.. $2 ^ n\}$$$

proof (*cases* $x < 2^{\wedge} n$)

case *False*

then have $x - 2^{\wedge} n < 2^{\wedge} n$ **and** $x = (x - 2^{\wedge} n) + 2^{\wedge} n$

using $\langle x < 2 * 2^{\wedge} n \rangle$ **by** *simp-all*

then show *?thesis* **by** *force*

qed *simp*

qed

show $\{.. $2 ^ n\} \cup (\lambda i. i + 2 ^ n) \cdot \{.. $2 ^ n\} \subseteq \{.. $(2::nat) ^ n\}$$$$

by *auto*

qed

have *inj*: *inj-on* $(\lambda i. i + (2::nat) ^ n)\ \{.. $2 ^ n\}$$

by (*simp add: inj-on-def*)

have *disj*: $\{.. $(2::nat) ^ n\} \cap (\lambda i. i + 2 ^ n) \cdot \{.. $2 ^ n\} = \{\}$$$

by *auto*

have $mle\ (Suc\ n)\ f\ (p\ \# \ ps) = (\sum idx \in \{.. $2 ^ n\}. eq-pi\ (p\ \# \ ps)\ idx * f\ idx)$$

by (*simp add: mle-def*)

also have $\dots = (\sum idx < 2^{\wedge} n. eq-pi\ (p\ \# \ ps)\ idx * f\ idx) + (\sum idx < 2^{\wedge} n. eq-pi\ (p\ \# \ ps)\ (idx + 2^{\wedge} n) * f\ (idx + 2^{\wedge} n))$

by (*subst split2*) (*simp add: sum.union-disjoint disj inj sum.reindex*)

also have $\dots = (1 - p) * (\sum idx < 2^{\wedge} n. eq-pi\ ps\ idx * f\ idx) + p * (\sum idx < 2^{\wedge} n. eq-pi\ ps\ idx * f\ (idx + 2^{\wedge} n))$

proof –

have *lo*: $(\sum idx < 2^{\wedge} n. eq-pi\ (p\ \# \ ps)\ idx * f\ idx)$

$= (\sum idx < 2^{\wedge} n. (1 - p) * (eq-pi\ ps\ idx * f\ idx))$

proof (*intro sum.cong refl*)

fix $idx :: nat$ **assume** $idx \in \{.. $2 ^ n\}$$

then have $idx < 2^{\wedge} length\ ps$ **using** *len* **by** *simp*

then show $eq-pi\ (p\ \# \ ps)\ idx * f\ idx = (1 - p) * (eq-pi\ ps\ idx * f\ idx)$

by (*simp add: eq-pi-cons-lo mult.assoc*)

qed

have *hi*: $(\sum idx < 2^{\wedge} n. eq-pi\ (p\ \# \ ps)\ (idx + 2^{\wedge} n) * f\ (idx + 2^{\wedge} n))$

$= (\sum idx < 2^{\wedge} n. p * (eq-pi\ ps\ idx * f\ (idx + 2^{\wedge} n)))$

proof (*intro sum.cong refl*)

fix $idx :: nat$ **assume** $idx \in \{.. $2 ^ n\}$$

then have $idx < 2^{\wedge} length\ ps$ **using** *len* **by** *simp*

have $eq-pi\ (p\ \# \ ps)\ (idx + 2^{\wedge} n) = p * eq-pi\ ps\ idx$

using *eq-pi-cons-hi*[*OF idxlt*] *len* **by** *simp*

then show $eq-pi\ (p\ \# \ ps)\ (idx + 2^{\wedge} n) * f\ (idx + 2^{\wedge} n)$

$= p * (eq-pi\ ps\ idx * f\ (idx + 2^{\wedge} n))$

by (*simp add: mult.assoc*)

```

qed
show ?thesis by (simp add: lo hi sum-distrib-left)
qed
also have ... = (1 - p) * mle n f ps + p * mle n (λi. f (i + 2 ^ n)) ps
  by (simp add: mle-def)
finally show ?thesis .
qed

```

Schwartz-Zippel for multilinear value tables: a not-identically-zero table has a vanishing MLE at a uniformly random point tuple with probability at most $n/|a|$. This is the output-layer seed of the B3 chain (the claimed-output MLE vs the true output MLE at the random $z=0$), phrased over the difference table. Proof mirrors the AFP *soundness-inductive* recursion: fix the head coordinate, at most ONE head value can make the residual table identically zero, and the induction hypothesis covers the rest.

```

lemma mle-zero-prob:
  fixes f :: nat ⇒ 'a::{finite, field}
  assumes ∃ idx < 2 ^ n. f idx ≠ 0
  shows measure-pmf.prob (pmf-of-set (tuples UNIV n)) {zs. mle n f zs = 0}
    ≤ real n / real CARD('a)
  using assms
proof (induction n arbitrary: f)
  case 0
  then have f0: f 0 ≠ 0 by auto
  have t0: tuples (UNIV :: 'a set) 0 = {} by (rule tuples-Zero)
  have mle 0 f ({} :: 'a list) = f 0 by (simp add: mle-def)
  then have {} :: 'a list ∩ {zs. mle 0 f zs = 0} = {} using f0 by auto
  then have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) 0))
    {zs. mle 0 f zs = 0} = 0
    by (simp add: t0 measure-pmf-of-set)
  then show ?case by simp
next
  case (Suc n)
  let ?q = real CARD('a)
  let ?g = λa i. (1 - a) * f i + a * f (i + 2 ^ n)
  let ?P = λa. measure-pmf.prob (pmf-of-set (tuples UNIV n)) {rs. mle n (?g a) rs = 0}

  have q-pos: 0 < ?q by (simp add: card-gt-0-iff)

  — fix the head coordinate
  have measure-pmf.prob (pmf-of-set (tuples UNIV (Suc n))) {zs. mle (Suc n) f zs = 0}
    = (∑ a ∈ (UNIV :: 'a set).
      measure-pmf.prob (pmf-of-set (tuples UNIV n)) {rs. mle (Suc n) f (a # rs) = 0})
      / ?q
    by (rule prob-tuples-fixed-hd)

  — on the support, the head-fixed MLE is the MLE of the blended table
  moreover have blend: ∧ a. measure-pmf.prob (pmf-of-set (tuples UNIV n))
    {rs. mle (Suc n) f (a # rs) = 0} = ?P a

  proof —
    fix a :: 'a
    have measure-pmf.prob (pmf-of-set (tuples UNIV n)) {rs. mle (Suc n) f (a # rs) = 0}
      = measure-pmf.prob (pmf-of-set (tuples UNIV n)) {rs. mle n (?g a) rs = 0}

```

```

proof (rule prob-cong)
  fix rs :: 'a list assume rs ∈ set-pmf (pmf-of-set (tuples UNIV n))
  then have rs ∈ tuples (UNIV :: 'a set) n
    by (simp add: set-pmf-of-set tuples-finite)
  then have len: length rs = n by (auto)
  have mle (Suc n) f (a # rs)
    = (1 - a) * mle n f rs + a * mle n (λi. f (i + 2 ^ n)) rs
    by (rule mle-cons[OF len])
  also have ... = mle n (?g a) rs
  proof -
    have mle n (?g a) rs
      = mle n (λi. (1 - a) * f i) rs + mle n (λi. a * f (i + 2 ^ n)) rs
      using mle-add[of n λi. (1 - a) * f i λi. a * f (i + 2 ^ n) rs] by simp
    also have ... = (1 - a) * mle n f rs + a * mle n (λi. f (i + 2 ^ n)) rs
      by (simp add: mle-scale)
    finally show ?thesis by simp
  qed
  finally show (rs ∈ {rs. mle (Suc n) f (a # rs) = 0})
    = (rs ∈ {rs. mle n (?g a) rs = 0}) by simp
qed
then show measure-pmf.prob (pmf-of-set (tuples UNIV n))
  {rs. mle (Suc n) f (a # rs) = 0} = ?P a by simp
qed

— at most one head value blends the table to identically zero
moreover have (∑ a∈(UNIV::'a set). ?P a) ≤ real n + 1
proof -
  obtain i0 where i0: i0 < 2 ^ Suc n f i0 ≠ 0 using Suc.prem by auto
  obtain j where j: j < 2 ^ n and jnz: f j ≠ 0 ∨ f (j + 2 ^ n) ≠ 0
  proof (cases i0 < 2 ^ n)
    case True
      then show ?thesis using i0 that[of i0] by auto
    next
      case False
        have i0 - 2 ^ n < 2 ^ n using i0(1) False by simp
        moreover have (i0 - 2 ^ n) + 2 ^ n = i0 using False by simp
        ultimately show ?thesis using i0(2) that[of i0 - 2 ^ n] by auto
  qed
  have bad-sub: {a. ∀ i < 2 ^ n. ?g a i = 0} ⊆ {a. ?g a j = 0}
    using j by auto
  have bad-card: card {a :: 'a. ?g a j = 0} ≤ 1
  proof (cases f (j + 2 ^ n) = f j)
    case True
      then have ∧a. ?g a j = f j by (simp add: algebra-simps)
      moreover have f j ≠ 0 using True jnz by auto
      ultimately have {a :: 'a. ?g a j = 0} = {} by simp
      then show ?thesis by simp
    next
      case False
        have {a :: 'a. ?g a j = 0} ⊆ {f j / (f j - f (j + 2 ^ n))}
        proof
          fix a assume a ∈ {a. ?g a j = 0}
          then have f j + a * (f (j + 2 ^ n) - f j) = 0 by (simp add: algebra-simps)

```

```

then have  $a * (f j - f (j + 2 \wedge n)) = f j$  by (simp add: algebra-simps)
then have  $a = f j / (f j - f (j + 2 \wedge n))$ 
  using False by (metis eq-divide-imp right-minus-eq)
then show  $a \in \{f j / (f j - f (j + 2 \wedge n))\}$  by simp
qed
then have card  $\{a :: 'a. ?g a j = 0\}$ 
   $\leq$  card  $\{f j / (f j - f (j + 2 \wedge n))\}$ 
  using card-mono[of  $\{f j / (f j - f (j + 2 \wedge n))\}$   $\{a. ?g a j = 0\}$ ]
  by simp
then show ?thesis by simp
qed
have IH-good:  $\bigwedge a. \exists i < 2 \wedge n. ?g a i \neq 0 \implies ?P a \leq \text{real } n / ?q$ 
  by (rule Suc.IH)
have P-le-1:  $\bigwedge a. ?P a \leq 1$ 
  by (rule measure-pmf.prob-le-1)
have  $(\sum a \in (UNIV :: 'a \text{ set}). ?P a)$ 
   $\leq (\sum a \in (UNIV :: 'a \text{ set}). \text{if } \forall i < 2 \wedge n. ?g a i = 0 \text{ then } 1 \text{ else } \text{real } n / ?q)$ 
proof (intro sum-mono)
  fix  $a :: 'a$  assume  $a \in UNIV$ 
  show  $?P a \leq (\text{if } \forall i < 2 \wedge n. ?g a i = 0 \text{ then } 1 \text{ else } \text{real } n / ?q)$ 
  proof (cases  $\forall i < 2 \wedge n. ?g a i = 0$ )
    case True
      then show ?thesis using P-le-1 by simp
    next
      case False
        then have  $\exists i < 2 \wedge n. ?g a i \neq 0$  by auto
        then show ?thesis using IH-good by simp
  qed
qed
also have  $\dots = (\sum a \in \{a :: 'a. \forall i < 2 \wedge n. ?g a i = 0\}. 1)$ 
   $+ (\sum a \in (UNIV :: 'a \text{ set}) - \{a. \forall i < 2 \wedge n. ?g a i = 0\}. \text{real } n / ?q)$ 
proof -
  have set-eq:  $\{a :: 'a. \exists i < 2 \wedge n. ?g a i \neq 0\}$ 
     $= (UNIV :: 'a \text{ set}) - \{a. \forall i < 2 \wedge n. ?g a i = 0\}$ 
  by auto
  show ?thesis
    by (subst sum.If-cases) (simp-all add: set-eq Compl-eq-Diff-UNIV)
qed
also have  $\dots \leq 1 + \text{real } CARD('a) * (\text{real } n / ?q)$ 
proof -
  have  $(\sum a \in \{a :: 'a. \forall i < 2 \wedge n. ?g a i = 0\}. (1::\text{real}))$ 
     $= \text{real } (\text{card } \{a :: 'a. \forall i < 2 \wedge n. ?g a i = 0\})$  by simp
  also have  $\dots \leq 1$ 
    using card-mono[OF bad-sub] bad-card
    by (smt (verit) of-nat-le-1-iff finite-code le-trans)
  finally have first:  $(\sum a \in \{a :: 'a. \forall i < 2 \wedge n. ?g a i = 0\}. (1::\text{real})) \leq 1$  .
  have  $(\sum a \in (UNIV :: 'a \text{ set}) - \{a. \forall i < 2 \wedge n. ?g a i = 0\}. \text{real } n / ?q)$ 
     $= \text{real } (\text{card } ((UNIV :: 'a \text{ set}) - \{a. \forall i < 2 \wedge n. ?g a i = 0\})) * (\text{real } n / ?q)$ 
  by simp
  also have  $\dots \leq \text{real } CARD('a) * (\text{real } n / ?q)$ 
    by (intro mult-right-mono) (simp-all add: card-mono)
  finally have second:  $(\sum a \in (UNIV :: 'a \text{ set}) - \{a. \forall i < 2 \wedge n. ?g a i = 0\}. \text{real } n / ?q)$ 
     $\leq \text{real } CARD('a) * (\text{real } n / ?q)$  .

```

```

    show ?thesis using first second by simp
qed
also have ... = real n + 1 using q-pos by simp
finally show ?thesis by simp
qed
ultimately have measure-pmf.prob (pmf-of-set (tuples UNIV (Suc n)))
  {zs. mle (Suc n) f zs = 0} ≤ (real n + 1) / ?q
  by (simp add: divide-right-mono)
then show ?case by (simp add: add.commute)
qed

```

The output-layer seed in two-table form: if the claimed output table differs anywhere on the cube from the true one, their MLEs agree at a uniformly random z -0 tuple with probability at most $n/|a|$ (reduce.rs::verify draws z -0 and evaluates the claimed-output MLE m_0 ; the chain starts from the claim V -0 (z -0) = m_0).

```

lemma mle-agree-prob:
  fixes f g :: nat ⇒ 'a::{finite, field}
  assumes ∃ idx < 2 ^ n. f idx ≠ g idx
  shows measure-pmf.prob (pmf-of-set (tuples UNIV n)) {zs. mle n f zs = mle n g zs}
    ≤ real n / real CARD('a)
proof -
  have diff: ∃ idx < 2 ^ n. f idx - g idx ≠ 0
    using assms by auto
  have {zs :: 'a list. mle n f zs = mle n g zs} = {zs. mle n (λ i. f i - g i) zs = 0}
  proof -
    have ∧ zs :: 'a list. mle n (λ i. f i - g i) zs = mle n f zs - mle n g zs
      by (simp add: mle-def sum-subtractf algebra-simps)
    then show ?thesis by auto
  qed
then show ?thesis
  using mle-zero-prob[OF diff] by simp
qed

```

41 Tuple-space splitting tools (B3 probability plumbing)

The AFP Probability_Tools provide single-head splitting (*prob-tuples-fixed-hd*). The chain bound consumes per-layer SEGMENTS of the flat challenge tuple, so we derive the k -element generalisation, its prefix-marginal corollary, and the single-coordinate bridge to a plain uniform draw.

```

lemma card-tuples-UNIV: card (tuples (UNIV :: 'a::finite set) n) = CARD('a) ^ n
  unfolding tuples-def
  using card-lists-length-eq[of UNIV :: 'a set] by simp

```

```

lemma prob-tuples-split-seg:
  fixes E :: 'a::finite list ⇒ bool
  shows measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (k + m))) {zs. E zs}
    = (∑ xs ∈ tuples (UNIV :: 'a set) k.
      measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m)) {ys. E (xs @ ys)})
      / real (CARD('a)) ^ k
proof (induction k arbitrary: E)

```

```

case 0
show ?case by (simp add: tuples-Zero)
next
case (Suc k)
have hd-split:
  measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (Suc (k + m)))) {zs. E zs}
  = (∑ a ∈ (UNIV :: 'a set).
    measure-pmf.prob (pmf-of-set (tuples UNIV (k + m))) {rs. E (a # rs)})
    / real CARD('a)
  by (rule prob-tuples-fixed-hd)
have IHa: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (k + m))) {rs. E (a # rs)}
  = (∑ xs ∈ tuples (UNIV :: 'a set) k.
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m)) {ys. E (a # xs @ ys)})
    / real (CARD('a) ^ k for a
  by (rule Suc.IH)
have bij: bij-betw (λ(a, xs). a # xs) ((UNIV :: 'a set) × tuples (UNIV :: 'a set) k)
  (tuples (UNIV :: 'a set) (Suc k))
  by (intro bij-betw-imageI) (auto simp add: inj-on-def tuples-Suc)
have reindex:
  (∑ xs' ∈ tuples (UNIV :: 'a set) (Suc k).
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m)) {ys. E (xs' @ ys)})
  = (∑ p ∈ (UNIV :: 'a set) × tuples (UNIV :: 'a set) k.
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m))
      {ys. E ((case p of (a, xs) ⇒ a # xs) @ ys)})
  by (rule sum.reindex-bij-betw[OF bij, symmetric])
have prod-sum:
  (∑ p ∈ (UNIV :: 'a set) × tuples (UNIV :: 'a set) k.
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m))
      {ys. E ((case p of (a, xs) ⇒ a # xs) @ ys)})
  = (∑ a ∈ (UNIV :: 'a set). ∑ xs ∈ tuples (UNIV :: 'a set) k.
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m)) {ys. E (a # xs @ ys)})
  by (simp add: sum.cartesian-product case-prod-beta)
have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (Suc k + m))) {zs. E zs}
  = (∑ a ∈ (UNIV :: 'a set).
    (∑ xs ∈ tuples (UNIV :: 'a set) k.
      measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m)) {ys. E (a # xs @ ys)})
    / real (CARD('a) ^ k) / real CARD('a)
  using hd-split IHa by simp
also have ... = (∑ a ∈ (UNIV :: 'a set). ∑ xs ∈ tuples (UNIV :: 'a set) k.
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m))
      {ys. E (a # xs @ ys)})
    / real (CARD('a) ^ k / real CARD('a)
  by (simp add: sum-divide-distrib)
also have ... = (∑ xs' ∈ tuples (UNIV :: 'a set) (Suc k).
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m))
      {ys. E (xs' @ ys)})
    / real (CARD('a) ^ Suc k
  unfolding reindex prod-sum by (simp add: divide-divide-eq-left mult.commute)
finally show ?case by simp
qed

```

lemma prob-tuples-prefix:
fixes E :: 'a::finite list ⇒ bool

```

shows measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (k + m))) {zs. E (take k zs)}
  = measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) k)) {xs. E xs}
proof -
  have per: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m))
    {ys. E (take k (xs @ ys))}
    = (if E xs then 1 else 0) if xs: xs ∈ tuples (UNIV :: 'a set) k for xs
  proof -
    from xs have len: length xs = k by auto
    have tk: take k (xs @ ys) = xs for ys :: 'a list
    using len by simp
    show ?thesis
    proof (cases E xs)
      case True
      then have {ys :: 'a list. E (take k (xs @ ys))} = UNIV using tk by auto
      then show ?thesis
        using True by (simp add: measure-pmf-of-set)
    next
      case False
      then have {ys :: 'a list. E (take k (xs @ ys))} = {} using tk by auto
      then show ?thesis using False by simp
    qed
  qed
have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (k + m))) {zs. E (take k zs)}
  = (∑ xs ∈ tuples (UNIV :: 'a set) k.
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) m)) {ys. E (take k (xs @ ys))})
    / real (CARD('a)) ^ k
  by (rule prob-tuples-split-seg)
also have ... = (∑ xs ∈ tuples (UNIV :: 'a set) k. if E xs then 1 else 0)
  / real (CARD('a)) ^ k
  by (intro arg-cong2[where f = (/)] sum.cong refl per)
also have ... = real (card (tuples (UNIV :: 'a set) k ∩ {xs. E xs}))
  / real (CARD('a)) ^ k
proof -
  have (∑ xs ∈ tuples (UNIV :: 'a set) k. if E xs then (1::real) else 0)
    = (∑ xs ∈ tuples (UNIV :: 'a set) k ∩ {xs. E xs}. 1)
  by (subst sum.If-cases) simp-all
  then show ?thesis by simp
qed
also have ... = measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) k)) {xs. E xs}
  by (simp add: measure-pmf-of-set card-tuples-UNIV)
finally show ?thesis .
qed

```

lemma prob-tuples-single:

```

fixes Q :: 'a::finite ⇒ bool
shows measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) 1)) {ys. Q (ys ! 0)}
  = measure-pmf.prob (pmf-of-set (UNIV :: 'a set)) {r. Q r}
proof -
  have z: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) 0))
    (if Q a then UNIV else {}) = (if Q a then 1 else 0) for a :: 'a
  by (cases Q a) (simp-all add: tuples-Zero measure-pmf-of-set)
  have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (Suc 0))) {ys. Q (ys ! 0)}
    = (∑ a ∈ (UNIV :: 'a set).

```

```

      measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) 0)) {rs. Q ((a # rs) ! 0)}
    / real CARD('a)
  by (rule prob-tuples-fixed-hd)
also have ... = (∑ a ∈ (UNIV :: 'a set). if Q a then 1 else 0) / real CARD('a)
  by (simp add: z)
also have ... = real (card ((UNIV :: 'a set) ∩ {r. Q r})) / real CARD('a)
  by (subst sum.If-cases) simp-all
also have ... = measure-pmf.prob (pmf-of-set (UNIV :: 'a set)) {r. Q r}
  by (simp add: measure-pmf-of-set)
finally show ?thesis by simp
qed

```

42 The layer-reduction chain event (B3)

The chain randomness is one flat tuple: per layer, $2s$ sumcheck challenges followed by ONE carry challenge (reduce.rs::verify samples r after observing the carried evaluations). *chain-rlen* is its total length; *gkr-chain-wf* pins the shape invariants of the layer chain: gate fan-in inside the below-cube, the head width being the next layer's width (reduce.rs *s-in*), and the below vector being the next layer's evaluation (the defining recurrence of *circuit-values*).

definition *chain-rlen* :: *nat list* $\Rightarrow$ *nat* **where**

$$\text{chain-rlen } ss = (\sum s \leftarrow ss. 2 * s + 1)$$

lemma *chain-rlen-simps* [simp]:

$$\text{chain-rlen } [] = 0$$

$$\text{chain-rlen } (s \# ss) = 2 * s + 1 + \text{chain-rlen } ss$$

by (simp-all add: chain-rlen-def)

fun *gkr-chain-wf* :: *nat list* $\Rightarrow$ '*f::comm-ring-1 layer list* $\Rightarrow$ '*f list list* $\Rightarrow$ *bool* **where**

$$\text{gkr-chain-wf } [] [] [] \longleftrightarrow \text{True}$$

| *gkr-chain-wf* [s] [L] [b] $\longleftrightarrow$

$$(\forall g \in \text{set } (\text{layer-gates } L). \text{g-in1 } g < 2^s \wedge \text{g-in2 } g < 2^s)$$

| *gkr-chain-wf* (s # s' # ss) (L # L' # Ls) (b # b' # bs) $\longleftrightarrow$

$$(\forall g \in \text{set } (\text{layer-gates } L). \text{g-in1 } g < 2^s \wedge \text{g-in2 } g < 2^s) \wedge$$

$$s = \text{layer-width-bits } L' \wedge b = \text{layer-eval } L' b' \wedge$$

$$\text{gkr-chain-wf } (s' \# ss) (L' \# Ls) (b' \# bs)$$

| *gkr-chain-wf* ss Ls bels $\longleftrightarrow$ False

The chain BAD event, for an arbitrary adversary. The adversary A supplies, per layer index and per randomness prefix, an AFP prover (with its state and previous-challenge input) and the two carried evaluations (tx , ty) (reduce.rs: *lp.eval-x/lp.eval-y*) - the prover triple read at the layer entry (empty local prefix), the carried evaluations after the layer's sumcheck challenges. Per layer the event is:

- the AFP sumcheck for the TRUE layer polynomial (the representative $P L s$ claims b) accepts the incoming claim value v , OR
- the carried evaluations are FALSE (differ from the below-vector MLE at the drawn point (x^*, y^*)) and the chain continues on the next incoming claim $[(x^*, 1), (y^*, r)]$ with claim value $(tx - \text{const}^\sim(x^*)) + r (ty - \text{const}^\sim(y^*))$ - literally reduce.rs::verify's carry.

MODEL BOUNDARY (the bridge to the concrete verifier, recorded): the REAL verifier's layer acceptance implies this event. Its final check compares the sumcheck's residual value against the wiring-MLE reconstruction at (x^*, y^*) assembled from (tx, ty) ; when (tx, ty) are the true below-MLE evaluations, that reconstruction IS the representative polynomial's value at the challenge point (the representative agrees with the layer integrand as a polynomial, which the concrete mpoly representative realises at instantiation), so rounds + final check amount to AFP sumcheck acceptance; otherwise the carried evaluations are false, which is the second disjunct. At the LAST layer the carried evaluations are checked directly against the verifier-computed input MLE (lib.rs input-claim discharge, seal S-4), so a false carry is rejected deterministically - hence no continuation disjunct there.

43 The chain union bound (B3)

context *gkr-layer-sumcheck*
begin

fun *gkr-chain-bad* ::

```

('a layer  $\Rightarrow$  nat  $\Rightarrow$  ('a list  $\times$  'a) list  $\Rightarrow$  'a list  $\Rightarrow$  'p)
 $\Rightarrow$  (nat  $\Rightarrow$  'a list  $\Rightarrow$  ('p, 'a, 'a, nat, 's) prover  $\times$  's  $\times$  'a  $\times$  'a  $\times$  'a)
 $\Rightarrow$  nat list  $\Rightarrow$  'a layer list  $\Rightarrow$  'a list list
 $\Rightarrow$  ('a list  $\times$  'a) list  $\Rightarrow$  'a  $\Rightarrow$  'a list  $\Rightarrow$  bool where
gkr-chain-bad P A (s # ss) (L # Ls) (b # bs) claims v rs =
  ((case A 0 [] of (pr, ps, r0, tx0, ty0)  $\Rightarrow$ 
    sumcheck pr ps ({0, 1}, P L s claims b, v) r0
    (zip (upt 0 (2 * s)) (take (2 * s) rs)))
   $\vee$  (Ls  $\neq$  []  $\wedge$ 
    (case A 0 (take (2 * s) rs) of (pr1, ps1, r1, tx, ty)  $\Rightarrow$ 
      (tx  $\neq$  mle s (!) b) (take s rs)  $\vee$ 
      ty  $\neq$  mle s (!) b) (take s (drop s rs)))  $\wedge$ 
      gkr-chain-bad P ( $\lambda j$  pfx. A (Suc j) (take (2 * s + 1) rs @ pfx)) ss Ls bs
      [(take s rs, 1), (take s (drop s rs), rs ! (2 * s))]
      ((tx - const-mle (hd Ls) (take s rs))
       + rs ! (2 * s) * (ty - const-mle (hd Ls) (take s (drop s rs))))
      (drop (2 * s + 1) rs))))
| gkr-chain-bad P A ss Ls bels claims v rs = False

```

The chain event is unfolded ONLY where a proof explicitly says so: under the default simpset the recursive equation would re-expand nested occurrences and normalise the take/drop plumbing of the adversary reindexing into mismatched forms (the environment-sensitive simp class this corpus guards against).

declare *gkr-chain-bad.simps* [*simp del*]

lemma *gkr-chain-bad-nil-empty* [*simp*]:

```

Collect (gkr-chain-bad P A [] [] [] claims v) = {}
by (auto simp add: gkr-chain-bad.simps)

```

One chain step costs at most $2s * dbnd / |a|$ (a false incoming claim surviving the layer sumcheck, via AFP soundness) plus $1 / |a|$ (false carried evaluations colliding with the true

next claim over the fresh carry challenge); the final layer has no carry continuation. The bound telescopes by induction along the layer chain over the flat challenge tuple.

lemma *gkr-chain-bad-bound*:

```

fixes P :: 'a layer  $\Rightarrow$  nat  $\Rightarrow$  ('a list  $\times$  'a) list  $\Rightarrow$  'a list  $\Rightarrow$  'p
and A :: nat  $\Rightarrow$  'a list  $\Rightarrow$  ('p, 'a, 'a, nat, 's) prover  $\times$  's  $\times$  'a  $\times$  'a  $\times$  'a
and dbnd :: nat  $\Rightarrow$  nat
  — the representative degree bound may depend on the layer width: a TOTAL-degree instance (the
  AFP mpoly one) has representatives of degree of order s, so a single constant bound over all widths
  would be undischargable there; a max-per-variable instance discharges the constant function  $\lambda\cdot. 4$ 
assumes wf: gkr-chain-wf ss Ls bels
and repr:  $\bigwedge L\ s\ \text{claims } b. \text{layer-poly-repr } (P\ L\ s\ \text{claims } b)\ L\ s\ \text{claims } b\ (dbnd\ s)$ 
and false0:  $v \neq \text{layer-claim-sum } (hd\ Ls)\ \text{claims } (hd\ bels)$ 
shows measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen ss)))
  {rs. gkr-chain-bad P A ss Ls bels claims v rs}
   $\leq (\sum s \leftarrow ss. \text{real } (2 * s) * \text{real } (dbnd\ s) + 1) / \text{real } CARD('a)$ 
using wf false0
proof (induction ss Ls bels arbitrary: claims v A rule: gkr-chain-wf.induct)
case (2 s L b claims v A)
obtain pr ps r0 tx0 ty0 where A00: A 0 [] = (pr, ps, r0, tx0, ty0)
using prod-cases5 by blast
define E1 where E1 xs  $\longleftrightarrow \text{sumcheck } pr\ ps\ (\{0, 1\}, P\ L\ s\ \text{claims } b, v)\ r0$ 
  (zip (upt 0 (2 * s)) xs) for xs :: 'a list
have gates:  $\forall g \in \text{set } (\text{layer-gates } L). g\text{-in1 } g < 2 \wedge s \wedge g\text{-in2 } g < 2 \wedge s$ 
using 2.premis(1) by simp
have f0:  $v \neq \text{layer-claim-sum } L\ \text{claims } b$ 
using 2.premis(2) by simp
have ev: {rs. gkr-chain-bad P A [s] [L] [b] claims v rs}
  = {rs. E1 (take (2 * s) rs)}
by (auto simp add: gkr-chain-bad.simps A00 E1-def)
have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (2 * s + 1)))
  {rs. E1 (take (2 * s) rs)}
  = measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (2 * s))) {xs. E1 xs}
by (rule prob-tuples-prefix)
also have ...  $\leq \text{real } (2 * s) * \text{real } (dbnd\ s) / \text{real } CARD('a)$ 
unfolding E1-def by (rule layer-sumcheck-soundness[OF repr gates f0])
finally have main: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (2 * s + 1)))
  {rs. gkr-chain-bad P A [s] [L] [b] claims v rs}
   $\leq \text{real } (2 * s) * \text{real } (dbnd\ s) / \text{real } CARD('a)$ 
by (simp add: ev)
have pad:  $\text{real } (2 * s) * \text{real } (dbnd\ s) / \text{real } CARD('a)$ 
   $\leq (\text{real } (2 * s) * \text{real } (dbnd\ s) + 1) / \text{real } CARD('a)$ 
by (intro divide-right-mono) simp-all
show ?case using order-trans[OF main pad] by simp
next
case (3 s s' ss L L' Ls b b' bs claims v A)
let ?q =  $\text{real } CARD('a)$ 
let ?M = chain-rlen (s' # ss)
let ?err' =  $(\sum t \leftarrow s' \# ss. \text{real } (2 * t) * \text{real } (dbnd\ t) + 1) / ?q$ 
obtain pr ps r0 tx0 ty0 where A00: A 0 [] = (pr, ps, r0, tx0, ty0)
using prod-cases5 by blast
define txf where txf xs = fst (snd (snd (snd (A 0 xs)))) for xs
define tyf where tyf xs = snd (snd (snd (snd (A 0 xs)))) for xs

```

have *wfc*: $(\forall g \in \text{set } (\text{layer-gates } L). \text{g-in1 } g < 2^{\wedge} s \wedge \text{g-in2 } g < 2^{\wedge} s) \wedge$
 $s = \text{layer-width-bits } L' \wedge b = \text{layer-eval } L' \text{ } b' \wedge$
 $\text{gkr-chain-wf } (s' \# ss) (L' \# Ls) (b' \# bs)$
using *3.premis(1)* **unfolding** *gkr-chain-wf.simps(3)* .
have *gates*: $\forall g \in \text{set } (\text{layer-gates } L). \text{g-in1 } g < 2^{\wedge} s \wedge \text{g-in2 } g < 2^{\wedge} s$
and *swb*: $s = \text{layer-width-bits } L'$
and *beq*: $b = \text{layer-eval } L' \text{ } b'$
and *wf'*: $\text{gkr-chain-wf } (s' \# ss) (L' \# Ls) (b' \# bs)$
using *wfc* **by** *blast+*
have *f0*: $v \neq \text{layer-claim-sum } L \text{ claims } b$
using *3.premis(2)* **by** *simp*
have *q-pos*: $0 < ?q$ **by** (*simp add: card-gt-0-iff*)
have *err'-nonneg*: $0 \leq ?err'$
by (*intro divide-nonneg-nonneg sum-list-nonneg*) *auto*

— the three sub-events

define *S1* **where** $S1 = \{rs :: 'a \text{ list}.$
 $\text{sumcheck pr ps } (\{0, 1\}, P \text{ } L \text{ } s \text{ claims } b, v) \text{ } r0 \text{ } (\text{zip } (\text{upt } 0 \text{ } (2 * s)) (\text{take } (2 * s) \text{ } rs))\}$
define *TF* **where** $TF \text{ } rs \longleftrightarrow$
 $(\text{txf } (\text{take } (2 * s) \text{ } rs) \neq \text{mle } s \text{ } (!) \text{ } b) (\text{take } s \text{ } rs) \vee$
 $\text{tyf } (\text{take } (2 * s) \text{ } rs) \neq \text{mle } s \text{ } (!) \text{ } b) (\text{take } s \text{ } (\text{drop } s \text{ } rs)))$ **for** $rs :: 'a \text{ list}$
define *NV* **where** $NV \text{ } rs =$
 $(\text{txf } (\text{take } (2 * s) \text{ } rs) - \text{const-mle } L' (\text{take } s \text{ } rs))$
 $+ rs ! (2 * s) * (\text{tyf } (\text{take } (2 * s) \text{ } rs) - \text{const-mle } L' (\text{take } s \text{ } (\text{drop } s \text{ } rs)))$
for $rs :: 'a \text{ list}$
define *NC* **where** $NC \text{ } rs = [(\text{take } s \text{ } rs, 1 :: 'a), (\text{take } s \text{ } (\text{drop } s \text{ } rs), rs ! (2 * s))]$
for $rs :: 'a \text{ list}$
define *S2* **where** $S2 = \{rs :: 'a \text{ list}. TF \text{ } rs \wedge NV \text{ } rs = \text{layer-claim-sum } L' (NC \text{ } rs) \text{ } b'\}$
define *S3* **where** $S3 = \{rs :: 'a \text{ list}. TF \text{ } rs \wedge NV \text{ } rs \neq \text{layer-claim-sum } L' (NC \text{ } rs) \text{ } b' \wedge$
 $\text{gkr-chain-bad } P (\lambda j \text{ } pfx. A (\text{Suc } j) (\text{take } (2 * s + 1) \text{ } rs @ pfx)) (s' \# ss) (L' \# Ls)$
 $(b' \# bs) (NC \text{ } rs) (NV \text{ } rs) (\text{drop } (2 * s + 1) \text{ } rs))\}$

have *unf*: $\text{gkr-chain-bad } P \text{ } A (s \# s' \# ss) (L \# L' \# Ls) (b \# b' \# bs) \text{ claims } v \text{ } rs$
 $\longleftrightarrow (rs \in S1 \vee$
 $(TF \text{ } rs \wedge \text{gkr-chain-bad } P (\lambda j \text{ } pfx. A (\text{Suc } j) (\text{take } (2 * s + 1) \text{ } rs @ pfx))$
 $(s' \# ss) (L' \# Ls) (b' \# bs) (NC \text{ } rs) (NV \text{ } rs) (\text{drop } (2 * s + 1) \text{ } rs)))$
for $rs :: 'a \text{ list}$
by (*subst gkr-chain-bad.simps(1)*)
 $(\text{simp add: } S1\text{-def } TF\text{-def } NV\text{-def } NC\text{-def } \text{txf-def } \text{tyf-def } A00 \text{ case-prod-beta})$
have *subset*: $\{rs. \text{gkr-chain-bad } P \text{ } A (s \# s' \# ss) (L \# L' \# Ls) (b \# b' \# bs) \text{ claims } v \text{ } rs\}$
 $\subseteq S1 \cup S2 \cup S3$

proof

fix $rs :: 'a \text{ list}$
assume $rs \in \{rs. \text{gkr-chain-bad } P \text{ } A (s \# s' \# ss) (L \# L' \# Ls) (b \# b' \# bs) \text{ claims } v \text{ } rs\}$
then have $rs \in S1 \vee$
 $(TF \text{ } rs \wedge \text{gkr-chain-bad } P (\lambda j \text{ } pfx. A (\text{Suc } j) (\text{take } (2 * s + 1) \text{ } rs @ pfx))$
 $(s' \# ss) (L' \# Ls) (b' \# bs) (NC \text{ } rs) (NV \text{ } rs) (\text{drop } (2 * s + 1) \text{ } rs))$
using *unf* **by** *blast*
then show $rs \in S1 \cup S2 \cup S3$
by (*auto simp add: S2-def S3-def*)

qed

— bound the sumcheck sub-event

have $p1$: $\text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) (\text{chain-rlen} (s \# s' \# ss)))) S1$
 $\leq \text{real} (2 * s) * \text{real} (\text{dbnd } s) / ?q$

proof –

have idx : $\text{chain-rlen} (s \# s' \# ss) = 2 * s + (1 + ?M)$ **by** *simp*

have $\text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) (2 * s + (1 + ?M)))) S1$
 $= \text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) (2 * s)))$
 $\{xs. \text{sumcheck pr ps } \{0, 1\}, P L s \text{ claims } b, v\} r0 (\text{zip} (\text{upt } 0 (2 * s)) xs)\}$

unfolding $S1\text{-def}$

by (*rule prob-tuples-prefix*)

also have $\dots \leq \text{real} (2 * s) * \text{real} (\text{dbnd } s) / ?q$

by (*rule layer-sumcheck-soundness[OF repr gates f0]*)

finally show $?thesis$ **by** (*simp add: idx*)

qed

— bound the carry-collision sub-event

have $p2$: $\text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) (\text{chain-rlen} (s \# s' \# ss)))) S2$
 $\leq 1 / ?q$

proof –

define $E2$ **where** $E2 \text{ pfx} \longleftrightarrow TF \text{ pfx} \wedge NV \text{ pfx} = \text{layer-claim-sum } L' (NC \text{ pfx}) b'$

for $\text{pfx} :: 'a \text{ list}$

have stable : $TF (\text{take} (2 * s + 1) rs) = TF rs \wedge NV (\text{take} (2 * s + 1) rs) = NV rs \wedge$
 $NC (\text{take} (2 * s + 1) rs) = NC rs$ **for** $rs :: 'a \text{ list}$

by (*simp add: TF-def NV-def NC-def take-take drop-take min-absorb1 min-absorb2*)

have st1 : $TF (\text{take} (2 * s + 1) rs) = TF rs$ **for** $rs :: 'a \text{ list}$

using stable **by** *blast*

have st2 : $NV (\text{take} (2 * s + 1) rs) = NV rs$ **for** $rs :: 'a \text{ list}$

using stable **by** *blast*

have st3 : $NC (\text{take} (2 * s + 1) rs) = NC rs$ **for** $rs :: 'a \text{ list}$

using stable **by** *blast*

have $S2\text{-pfx}$: $S2 = \{rs. E2 (\text{take} (2 * s + 1) rs)\}$

unfolding $S2\text{-def}$ $E2\text{-def}$ **by** (*simp only: st1 st2 st3*)

have idx : $\text{chain-rlen} (s \# s' \# ss) = (2 * s + 1) + ?M$ **by** *simp*

have $\text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) ((2 * s + 1) + ?M)))$
 $\{rs. E2 (\text{take} (2 * s + 1) rs)\}$

$= \text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) (2 * s + 1))) \{pfx. E2 \text{ pfx}\}$

by (*rule prob-tuples-prefix*)

also have $\dots \leq 1 / ?q$

proof –

have split1 : $\text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) (2 * s + 1)))$

$\{pfx. E2 \text{ pfx}\}$

$= (\sum xs \in \text{tuples} (\text{UNIV} :: 'a \text{ set}) (2 * s).$

$\text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) 1)) \{ys. E2 (xs @ ys)\})$

$/ ?q ^ (2 * s)$

by (*rule prob-tuples-split-seg*)

have per : $\text{measure-pmf.prob} (\text{pmf-of-set} (\text{tuples} (\text{UNIV} :: 'a \text{ set}) 1)) \{ys. E2 (xs @ ys)\}$

$\leq 1 / ?q$ **if** xs : $xs \in \text{tuples} (\text{UNIV} :: 'a \text{ set}) (2 * s)$ **for** xs

proof –

from xs **have** len : $\text{length } xs = 2 * s$ **by** *auto*

have tk1 : $\text{take} (2 * s) (xs @ ys) = xs$ **for** $ys :: 'a \text{ list}$

using len **by** *simp*

have tk2 : $\text{take } s (xs @ ys) = \text{take } s xs$ **for** $ys :: 'a \text{ list}$

using len **by** (*simp add: take-append*)

have tk3 : $\text{take } s (\text{drop } s (xs @ ys)) = \text{drop } s xs$ **for** $ys :: 'a \text{ list}$

```

using len by (simp add: drop-append take-append)
have nth1: (xs @ ys) ! (2 * s) = ys ! 0 for ys :: 'a list
using len by (simp add: nth-append)
show ?thesis
proof (cases txf xs ≠ mle s (!) b) (take s xs) ∨
      tyf xs ≠ mle s (!) b) (drop s xs))
  case True
  have ev2: {ys :: 'a list. E2 (xs @ ys)}
    = {ys. (txf xs - const-mle L' (take s xs))
      + ys ! 0 * (tyf xs - const-mle L' (drop s xs))
      = layer-claim-sum L' [(take s xs, 1), (drop s xs, ys ! 0)] b'}
  using True
  by (auto simp add: E2-def TF-def NV-def NC-def tk1 tk2 tk3 nth1 len)
  have hyp: txf xs ≠ mle (layer-width-bits L') (!) (layer-eval L' b') (take s xs) ∨
    tyf xs ≠ mle (layer-width-bits L') (!) (layer-eval L' b') (drop s xs)
  using True swb beq by simp
  have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) 1)) {ys. E2 (xs @ ys)}
    = measure-pmf.prob (pmf-of-set (UNIV :: 'a set))
      {r. (txf xs - const-mle L' (take s xs))
        + r * (tyf xs - const-mle L' (drop s xs))
        = layer-claim-sum L' [(take s xs, 1), (drop s xs, r)] b'}
  unfolding ev2
  by (rule prob-tuples-single[where Q = λr. (txf xs - const-mle L' (take s xs))
    + r * (tyf xs - const-mle L' (drop s xs))
    = layer-claim-sum L' [(take s xs, 1), (drop s xs, r)] b'])
  also have ... ≤ 1 / ?q
    by (rule carried-claim-carry-sound[OF hyp])
  finally show ?thesis .
next
  case False
  then have {ys :: 'a list. E2 (xs @ ys)} = {}
    by (auto simp add: E2-def TF-def tk1 tk2 tk3 len)
  then show ?thesis using q-pos by simp
qed
qed
have qnz: ?q ^ (2 * s) ≠ 0 using q-pos by simp
have sumb: (∑ xs ∈ tuples (UNIV :: 'a set) (2 * s).
  measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) 1)) {ys. E2 (xs @ ys)})
  ≤ ?q ^ (2 * s) * (1 / ?q)
proof -
  have (∑ xs ∈ tuples (UNIV :: 'a set) (2 * s).
  measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) 1)) {ys. E2 (xs @ ys)})
  ≤ (∑ xs ∈ tuples (UNIV :: 'a set) (2 * s). 1 / ?q)
    by (rule sum-mono) (rule per)
  then show ?thesis by (simp add: card-tuples-UNIV)
qed
have qp2: 0 ≤ ?q ^ (2 * s) by (intro zero-le-power) simp
have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (2 * s + 1))) {pfx. E2 pfx}
  ≤ (?q ^ (2 * s) * (1 / ?q)) / ?q ^ (2 * s)
  unfolding split1 by (rule divide-right-mono[OF sumb qp2])
also have ... = 1 / ?q
  using qnz by simp
finally show ?thesis .

```

qed
 finally show *?thesis* using *S2-pfx idx* by simp
 qed

— bound the continuation sub-event by the induction hypothesis
 have *p3*: *measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s # s' # ss)))) S3*
 $\leq ?err'$
 proof —
 have *idx*: *chain-rlen (s # s' # ss) = (2 * s + 1) + ?M* by simp
 have *split1*: *measure-pmf.prob*
 $(\text{pmf-of-set } (\text{tuples } (\text{UNIV} :: 'a \text{ set}) ((2 * s + 1) + ?M))) S3$
 $= (\sum \text{seg} \in \text{tuples } (\text{UNIV} :: 'a \text{ set}) (2 * s + 1).$
 $\text{measure-pmf.prob } (\text{pmf-of-set } (\text{tuples } (\text{UNIV} :: 'a \text{ set}) ?M))$
 $\{\text{rest. seg @ rest} \in S3\})$
 $/ ?q \wedge (2 * s + 1)$
 using *prob-tuples-split-seg* [where *E* = $\lambda z s. z s \in S3$ and *k* = $2 * s + 1$
 and *m* = *chain-rlen (s' # ss)*] by simp
 have *per*: *measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?M))*
 $\{\text{rest. seg @ rest} \in S3\} \leq ?err'$
 if *seg*: *seg* $\in \text{tuples } (\text{UNIV} :: 'a \text{ set}) (2 * s + 1)$ for *seg*
 proof —
 from *seg* have *len*: *length seg = 2 * s + 1* by auto
 have *tkA*: *take (2 * s) (seg @ rest) = take (2 * s) seg* for *rest* :: 'a list
 using *len* by (simp add: take-append)
 have *tkB*: *take s (seg @ rest) = take s seg* for *rest* :: 'a list
 using *len* by (simp add: take-append)
 have *tkC*: *take s (drop s (seg @ rest)) = take s (drop s seg)* for *rest* :: 'a list
 using *len* by (simp add: drop-append take-append)
 have *nthA*: *(seg @ rest) ! (2 * s) = seg ! (2 * s)* for *rest* :: 'a list
 using *len* by (simp add: nth-append)
 have *tkF*: *take (2 * s + 1) (seg @ rest) = seg* for *rest* :: 'a list
 using *len* by simp
 have *drF*: *drop (2 * s + 1) (seg @ rest) = rest* for *rest* :: 'a list
 using *len* by simp
 have *stable*: *TF (seg @ rest) = TF seg* $\wedge$ *NV (seg @ rest) = NV seg* $\wedge$
 $NC (seg @ rest) = NC seg$ for *rest* :: 'a list
 by (simp add: TF-def NV-def NC-def tkA tkB tkC nthA len)
 show *?thesis*
 proof (cases *TF seg* $\wedge$ *NV seg* $\neq$ *layer-claim-sum L' (NC seg) b'*)
 case True
 have *sub*: $\{\text{rest. seg @ rest} \in S3\}$
 $\subseteq \{\text{rest. gkr-chain-bad } P (\lambda j \text{ pfx. } A (\text{Suc } j) (\text{seg @ pfx})) (s' \# ss)$
 $(L' \# Ls) (b' \# bs) (NC \text{ seg}) (NV \text{ seg}) \text{ rest}\}$
 by (auto simp add: S3-def stable tkF drF len)
 have *measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?M))*
 $\{\text{rest. seg @ rest} \in S3\}$
 $\leq \text{measure-pmf.prob } (\text{pmf-of-set } (\text{tuples } (\text{UNIV} :: 'a \text{ set}) ?M))$
 $\{\text{rest. gkr-chain-bad } P (\lambda j \text{ pfx. } A (\text{Suc } j) (\text{seg @ pfx})) (s' \# ss)$
 $(L' \# Ls) (b' \# bs) (NC \text{ seg}) (NV \text{ seg}) \text{ rest}\}$
 using *sub* by (intro prob-mono) auto
 also have $\dots \leq ?err'$
 using True by (intro 3.IH[OF wf]) simp
 finally show *?thesis* .

```

next
  case False
  then have  $\{rest. seg @ rest \in S3\} = \{\}$ 
    by (auto simp add: S3-def stable)
  then show ?thesis using err'-nonneg by simp
qed
qed
have qnz:  $?q \wedge (2 * s + 1) \neq 0$  using q-pos by simp
have sumb:  $(\sum seg \in tuples (UNIV :: 'a set) (2 * s + 1). measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?M)) \{rest. seg @ rest \in S3\}) \leq ?q \wedge (2 * s + 1) * ?err'$ 
proof -
  have  $(\sum seg \in tuples (UNIV :: 'a set) (2 * s + 1). measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?M)) \{rest. seg @ rest \in S3\}) \leq (\sum seg \in tuples (UNIV :: 'a set) (2 * s + 1). ?err')$ 
    by (rule sum-mono) (rule per)
  then show ?thesis by (simp add: card-tuples-UNIV)
qed
have qp3:  $0 \leq ?q \wedge (2 * s + 1)$  by (intro zero-le-power) simp
have  $measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ((2 * s + 1) + ?M))) S3 \leq (?q \wedge (2 * s + 1) * ?err') / ?q \wedge (2 * s + 1)$ 
  unfolding split1 by (rule divide-right-mono[OF sumb qp3])
also have  $\dots = ?err'$ 
  using qnz by simp
finally show ?thesis by (simp add: idx)
qed

— assemble
have u2:  $measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) (S1 \cup S2) \leq measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) S1 + measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) S2$ 
  by (rule measure-Un-le) simp-all
have  $measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) \{rs. gkr-chain-bad P A (s \# s' \# ss) (L \# L' \# Ls) (b \# b' \# bs) claims v rs\} \leq measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) (S1 \cup S2 \cup S3)$ 
  using subset by (intro prob-mono) auto
also have  $\dots \leq measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) (S1 \cup S2) + measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) S3$ 
  by (rule measure-Un-le) simp-all
also have  $\dots \leq (measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) S1 + measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) S2 + measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s \# s' \# ss)))) S3)$ 

```

```

      (chain-rlen (s # s' # ss)))) S3
  by (rule add-right-mono[OF u2])
  also have ... ≤ (real (2 * s) * real (dbnd s) / ?q + 1 / ?q) + ?err'
  by (intro add-mono p1 p2 p3)
  also have ... = (∑ t←s # s' # ss. real (2 * t) * real (dbnd t) + 1) / ?q
  by (simp add: add-divide-distrib)
  finally show ?case .
qed (auto simp add: gkr-chain-bad.simps measure-empty)

```

Theorem B, assembled (target name of the recorded B3 plan): if the claimed output table differs anywhere on the output cube from the true circuit output, then for ANY adversary the chain bad event - seeded by the verifier-computed claimed-output MLE at the random $z\text{-}0$ tuple (reduce.rs: $m0$), threaded through every layer reduction and carry - has probability at most

$$(s\text{-out} + \text{sum over layers of } (2 \text{ s-i dbnd} + 1)) / |'a|$$

over the flat verifier randomness $z\text{-}0$ @ *per-layer challenges*. The output-layer summand is the multilinear Schwartz-Zippel seed (*mle-agree-prob*); each layer contributes its sumcheck soundness plus one carry collision.

theorem *gkr-assembly-soundness*:

```

fixes P :: 'a layer ⇒ nat ⇒ ('a list × 'a) list ⇒ 'a list ⇒ 'p
  and A :: nat ⇒ 'a list ⇒ ('p, 'a, 'a, nat, 's) prover × 's × 'a × 'a × 'a
  and outs :: nat ⇒ 'a
  and dbnd :: nat ⇒ nat
assumes wf: gkr-chain-wf (s0 # ss) (L0 # Ls) (b0 # bs)
  and repr: ⋀ L s claims b. layer-poly-repr (P L s claims b) L s claims b (dbnd s)
  and outs-ne: ∃ idx < 2 ^ layer-width-bits L0. outs idx ≠ layer-eval L0 b0 ! idx
shows measure-pmf.prob
  (pmf-of-set (tuples (UNIV :: 'a set)
    (layer-width-bits L0 + chain-rlen (s0 # ss))))
  {rs. gkr-chain-bad P (λj pfx. A j (take (layer-width-bits L0) rs @ pfx))
    (s0 # ss) (L0 # Ls) (b0 # bs)
    [(take (layer-width-bits L0) rs, 1)]
    (mle (layer-width-bits L0) outs (take (layer-width-bits L0) rs)
      - const-mle L0 (take (layer-width-bits L0) rs))
    (drop (layer-width-bits L0) rs)}
  ≤ (real (layer-width-bits L0)
    + (∑ s←s0 # ss. real (2 * s) * real (dbnd s) + 1)) / real CARD('a)

```

proof –

```

define n0 where n0 = layer-width-bits L0
have outs-ne': ∃ idx < 2 ^ n0. outs idx ≠ layer-eval L0 b0 ! idx
  using outs-ne by (simp add: n0-def)
let ?q = real CARD('a)
let ?N = n0 + chain-rlen (s0 # ss)
let ?err = (∑ s←s0 # ss. real (2 * s) * real (dbnd s) + 1) / ?q
define ES where ES z0 ⟷ mle n0 outs z0 = mle n0 (!) (layer-eval L0 b0)) z0
  for z0 :: 'a list
define SEED where SEED = {rs :: 'a list. ES (take n0 rs)}
define REST where REST = {rs :: 'a list.
  mle n0 outs (take n0 rs) - const-mle L0 (take n0 rs)
  ≠ layer-claim-sum L0 [(take n0 rs, 1)] b0 ∧
  gkr-chain-bad P (λj pfx. A j (take n0 rs @ pfx)) (s0 # ss) (L0 # Ls) (b0 # bs)
}

```

```

    [(take n0 rs, 1)]
    (mle n0 outs (take n0 rs) - const-mle L0 (take n0 rs)) (drop n0 rs)}
have q-pos:  $0 < ?q$  by (simp add: card-gt-0-iff)
have err-nonneg:  $0 \leq ?err$ 
by (intro divide-nonneg-nonneg sum-list-nonneg) auto

have claim-sum1: layer-claim-sum L0 [(z0, 1)] b0
  = mle n0 (!) (layer-eval L0 b0) z0 - const-mle L0 z0 for z0 :: 'a list
by (simp add: layer-claim-sum-def n0-def)

have subset: {rs. gkr-chain-bad P ( $\lambda j$  pfx. A j (take n0 rs @ pfx))
  (s0 # ss) (L0 # Ls) (b0 # bs) [(take n0 rs, 1)]
  (mle n0 outs (take n0 rs) - const-mle L0 (take n0 rs)) (drop n0 rs)}
   $\subseteq SEED \cup REST$ 
proof
  fix rs :: 'a list
  assume rs: rs  $\in$  {rs. gkr-chain-bad P ( $\lambda j$  pfx. A j (take n0 rs @ pfx))
    (s0 # ss) (L0 # Ls) (b0 # bs) [(take n0 rs, 1)]
    (mle n0 outs (take n0 rs) - const-mle L0 (take n0 rs)) (drop n0 rs)}
  show rs  $\in SEED \cup REST$ 
  proof (cases mle n0 outs (take n0 rs) = mle n0 (!) (layer-eval L0 b0) (take n0 rs))
    case True
      then show ?thesis by (simp add: SEED-def ES-def)
    next
      case False
        then have mle n0 outs (take n0 rs) - const-mle L0 (take n0 rs)
           $\neq$  layer-claim-sum L0 [(take n0 rs, 1)] b0
          by (simp add: claim-sum1)
        then show ?thesis using rs by (auto simp add: REST-def)
  qed
qed

have pSEED: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?N)) SEED
   $\leq$  real n0 / ?q
proof -
  have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set)
    (n0 + chain-rlen (s0 # ss)))) SEED
    = measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) n0)) {z0. ES z0}
  unfolding SEED-def by (rule prob-tuples-prefix)
  also have ...  $\leq$  real n0 / ?q
  unfolding ES-def by (rule mle-agree-prob[OF outs-ne'])
  finally show ?thesis .
qed

have pREST: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?N)) REST  $\leq$  ?err
proof -
  have split1: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set)
    (n0 + chain-rlen (s0 # ss)))) REST
    = ( $\sum$  z0  $\in$  tuples (UNIV :: 'a set) n0.
      measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s0 # ss))))
      {rest. z0 @ rest  $\in$  REST})
    / ?q ^ n0
  using prob-tuples-split-seg[where E =  $\lambda$ zs. zs  $\in$  REST and k = n0]

```

```

    and m = chain-rlen (s0 # ss)] by simp
have per: measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s0 # ss))))
    {rest. z0 @ rest ∈ REST} ≤ ?err
if z0: z0 ∈ tuples (UNIV :: 'a set) n0 for z0
proof -
  from z0 have len: length z0 = n0 by auto
  have tk: take n0 (z0 @ rest) = z0 for rest :: 'a list
    using len by simp
  have dr: drop n0 (z0 @ rest) = rest for rest :: 'a list
    using len by simp
  show ?thesis
  proof (cases mle n0 outs z0 - const-mle L0 z0
    ≠ layer-claim-sum L0 [(z0, 1)] b0)
    case True
    have sub: {rest. z0 @ rest ∈ REST}
      ⊆ {rest. gkr-chain-bad P (λj pfx. A j (z0 @ pfx)) (s0 # ss)
        (L0 # Ls) (b0 # bs) [(z0, 1)]
        (mle n0 outs z0 - const-mle L0 z0) rest}
      by (auto simp add: REST-def tk dr len)
    have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set)
      (chain-rlen (s0 # ss)))) {rest. z0 @ rest ∈ REST}
      ≤ measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set)
      (chain-rlen (s0 # ss))))
        {rest. gkr-chain-bad P (λj pfx. A j (z0 @ pfx)) (s0 # ss)
        (L0 # Ls) (b0 # bs) [(z0, 1)]
        (mle n0 outs z0 - const-mle L0 z0) rest}
      using sub by (intro prob-mono) auto
    also have ... ≤ ?err
      using True by (intro gkr-chain-bad-bound[OF wf repr]) simp
    finally show ?thesis .
  next
    case False
    then have {rest. z0 @ rest ∈ REST} = {}
      by (auto simp add: REST-def tk len)
    then show ?thesis using err-nonneg by simp
  qed
qed
have qnz: ?q ^ n0 ≠ 0 using q-pos by simp
have sumb: (∑ z0 ∈ tuples (UNIV :: 'a set) n0.
  measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s0 # ss))))
  {rest. z0 @ rest ∈ REST})
  ≤ ?q ^ n0 * ?err
proof -
  have (∑ z0 ∈ tuples (UNIV :: 'a set) n0.
    measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) (chain-rlen (s0 # ss))))
    {rest. z0 @ rest ∈ REST})
    ≤ (∑ z0 ∈ tuples (UNIV :: 'a set) n0. ?err)
    by (rule sum-mono) (rule per)
  then show ?thesis by (simp add: card-tuples-UNIV)
qed
have qpn: 0 ≤ ?q ^ n0 by (intro zero-le-power) simp
have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set)
  (n0 + chain-rlen (s0 # ss)))) REST

```

```

    ≤ (?q ^ n0 * ?err) / ?q ^ n0
  unfolding split1 by (rule divide-right-mono[OF sumb qpn])
  also have ... = ?err
  using qnz by simp
  finally show ?thesis .
qed

have measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?N))
  {rs. gkr-chain-bad P (λj pfx. A j (take n0 rs @ pfx))
    (s0 # ss) (L0 # Ls) (b0 # bs) [(take n0 rs, 1)]
    (mle n0 outs (take n0 rs) - const-mle L0 (take n0 rs)) (drop n0 rs)}
  ≤ measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?N)) (SEED ∪ REST)
  using subset by (intro prob-mono) auto
also have ... ≤ measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?N)) SEED
  + measure-pmf.prob (pmf-of-set (tuples (UNIV :: 'a set) ?N)) REST
  by (rule measure-Un-le) simp-all
also have ... ≤ real n0 / ?q + ?err
  by (intro add-mono pSEED pREST)
also have ... = (real n0 + (∑ s←s0 # ss. real (2 * s) * real (dbnd s) + 1)) / ?q
  by (simp add: add-divide-distrib)
finally show ?thesis by (simp add: n0-def)
qed

corollary gkr-assembly-soundness-deg4:
  fixes P :: 'a layer ⇒ nat ⇒ ('a list × 'a) list ⇒ 'a list ⇒ 'p
  and A :: nat ⇒ 'a list ⇒ ('p, 'a, 'a, nat, 's) prover × 's × 'a × 'a × 'a
  and outs :: nat ⇒ 'a
  assumes gkr-chain-wf (s0 # ss) (L0 # Ls) (b0 # bs)
  and ∧ L s claims b. layer-poly-repr (P L s claims b) L s claims b 4
  and ∃ idx < 2 ^ layer-width-bits L0. outs idx ≠ layer-eval L0 b0 ! idx
  shows measure-pmf.prob
    (pmf-of-set (tuples (UNIV :: 'a set)
      (layer-width-bits L0 + chain-rlen (s0 # ss))))
    {rs. gkr-chain-bad P (λj pfx. A j (take (layer-width-bits L0) rs @ pfx))
      (s0 # ss) (L0 # Ls) (b0 # bs)
      [(take (layer-width-bits L0) rs, 1)]
      (mle (layer-width-bits L0) outs (take (layer-width-bits L0) rs)
        - const-mle L0 (take (layer-width-bits L0) rs))
      (drop (layer-width-bits L0) rs)}
    ≤ (real (layer-width-bits L0)
      + (∑ s←s0 # ss. real (2 * s) * 4 + 1)) / real CARD('a)
  proof -
    have r: ∧ L s claims b. layer-poly-repr (P L s claims b) L s claims b ((λ-. 4) s)
    using assms(2) by simp
    show ?thesis
    using gkr-assembly-soundness[OF assms(1) r assms(3)] by simp
  qed
end

end

end

```

```

theory GKR-Batching
  imports Main
begin

```

44 Abstract single-proof pipeline

The batching crate is generic: it neither inspects proofs nor knows the GKR structure (the actual proving is injected as a closure). We model the single-proof pipeline as an abstract proving function and an abstract verification predicate. No locale assumptions are needed: determinism of both maps is automatic because HOL functions are functions, and Theorem D needs nothing else.

```

locale gkr-batching =
  fixes prove :: 'op  $\Rightarrow$  'proof
  and verify :: 'op  $\Rightarrow$  'proof  $\Rightarrow$  bool
begin

```

The v0.1 batch prover: one proof per operation, in queue order ('BatchResult.proofs' aligned with the drained jobs).

```

definition batch-prove :: 'op list  $\Rightarrow$  'proof list where
  batch-prove ops = map prove ops

```

Batch verification: the proof list is aligned with the operation list, and every position verifies individually. The length conjunct is the verifier-side counterpart of the alignment contract of 'BatchResult'.

```

definition batch-verify :: 'op list  $\Rightarrow$  'proof list  $\Rightarrow$  bool where
  batch-verify ops proofs  $\longleftrightarrow$ 
    length proofs = length ops  $\wedge$ 
    ( $\forall i < \text{length ops. verify (ops ! i) (proofs ! i)$ )

```

44.1 Structural lemmas (cache harmlessness, formally)

```

lemma batch-prove-length [simp]:
  length (batch-prove ops) = length ops
by (simp add: batch-prove-def)

```

Position i of a batch carries the proof of operation i and of nothing else: batched proving is element-wise the single-proof path.

```

lemma batch-prove-nth [simp]:
   $i < \text{length ops} \implies \text{batch-prove ops ! } i = \text{prove (ops ! } i)$ 
by (simp add: batch-prove-def)

```

Batch-context independence: an operation's proof does not depend on the jobs batched before or after it (so the scheduler's choice of batch boundaries cannot alter any proof).

```

lemma batch-prove-context-independent:
  batch-prove (pre @ op # post) ! length pre = prove op
by (simp add: batch-prove-def nth-append)

```

Two occurrences of the same operation receive the same proof - the observable consequence of sharing one compiled circuit per kind.

lemma *batch-prove-same-op-same-proof*:
 $\llbracket i < \text{length ops}; j < \text{length ops}; \text{ops} ! i = \text{ops} ! j \rrbracket$
 $\implies \text{batch-prove ops} ! i = \text{batch-prove ops} ! j$
by *simp*

44.2 Interface lemmas for batch verification

lemma *batch-verify-length*:
 $\text{batch-verify ops proofs} \implies \text{length proofs} = \text{length ops}$
by (*simp add: batch-verify-def*)

A verified batch verifies at every position (the consumption interface for downstream soundness composition: any single claim can be extracted from a verified batch).

lemma *batch-verify-imp-each*:
 $\llbracket \text{batch-verify ops proofs}; i < \text{length ops} \rrbracket$
 $\implies \text{verify (ops} ! i) (\text{proofs} ! i)$
by (*simp add: batch-verify-def*)

Equivalent pointwise characterisation via *list-all2*.

lemma *batch-verify-conv-list-all2*:
 $\text{batch-verify ops proofs} \longleftrightarrow \text{list-all2 verify ops proofs}$
by (*auto simp: batch-verify-def list-all2-conv-all-nth*)

44.3 Theorem D - batching correctness

(i) A batch of honestly produced proofs verifies iff every operation verifies under the single-proof pipeline. Note the right-hand side ranges over the SET of operations: since proving is a function of the operation alone, duplicates cannot make a batch behave differently from its underlying set of claims.

theorem *theorem-D-batching-correctness*:
 $\text{batch-verify ops} (\text{batch-prove ops}) \longleftrightarrow$
 $(\forall op \in \text{set ops. verify op (prove op)})$
by (*simp add: batch-verify-def all-set-conv-all-nth*)

(ii) Individual result preservation: the verification outcome of the i -th batched proof coincides with the outcome the single-proof pipeline delivers for that operation - acceptances and rejections are preserved position by position, whatever the rest of the batch does.

theorem *theorem-D-individual-preservation*:
assumes $i < \text{length ops}$
shows $\text{verify (ops} ! i) (\text{batch-prove ops} ! i) \longleftrightarrow$
 $\text{verify (ops} ! i) (\text{prove (ops} ! i))$
using *assms* **by** *simp*

corollary *theorem-D-accept-preservation*:
 $\llbracket i < \text{length ops}; \text{verify (ops} ! i) (\text{prove (ops} ! i)) \rrbracket$
 $\implies \text{verify (ops} ! i) (\text{batch-prove ops} ! i)$
by *simp*

corollary *theorem-D-reject-preservation*:
 $\llbracket i < \text{length ops}; \neg \text{verify (ops} ! i) (\text{prove (ops} ! i)) \rrbracket$
 $\implies \neg \text{verify (ops} ! i) (\text{batch-prove ops} ! i)$

by *simp*

(iii) Compile-cache harmlessness is not a separate proposition in this model but a consequence of its shape: *batch-prove* IS *map prove* (definitionally), so reusing one compiled circuit across a batch cannot be observed by any verifier. The lemmas *batch-prove-nth*, *batch-prove-context-independent* and *batch-prove-same-op-same-proof* state the observable content: each proof is determined by its own operation only.

end

45 Parameterised batch pipe - the re-instantiation surface

v0.1 proves per operation; a v0.2 amortized batch prover may compute the proof list differently (shared preprocessing, reordered evaluation, fused transcripts...). To keep Theorem D reusable WITHOUT rework, the batch pipe itself is a locale parameter: any pipe that (a) preserves length and (b) delivers at every position the proof of the single-proof path inherits all of Theorem D by a single interpretation. This mirrors the Rust structure, where the actual proving is INJECTED into *BatchProver::prove-round* as the *prove-batch* closure; the two locale assumptions are precisely the contract that closure must meet.

locale *batch-pipeline* = *gkr-batching* *prove* *verify*
 for *prove* :: 'op $\Rightarrow$ 'proof and *verify* :: 'op $\Rightarrow$ 'proof $\Rightarrow$ bool +
 fixes *batch-pipe* :: 'op list $\Rightarrow$ 'proof list
 assumes *batch-pipe-length*: *length* (*batch-pipe ops*) = *length ops*
 and *batch-pipe-nth*: *i* < *length ops* $\implies$ *batch-pipe ops* ! *i* = *prove* (*ops* ! *i*)
begin

Any conforming pipe is extensionally the v0.1 pipe.

lemma *batch-pipe-eq-batch-prove*:
batch-pipe ops = *batch-prove ops*
by (*rule nth-equalityI*) (*simp-all add: batch-pipe-length batch-pipe-nth*)

theorem *theorem-D-pipe-batching-correctness*:
batch-verify ops (*batch-pipe ops*) $\longleftrightarrow$
 ($\forall$ *op* $\in$ *set ops*. *verify op* (*prove op*))
by (*simp add: batch-pipe-eq-batch-prove theorem-D-batching-correctness*)

theorem *theorem-D-pipe-individual-preservation*:
 assumes *i* < *length ops*
 shows *verify* (*ops* ! *i*) (*batch-pipe ops* ! *i*) $\longleftrightarrow$
verify (*ops* ! *i*) (*prove* (*ops* ! *i*))
using *assms* **by** (*simp add: batch-pipe-nth*)

end

The v0.1 batch prover is itself an instance of the parameterised surface: ‘map prove’ discharges both pipe obligations definitionally. Registered as a sublocale, every interpretation of *gkr-batching* automatically carries the pipe-level theorems for the v0.1 pipe (qualifier *v01*).

sublocale *gkr-batching* $\subseteq$ *v01*: *batch-pipeline* *prove* *verify* *batch-prove*
by *unfold-locales simp-all*

46 Activation instance ($N = 2$, accept/reject mixed)

A concrete, non-vacuous instance: proofs are squares, and the verifier additionally rejects the operation 0 , so honest proving does NOT imply acceptance - both outcomes occur. The batch $[3, 0]$ exercises Theorem D on a mixed batch: position 0 accepts exactly as a single proof, position 1 rejects exactly as a single proof, and the whole-batch conjunction rejects.

definition *demo-prove* :: $\text{nat} \Rightarrow \text{nat}$ **where**
demo-prove $n = n * n$

definition *demo-verify* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{bool}$ **where**
demo-verify $n\ p \longleftrightarrow p = n * n \wedge 0 < n$

interpretation *demo*: *gkr-batching demo-prove demo-verify*
by *unfold-locales*

Single-proof pipeline: acceptance and rejection both inhabited.

lemma *demo-accepts-3*: *demo-verify* 3 (*demo-prove* 3)
by (*simp add: demo-verify-def demo-prove-def*)

lemma *demo-rejects-0*: $\neg$ *demo-verify* 0 (*demo-prove* 0)
by (*simp add: demo-verify-def demo-prove-def*)

lemma *demo-rejects-wrong-proof*: $\neg$ *demo-verify* 3 10
by (*simp add: demo-verify-def*)

The mixed batch $[3, 0]$: concrete proof vector.

lemma *demo-batch-proofs*: *demo.batch-prove* $[3, 0] = [9, 0]$
by (*simp add: demo.batch-prove-def demo-prove-def*)

Position-wise preservation activates on both outcomes.

lemma *demo-mixed-position-0-accepts*:
demo-verify 3 (*demo.batch-prove* $[3, 0] ! 0$)
by (*simp add: demo-batch-proofs demo-verify-def*)

lemma *demo-mixed-position-1-rejects*:
 $\neg$ *demo-verify* 0 (*demo.batch-prove* $[3, 0] ! 1$)
by (*simp add: demo-batch-proofs demo-verify-def*)

The whole mixed batch rejects (one bad claim suffices) ...

lemma *demo-mixed-batch-rejects*:
 $\neg$ *demo.batch-verify* $[3, 0]$ (*demo.batch-prove* $[3, 0]$)
by (*simp add: demo.theorem-D-batching-correctness demo-verify-def demo-prove-def*)

... while an all-honest, all-valid batch accepts.

lemma *demo-all-accepting-batch*:
demo.batch-verify $[3, 5]$ (*demo.batch-prove* $[3, 5]$)
by (*simp add: demo.theorem-D-batching-correctness demo-verify-def demo-prove-def*)

Batch verification is not vacuous on malformed inputs either.

lemma *demo-length-mismatch-rejects*:

```

  ¬ demo.batch-verify [3] []
  by (simp add: demo.batch-verify-def)

```

```

lemma demo-forged-batch-rejects:
  ¬ demo.batch-verify [3, 5] [9, 26]
  by (simp add: demo.batch-verify-conv-list-all2 demo-verify-def)

```

46.1 Re-instantiation demo (the v0.2 path)

A pipe that processes the batch in REVERSE order and restores order afterwards is extensionally the honest pipe, so it inherits Theorem D through the two pointwise obligations alone. This is the no-rework path a v0.2 amortized prover takes: discharge (a) length preservation and (b) pointwise agreement with the single-proof path, inherit every theorem of *batch-pipeline*.

```

definition demo-rev-pipe :: nat list ⇒ nat list where
  demo-rev-pipe ops = rev (map demo-prove (rev ops))

```

```

lemma demo-rev-pipe-eq-map:
  demo-rev-pipe ops = map demo-prove ops
  by (simp add: demo-rev-pipe-def rev-map)

```

```

interpretation demo-rev: batch-pipeline demo-prove demo-verify demo-rev-pipe
  by unfold-locales (simp-all add: demo-rev-pipe-eq-map)

```

```

lemma demo-rev-pipe-mixed-batch-rejects:
  ¬ demo.batch-verify [3, 0] (demo-rev-pipe [3, 0])
  by (simp add: demo-rev.theorem-D-pipe-batching-correctness
    demo-verify-def demo-prove-def)

```

```

lemma demo-rev-pipe-position-0-accepts:
  demo-verify 3 (demo-rev-pipe [3, 0] ! 0)
  by (simp add: demo-rev-pipe-eq-map demo-verify-def demo-prove-def)

```

47 The amortized batch pipe (v0.2) - shared prepared state

The v0.2 throughput batch prover amortizes the per-batch setup - circuit compilation and the verifier's wiring derivation - into a PREPARED state built once per operation kind (crate root: *PreparedSync*, *prove-batch-prepared*, *prove-batch-parallel*; the batch semantics decision record lives in the repository under *docs/adr/*), and may evaluate the per-job proofs on any order-restoring parallel schedule.

The model content of that design is a single assumption: proving against the shared prepared state IS the single-proof pipeline, job by job (*prove-with prep = prove*). The executable counterpart is pinned bit-for-bit by the agreement tests in *tests/batch-v02.rs* (prepared and parallel proofs equal the single-path proofs across operation kinds, batch sizes and worker counts). Everything else is inherited: both v0.2 pipes discharge the two *batch-pipeline* obligations and carry all of Theorem D without re-proving anything.

```

locale gkr-batching-v02 = gkr-batching prove verify
  for prove :: 'op ⇒ 'proof and verify :: 'op ⇒ 'proof ⇒ bool +
  fixes prep :: 'prep
  and prove-with :: 'prep ⇒ 'op ⇒ 'proof

```

assumes *prove-with-agrees*: *prove-with prep op = prove op*
begin

The amortized sequential pipe (*prove-batch-prepared*): one shared prepared state, one proof per job, queue order preserved.

definition *prepared-pipe* :: 'op list $\Rightarrow$ 'proof list **where**
prepared-pipe ops = map (*prove-with prep*) ops

The parallel pipe (*prove-batch-parallel*): workers evaluate jobs independently and results are collected back IN INPUT ORDER. The index-driven form below mirrors that order-restoring collection. Scheduling - which worker computes which index, in what real-time order - has no counterpart in the function model, and that absence IS the determinism argument: each position's value is a function of its own operation alone, so worker count cannot reach any proof.

definition *parallel-pipe* :: 'op list $\Rightarrow$ 'proof list **where**
parallel-pipe ops = map ($\lambda i. \text{prove-with prep } (ops ! i)$) [0..*length ops*]

lemma *prepared-pipe-eq-map*: *prepared-pipe ops* = map *prove ops*
by (*simp add: prepared-pipe-def prove-with-agrees*)

lemma *parallel-pipe-eq-map*: *parallel-pipe ops* = map *prove ops*
proof –

have map ($\lambda i. \text{prove-with prep } (ops ! i)$) [0..*length ops*]
= map (*prove-with prep*) (map (!) ops) [0..*length ops*]
by (*simp add: map-map o-def*)
also have ... = map (*prove-with prep*) ops
by (*simp add: map-nth*)
finally show ?thesis
by (*simp add: parallel-pipe-def prove-with-agrees*)

qed

Both pipes re-instantiate the parameterised surface: the two pointwise obligations discharge from the agreement assumption alone, and every theorem of *batch-pipeline* follows for the amortized prover.

sublocale *prepared*: *batch-pipeline prove verify prepared-pipe*
by *unfold-locales (simp-all add: prepared-pipe-eq-map)*

sublocale *parallel*: *batch-pipeline prove verify parallel-pipe*
by *unfold-locales (simp-all add: parallel-pipe-eq-map)*

end

47.1 Activation instance (v0.2 pipes on the mixed batch)

Non-vacuity witness: the demo pipeline with an explicit (trivial) prepared state. The agreement assumption discharges by computation, and Theorem D activates on the same mixed batch as above through BOTH v0.2 pipes - rejection at a bad position, acceptance of an all-valid batch.

definition *demo-prove-with* :: unit $\Rightarrow$ nat $\Rightarrow$ nat **where**
demo-prove-with s n = n * n

interpretation *demo-v02*: *gkr-batching-v02* *demo-prove* *demo-verify* () *demo-prove-with*
by *unfold-locales* (*simp* *add*: *demo-prove-with-def* *demo-prove-def*)

lemma *demo-v02-prepared-mixed-batch-rejects*:
 $\neg$ *demo.batch-verify* [3, 0] (*demo-v02.prepared-pipe* [3, 0])
by (*simp* *add*: *demo-v02.prepared.theorem-D-pipe-batching-correctness*
demo-verify-def *demo-prove-def*)

lemma *demo-v02-parallel-position-0-accepts*:
demo-verify 3 (*demo-v02.parallel-pipe* [3, 0] ! 0)
by (*simp* *add*: *demo-v02.parallel-pipe-eq-map* *demo-verify-def* *demo-prove-def*)

lemma *demo-v02-parallel-all-valid-batch-accepts*:
demo.batch-verify [3, 5] (*demo-v02.parallel-pipe* [3, 5])
by (*simp* *add*: *demo-v02.parallel.theorem-D-pipe-batching-correctness*
demo-verify-def *demo-prove-def*)

end

References

- [1] A. G. Bosshard, C. Sprenger, and J. Bootle. The sumcheck protocol. *Archive of Formal Proofs*, February 2024. https://isa-afp.org/entries/Sumcheck_Protocol.html, Formal proof development.
- [2] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum. Delegating computation: Interactive proofs for muggles. *Journal of the ACM*, 62(4):27:1–27:64, 2015.
- [3] J. Thaler. Proofs, arguments, and zero-knowledge. *Foundations and Trends in Privacy and Security*, 4(2–4):117–660, 2022.
- [4] Verified zkEVM project. ArkLib: Formally verified arguments of knowledge in Lean. <https://github.com/Verified-zkEVM/ArkLib>, 2025.